noted

ON-PREMISES-MLOPS COCKPIT

iscte
UNIVERSITY INSTITUTE OF LISBON

NOTEBOOK COMPANION

Table of Contents

1. Introduction

1.1 Document Information

Field	Value
Document	noted Developer Manual
Project	noted - Integrated MLOps Platform
Version	1.0
Status	Complete
Audience	Developers and engineers working on noted

1.2 Purpose

This manual is the canonical technical reference for engineers building, operating, and extending noted. It is for developers joining or returning to the project who need to understand how the MLOps integrations work in practice and how the platform's main components are wired together. The manual is organised in two parts:

- **Chapters 2-6** use the Tutorial 3 notebook as a worked example to cover Hydra, MLflow, DVC, and Evidently in depth, ending with end-to-end scenarios that combine all four.
- **Chapters 7-15** walk through every main component of the noted platform itself, in dependency order: frontend, backend, notebook execution, Configuration Composer + Time Machine, model serving, AI assistant + MCP, knowledge graph, multi-language runtime, and infrastructure.

Each chapter follows the same five-section structure so the manual is uniform to read and to extend:

1. **Concept primer** - the framework or component on its own terms, no noted-specific assumptions.
2. **Where it lives in the code** - exact file paths, line numbers, and (for the notebook chapters) cell numbers.
3. **How noted bridges to it** - the specific abstractions, hooks, and contracts that connect the concept to the rest of the platform.
4. **Operations** - what a developer actually does with it (add, modify, debug, extend).
5. **Discussion-ready talking points** - 5-10 questions with concise, defensible answers. A knowledge anchor for future readers.

1.3 Conventions

- **Cell references** use the form cell N and refer to the indexed cells in notebooks/emi_tutorial3_jena_weather.ipynb.

- **Code references** use the form path/to/file.py:LINE so they are clickable in noted's editor and in modern Markdown viewers.
- **Tool references** are spelled with their canonical capitalisation (Hydra, MLflow, DVC, Evidently, Airflow, MinIO).
- **Abbreviations**: cfg for the Hydra-resolved DictConfig injected into the notebook kernel; MLOps for the union of versioning, tracking, configuration, orchestration, and serving disciplines.

1.4 Out of scope

- Cluster deployment topology beyond the local Docker Compose stack.
- Performance tuning beyond the few notes in the relevant chapters.
- Security model deep-dive (handled separately in the User Manual's setup pages).
- Curriculum-level theory on time-series forecasting; this manual stays on engineering.

2. Hydra

2.1 Concept primer

Hydra is a configuration framework for Python applications. Its value for MLOps is that it separates *what* a run depends on (the resolved configuration) from *how* that configuration was assembled (a stack of YAML fragments and CLI-style overrides). The framework is deceptively small - three ideas do most of the work:

1. **The defaults list.** A top-level config.yaml declares a defaults: list that points at *groups*. Each group is a directory, each option inside it a YAML file. data: jena_full_dataset selects config/data/jena_full_dataset.yaml. At compose time Hydra recursively merges each selected group's contents into a single tree. Changing data: jena_2012_dataset swaps one YAML file, the merge result changes, and nothing else in the calling code has to move.
2. **Group composition as a first-class operation.** Composition is not string substitution: groups can override each other, can set _self_ to anchor their position, and can be swapped entirely at invocation time with +group=option or group=option syntax. The same config.yaml can produce radically different resolved configs just by moving options in the defaults list.
3. **Overrides as fine-grained edits.** After the defaults are resolved, Hydra applies a list of key.path=value overrides. training.epochs=10 rewrites one leaf without touching anything around it. Overrides are ephemeral unless persisted - they live in the invocation, not in the YAML.

The resolved configuration is an OmegaConf.DictConfig - a nested dict with attribute access (cfg.data.file), type coercion, interpolation (${other.path}), and strict mode that catches typos at access time instead of at .get() time. When noted injects cfg into a running kernel it uses the same OmegaConf.create(...) call, so a notebook cell reading cfg.training.epochs cannot tell whether Hydra was invoked from the CLI or composed inside the backend and shipped over ZMQ.

Why "templates not config" matters. A critical mental model: the YAML files on disk are not the configuration. They are templates that *produce* a configuration when composed with a particular defaults list and override set. Two runs that read the same config.yaml can produce entirely different resolved configs if their defaults list picked different groups or their overrides differ. This is the whole reason Hydra exists and the reason every reproducibility story in noted ends with "...and then we hash the resolved config, not the templates."

Hashing as a reproducibility primitive. noted computes sha256(resolved_yaml) at compose time and stores it under __noted_hydra_hash__ in the kernel and under noted.hydra_config_hash as an MLflow tag. Two runs with byte-identical hashes are guaranteed to have seen byte-identical configs, regardless of which templates were edited between them. Two runs with different hashes cannot be casually compared - the hash is

the proof-of-identity for a configuration, and the Composer's baseline badge (Section 2.3) is the UI affordance that surfaces this fact to users who would otherwise not know their config had drifted.

What Hydra does not do. It does not orchestrate, track, or version. It has opinions about configuration and no opinions about anything else. In noted, Hydra is the gatekeeper for "what was this run told to do?"; MLflow is the gatekeeper for "what did this run actually do?"; and the two meet at noted.hydra_config_hash on each MLflow run. Keep these lanes straight and the rest of the integration stops being confusing.

2.2 Where Hydra lives in the notebook

The Tutorial 3 notebook (notebooks/emi_tutorial3_jena_weather.ipynb in the jena_weather project) is the worked example. It never imports hydra directly - cfg is injected by noted before any cell runs. Every cell that reads from cfg is therefore a consumer of the composed configuration, and every such access can be traced back to a specific YAML group or a specific override input in the Composer.

The notebook's configuration tree at v0.1 of this manual lives at config/ in the project root:

```
config/
  config.yaml                    <- defaults list + inlined training block
  data/
    jena_2012_dataset.yaml      <- one-year subset (DVC md5 d3956bd0...)
    jena_full_dataset.yaml      <- full 2009-2016
  model/
    gru_baseline.yaml
    gru_evolutionary.yaml
  scaler/
    minmax.yaml
    robust.yaml
    standard.yaml
```

config.yaml declares defaults: [data: jena_full_dataset, model: gru_baseline, scaler: standard] and inlines the entire training subtree (epochs, batch_size, learning_rate, clipnorm, early_stopping., *lr_reduction*.). There is no training/ group directory - training is inlined so every training knob surfaces as an override input in the Composer without forcing users to pick a "training preset" they don't care about.

Cell-by-cell consumer map:

Cell	cfg accesses	Purpose
11	cfg.seed	Seeds numpy, tf, random; replaces the old hardcoded SEED = 42
16	cfg.data.file	Resolves DATASET_PATH = PROJECT_ROOT / cfg.data.file - this is the load-bearing line that makes Composer data-switching actually switch the CSV

Cell	cfg accesses	Purpose
41	cfg.data.features	Column subset for training
44	cfg.data.target	Regression target name
59	cfg.data.split.train, cfg.data.split.val	Time-ordered split ratios
63	cfg.scaler.name	Selects scaler class from the scaler group
70	cfg.data.lookback, cfg.data.horizon	Window dimensions for the sliding-window generator
91	cfg.model.*, cfg.training.*, cfg.scaler.name	build_model_from_cfg() - entire model factory reads from cfg
94	cfg.training.early_stopping, cfg.training.lr_reduction	Passed to train_model(...) as es_cfg=..., lr_cfg=...
95	cfg.training.epochs, cfg.training.lr_reduction.factor, cfg.training.early_stopping.patience	Markdown cell referencing active values (rendered at execution time)
116	(reads none directly; logs run params)	MLflow log_params pulls from cfg indirectly

Cell 11 and cell 16 are the two most important. A previous revision of the notebook was not actually cfg-driven despite looking like it: SEED = 42 was hardcoded and DATASET_PATH pointed at a literal filename. Composer selections influenced training hyperparameters but not the two things users would most obviously want to control - seed and dataset. This is the kind of subtle regression that reproducibility primitives are supposed to catch, and the reason noted's hash-badge + MLflow tag design matters in practice.

2.3 How noted bridges to Hydra

noted's Hydra integration is a four-layer stack: a **backend composition engine**, a **kernel injection pipeline**, a **notebook metadata contract**, and a **Composer UI** with time-travel support. Nothing in Hydra itself is modified - noted composes by calling OmegaConf and by walking YAML files directly rather than using hydra.main, because the framework's decorator model assumes a CLI entry point and noted's entry point is a notebook kernel.

2.3.1 The composition engine: HydraManager + HydraSource + HydraCache

HydraManager (backend/app/managers/hydra_manager.py:23) is the orchestration layer. Its core methods:

- get_schema_from_source(source) [hydra_manager.py:147] - walks a source tree, discovers groups and options, returns {groups, schema, defaults, baseline_source}.

The schema enumerates every leaf in the resolved config.yaml so the Composer knows which inputs to render as override fields.

- compose_from_source(source, group_selections, overrides) [hydra_manager.py:318] - merges config.yaml with the selected group files, applies overrides, and returns {resolved, yaml, hash, sources}. The hash is sha256(yaml) and becomes the primary key for comparing two runs.
- assemble_bundle_from_source(source, ...) [hydra_manager.py:57] - produces the per-run archive: a flat dict[str, bytes] containing the full config/ tree plus selections.json (the group choices and override values that produced this resolved config) plus resolved.yaml (the composed output). This is what gets uploaded to MLflow.

HydraSource (backend/app/managers/hydra_source.py:27) is an abstraction over "where does the config tree live?". Two concrete implementations:

- LocalSource [hydra_source.py:56] reads from the project's on-disk config/ directory. Used when the baseline is project://config/.
- MlflowSource [hydra_source.py:113] reads from a past run's archived bundle. Used when the baseline is mlflow://<run_id>. It lazily fetches the bundle via HydraCache the first time a read is requested.

Both implement the same exists(), read_text(), walk() contract (file-system shaped), so HydraManager is source-agnostic. This is what makes Time Machine possible: the exact same composition code path resolves a config whether it comes from the working tree or from MLflow.

HydraCache (backend/app/managers/hydra_cache.py:27) is an in-memory LRU keyed by (notebook_uid, run_id) with MAX_ENTRIES=500 and FIFO eviction. The fetch_from_mlflow() method [hydra_cache.py:64] downloads the hydra/ artifact tree from a run on first access, flattens it into a dict[str, bytes], and returns it. Subsequent loads of the same run are cache hits. The cache is ephemeral - it has no disk backing and is rebuilt on restart - which is deliberate: the archived bundles are the ground truth in MLflow, the cache is a performance optimization.

2.3.2 Kernel injection: _build_hydra_injection

Before any cell executes, noted runs a short Python prelude in the kernel to make cfg and its hash available as module-level variables. ExecutionBridge._build_hydra_injection() (backend/app/managers/execution_bridge.py:877) is called both on single-cell execute (line 149-164) and on run-start (line 327-332).

The injected code (simplified):

```
import json as _json
__noted_hydra_config__ = _json.loads('<resolved_json>')
```

```
__noted_hydra_hash__ = 'sha256:<hash>'
try:
    from omegaconf import OmegaConf as _OC
    cfg = _OC.create(__noted_hydra_config__)
except Exception:
    cfg = __noted_hydra_config__  # dict fallback
```

The hydra_config dict passed into this call carries four fields that together encode the run's configuration identity: notebook_uid, baseline_source, group_selections, overrides. The backend composes the resolved tree on the fly for every injection - it does not trust a cached resolved value because overrides can change per invocation.

Execution of this prelude is via _execute_silent() (execution_bridge.py:411), which waits for the kernel's shell reply before returning control. The cell never sees the prelude output; only cfg and __noted_hydra_hash__ remain in the user namespace.

This is the tech-debt-flagged "invisible prelude" the notebook depends on. It makes the notebook fail to run outside noted unless the user writes their own cfg = compose(...) cell. That trade-off is tracked in the backlog for future cockpit design.

2.3.3 Notebook metadata contract

Two keys are persisted under notebook.metadata.noted:

- notebook_uid - a UUID generated lazily on the frontend the first time the user clicks Apply in the Composer (frontend/js/NotebookEditor.js:2333). Stable for the life of the notebook. This is what the HydraCache uses as part of its cache key, and what the backend uses to detect "is this notebook Hydra-using?".
- hydra_selections - a nested {group_selections: {data: "...", model: "..."}, overrides: {"seed": 42, "training.epochs": 10}} dict persisted on every Apply (NotebookEditor.js:2353-2362).
- hydra_baseline_source - either project://config/ (the default) or mlflow://<run_id> (set when the user switches to Experiment Run mode and applies a past run as the baseline).

Writing happens in the frontend; persistence goes through backend/app/routers/notebooks.py:101 which calls NotebookManager.update_notebook() to serialize the .ipynb file. The round-trip is sync per Apply click, not batched.

2.3.4 Per-run bundle archival

When a run begins executing, the Run Manager prelude emits a display_data message with the mime type application/x-noted-run-start. ExecutionBridge watches for this (line 599-613) and, on first detection, calls _log_hydra_bundle_for_run() (execution_bridge.py:695) in a fire-and-forget thread.

That function:

1. Re-composes the config from the notebook's current hydra_selections and baseline source.
2. Calls HydraManager.assemble_bundle_from_source(...) to produce the flat bundle.
3. Writes the bundle to a tempdir.
4. Uploads via client.log_artifacts(run_id, tmpdir, artifact_path="hydra").
5. Tags the run with noted.hydra_config_hash=<sha256>, noted.project_id=<id>, and (if in a git repo) noted.git_commit + mlflow.source.git.branch.

Failures are logged and ignored - bundle archival is additive metadata, not a correctness prerequisite. Deduplication per-session prevents double-logging if the prelude fires twice.

The archived tree on MLflow looks like:

```
hydra/
  config.yaml
  data/jena_2012_dataset.yaml
  data/jena_full_dataset.yaml
  model/gru_baseline.yaml
  ...
  selections.json          <- {group_selections, overrides}
  resolved.yaml            <- composed output
```

This is what MlflowSource reads back when a user selects that run in Time Machine.

2.3.5 Composer panel and Time Machine

The Composer (frontend/js/panels/explorer/ExplorerHydraViews.js) is a jsPanel tab with:

- A **mode toggle** between Local (read from project://config/) and Experiment Run (read from mlflow://<run_id>).
- Three **group dropdowns** (data/model/scaler) populated from the active schema.
- About **ten override inputs** for seed and the inlined training.* keys.
- An **Apply button** that calls NotebookEditor.setHydraSelections(...) which in turn persists metadata and triggers schema refresh + badge recomputation.

The four Composer-to-backend endpoints live at backend/app/routers/hydra.py:

- GET /api/hydra/experiments/{project_id} [line 182] - lists experiments that have at least one run tagged with the project id.
- GET /api/hydra/runs/{project_id}/{experiment_id} [line 233] - lists runs that have a hydra/ artifact bundle.
- POST /api/hydra/compose-mlflow [line 277] - compose using an MLflow run as baseline plus user tweaks.

- POST /api/hydra/load-bundle [line 307] - fetch a past run's bundle into the cache and return the selections/overrides that produced it (so the Composer can pre-populate).

2.3.6 Baseline badge state machine

The badge in the notebook bar (NotebookEditor.js:2106) shows one of three labels paired with one of three state dots:

- Label: BASELINE (gray) when hydra_baseline_source is project://config/.
- Label: RUN xxxxxx (purple, short hash) when it is mlflow://<run_id>.
- Dot: green check when current selections match defaults (Local) or match the archived bundle (MLflow). Orange exclamation when they differ (drift). Red X when the baseline source is unreachable (e.g. MLflow run deleted).

Drift is computed by _computeBaselineBadgeState() (NotebookEditor.js:2166) and _selectionsEqual() (line 2250). The tooltip includes a Drift: section listing the exact keys that differ - this was added after a real incident where a user had legacy flat-format metadata with stale group names, and the badge was going orange with no diagnostic surface.

_refreshActiveSchema() (line 2399) fetches /api/hydra/schema/{project_id} against the current baseline source and is called after every Apply to keep the schema used for drift comparison in sync with the baseline.

2.4 Operations

Add a new group

1. Create the directory under config/, e.g. config/optimizer/.
2. Add one YAML option file per choice, e.g. config/optimizer/adam.yaml, config/optimizer/adamw.yaml.
3. Add optimizer: adam to the defaults: list in config.yaml.
4. Reopen the Composer in noted - the new group appears as a dropdown automatically. HydraManager.get_schema_from_source() rediscovers groups on every render.

Add a new override input

Anything that is a leaf in config.yaml (not referenced as a group) automatically surfaces as an override input. If training is inlined (as it is in Tutorial 3), adding training.optimizer_beta_1: 0.9 to config.yaml produces a new input in the Composer on next render. If training were a group, you would need to edit each training option file.

There is a known gap: group-file leaves are not currently exposed as override inputs. If a leaf only lives in data/jena_full_dataset.yaml and not in config.yaml, it cannot be

overridden via the Composer UI. The workaround for now is to inline the value in config.yaml. Extending _extract_schema to walk into selected group files is queued for future work.

Debug a hash mismatch

Symptom: two runs that should be “the same” produce different noted.hydra_config_hash tags, or the badge shows orange after you thought you only changed a defaults list alignment.

1. Open both runs in MLflow and download the hydra/selections.json artifact from each. A diff of selections reveals any change in group selection or override values.
2. If selections match, diff hydra/resolved.yaml. Any byte difference changes the hash. Whitespace and key ordering are preserved by OmegaConf’s dumper and do count as differences if templates were edited in between.
3. If resolved YAMLs match, suspect different Hydra library versions (unlikely in noted’s pinned stack) or a silent edit to a template file that happened to compose to the same leaves but a different byte layout.

The badge’s drift tooltip (_computeBaselineBadgeState) lists keys that differ from the baseline, which is usually enough to find the culprit without opening MLflow.

Load a past run’s config back into the notebook

1. Open the Composer.
2. Switch to Experiment Run mode.
3. Pick the experiment and run from the two dropdowns.
4. Click Apply. The backend fetches the run’s hydra/ bundle via load-bundle, validates composition produces the same hash, and pre-populates the Composer with the archived selections.
5. The notebook’s hydra_baseline_source is now mlflow://<run_id> and the badge reads RUN xxxxxx with a green dot.

From this state, any additional Composer tweak is a *delta* against the archived run. On next execute, the run’s bundle is re-archived under the new run id, so lineage is preserved at each generation.

2.5 Discussion-ready talking points

Q: Why compose configs inside noted instead of calling hydra.main? A: hydra.main assumes a script entry point with CLI arguments. Notebooks have neither. noted composes via OmegaConf.create(...) on the backend and ships the resolved tree to the kernel as an injected variable. This keeps notebook code free of Hydra imports and lets the same composition engine run against either a local source or an MLflow-archived bundle via the HydraSource abstraction.

Q: Why is “templates not config” a load-bearing idea? A: Because the YAML files on disk do not uniquely determine the resolved configuration - the defaults list and the overrides do. Two runs reading the same templates can produce different configs. The sha256(resolved_yaml) hash is the only proof-of-identity that survives template edits. The baseline badge surfaces this to users who would otherwise not realize their config drifted.

Q: What exactly does the config hash buy at compare time? A: Byte-identical noted.hydra_config_hash between two runs is a guarantee that they saw identical resolved configurations. It does not guarantee identical metrics (GPU nondeterminism, floating-point ordering, dataset hash can still differ) but it rules out configuration as the source of any observed divergence. In the Compare panel, two runs with matching hashes and differing metrics point at data or seed; two runs with differing hashes point at the config diff as the first place to look.

Q: What happens when an archived bundle is missing or partial? A: The original symptom - before the MlflowSource.walk() fix - was that bundles produced against an MLflow baseline contained only config/config.yaml with no group files. The fix made walk() correctly recurse into subdirectories. Old pre-fix bundles remain permanently incomplete and are detected by load-bundle validation: composition against a partial source fails the hash check. The Composer surfaces a red X on the badge and refuses to apply.

Q: Why is training inlined in config.yaml instead of being a group? A: Because every training knob is something users reasonably want to sweep independently. Making training a group forces users to pick a “training preset” as a whole, which is a bad abstraction for hyperparameter exploration. Inlining surfaces each knob as its own Composer input. The same reasoning could apply to future groups that currently only have one meaningful option.

Q: What is the difference between a Composer override and editing a YAML file? A: An override is ephemeral - it lives in the notebook’s hydra_selections metadata and produces a distinct resolved hash. Editing a YAML changes the template, which changes what every notebook composing against that template will resolve to. The convention is: exploratory sweeps use overrides (no git diff), decisions that should stick use YAML edits (committed to the repo). The badge treats both the same way - it only cares about the hash.

Q: Why is HydraCache in-memory with no disk backing? A: Because MLflow is already the durable store. The cache is a per-session speedup for repeatedly loading the same run in the Composer. A restart rebuilds on first access. If persistence is ever needed, pickling the OrderedDict would suffice, but no user pain has surfaced to justify it.

Q: What is the relationship between notebook_uid and run_id? A: notebook_uid identifies *which notebook* produced the data; run_id identifies *which execution* of it. Both keys are needed for the HydraCache because the same notebook can be re-executed against multiple past runs as baseline, and each (notebook, past_run) pair is a distinct cache entry. The notebook_uid is generated lazily so notebooks that never use Hydra never acquire one.

Q: How does noted reconcile DAG-produced runs with Run Manager-produced runs in the Composer's run dropdown? A: The Airflow DAG emits an identical hydra/ artifact tree via its log_hydra_lineage task. Any run with a hydra/ artifact and the expected project tag surfaces in the dropdown regardless of origin. True parity is the goal - a DAG-produced run is resurrectable in the notebook just like a Run Manager run.

3. MLflow

3.1 Concept primer

MLflow is the *accounting system* of an MLOps stack. Where Hydra answers “what was this run configured to do?”, MLflow answers “what did this run actually do, and what artifact did it produce?”. The framework has four primary concepts that noted uses:

1. **Tracking server and runs.** An MLflow tracking server stores per-run records: a UUID, a start/end time, parameters (immutable once logged), metrics (time-series of scalar values), tags (mutable key/value strings), and artifacts (any file). Runs live inside experiments, which are named buckets. Every training cell execution in noted produces one run if it entered a start_run() context, or zero runs if it did not.
2. **Model registry.** Separate from runs, the registry is a named catalog of model versions. mlflow.register_model(runs:/<run_id>/model, "Jena Weather Forecaster") takes the model artifact produced by a run and exposes it as version N of a registered name. Versions are immutable once created; what moves between them is *alias pointers* (@champion, @challenger, @staging).
3. **Logged Models (MLflow 3.x).** A new first-class entity introduced in MLflow 3.x. A Logged Model is the model artifact stored *with* its own ID, its own tags, its own lineage - distinct from the run that produced it. In the old model, the run owned the artifact and the registry owned versions; in the new model, the Logged Model is a third entity that can be referenced independently. noted’s Logged Models view in the Explorer reflects this: each Logged Model appears as a subtree under its producing run with MLmodel, conda.yaml, python_env.yaml, requirements.txt, and any supporting files visible.
4. **Signatures and flavors.** Every model artifact carries a *signature* (typed input/output schema: tensors with dtype and shape, or columnar specs) and one or more *flavors* (e.g. tensorflow, pyfunc, sklearn). The signature is what the serving client uses to validate a request payload before forwarding it to the model. The flavor tells mlflow.pyfunc.load_model how to reconstruct the model in memory.

What MLflow does not do. It does not compose configurations (that is Hydra). It does not orchestrate (that is Airflow). It does not version data (that is DVC). In noted, MLflow is the ledger; everything else writes into it.

Why aliases matter. @champion decouples *which version is in production* from *what its version number is*. A rollback from v7 to v6 is client.set_registered_model_alias(name, "champion", 6) - the alias hops, the serving client (pointing at @champion by default) picks up the change on its next health refresh, and there is no redeploy. Version numbers are stable historical identifiers; aliases are movable pointers. This is the single most important MLOps idiom to internalize.

3.2 Where MLflow lives in the notebook

The Tutorial 3 notebook imports MLflow in **cell 116** and uses it in exactly two places: the training run (cell 116) and the promotion step (cell 117). Everything else happens automatically, either in noted's Run Manager prelude or in the register_and_promote helper.

Cell 94 - training. No direct MLflow calls. train_model() is called with es_cfg and lr_cfg from Hydra. The Keras on_epoch_end callback reaches mlflow.log_metric(...) through the monkey-patched wrapper that noted installs at prelude time. If no run is active the log call is a no-op; if one is active, the metric streams to both the tracking server and to the live metrics panel via the application/x-noted-metric mime type.

Cell 116 - tracking and logging. After training, the user's code calls:

```
mlflow.set_experiment("Jena Weather")
with mlflow.start_run() as run:
    mlflow.log_params(...)          # hyperparameters from cfg
    mlflow.log_metrics(...)          # final test metrics
    mlflow.set_tag("task", "forecasting")
    mlflow.tensorflow.log_model(model, artifact_path="model",
                                signature=infer_signature(...))
```

However: because noted's Run Manager has already opened a run via the prelude (Section 3.3.1), this start_run() call will either *re-enter* the active run or open a nested run depending on MLflow's behavior. In practice, cell 116 is shaped to *continue* the Run Manager's run rather than open a new one. The if run is not None: guard at the top of cell 117 is what keeps this defensive: if for any reason there is no active run (e.g. the user executed cell 116 outside a Run Manager context), promotion is skipped rather than crashing.

Cell 117 - promotion.

```
if run is not None:
    result = register_and_promote(run_id=run.info.run_id,
                                  model_name="Jena Weather Forecaster",
                                  new_mae=test_mae)
```

register_and_promote lives at /home/logus/env/iscte/jena_weather/src/evaluation/promote.py:29. It:

1. Reads the current @champion version's test MAE via MlflowClient.get_model_version_by_alias(name, "champion").
2. Calls mlflow.register_model(f"runs:/{run_id}/model", model_name) to produce version N+1.

3. Compares new_mae < champion_mae; if better, calls client.set_registered_model_alias(name, "champion", new_version).
4. Returns {promoted: bool, improvement_pct: float, new_version: int}.

This is the whole promotion pipeline. Every other MLflow side-effect (tags, parameters, metrics, artifacts, bundle archival) happens earlier, either in the Run Manager prelude or as monkey-patched wrappers.

3.3 How noted bridges to MLflow

noted's MLflow integration is a tracking layer wrapped around a user-written notebook. The user writes MLflow calls as if they were in a bare script; noted's prelude transparently installs the run context, the metric streaming, and the lineage tags.

3.3.1 The Run Manager prelude

backend/app/managers/auto_instrumentation.py holds three injected code blobs:

- RUN_START_CODE (lines 15-26) - opens a run with mlflow.set_experiment(experiment_name) and mlflow.start_run(run_name=run_name), sets the instrumentation=experiments tag, stores the run handle in the kernel as run.
- RUN_END_CODE (lines 28-36) - closes the run with mlflow.end_run() after the last cell.
- METRICS_HOOK_CODE (lines 44-124) - monkey-patches mlflow.log_metric, mlflow.log_metrics, and mlflow.start_run so that each call additionally emits a noted-specific IPython display message.

get_run_start_code() (line 134) assembles the three blobs plus optional dataset-logging code (DVC hashes) and returns them as a single Python string. ExecutionBridge.execute_run() (line 277 of execution_bridge.py) silently executes that string before any cell runs, then executes get_run_end_code() after the last cell completes.

The variable run lives in the kernel namespace from prelude time onward; cell 117's if run is not None: is therefore a cross-cell check whose truth is set up by the prelude. This pattern is what the memory flags as "tech-debt invisible preludes" - it works, but it makes the notebook non-portable.

3.3.2 Live metrics via application/x-noted-metric

Inside METRICS_HOOK_CODE (around line 58), mlflow.log_metric and mlflow.log_metrics are replaced with wrappers that call through to the real function *and* emit an IPython display with a custom mime type:

```
IPython.display.display({
    'application/x-noted-metric': json.dumps({
```

```
        'run_id': run_id, 'key': key, 'value': value,
        'step': step, 'timestamp': timestamp
    })
}, raw=True)
```

The backend's IOPub dispatcher (execution_bridge.py:582-594) watches for this mime type. On receipt it:

1. Suppresses the display from the cell output (so the notebook cell does not print a giant JSON blob).
2. Parses the JSON.
3. Emits a metrics:update socket.io event to the frontend.

The frontend's live metrics panel subscribes to this event and updates the chart in real time, *during training*, with no polling and no hooks the user has to write. Every log_metric call in any library (Keras callback, custom logger, model.fit instrumentation) surfaces live because the patch is on MLflow itself.

3.3.3 Run-start hook via application/x-noted-run-start

The same metrics-hook patch wraps mlflow.start_run (around line 89) so that, on successful run creation, it emits:

```
IPython.display.display({
    'application/x-noted-run-start': json.dumps({
        'run_id': run_id, 'timestamp': timestamp
    })
}, raw=True)
```

ExecutionBridge._dispatch_iopub_msg (line 599-613) watches for this mime type and, on first occurrence per session, fires _log_hydra_bundle_for_run(run_id) in a background thread. This is the handoff from MLflow-aware code to Hydra-aware code: the run must exist before the bundle can be uploaded against it, so the bundle-upload side-effect is triggered by the run-start event.

3.3.4 Tag injection and git lineage

When _log_hydra_bundle_for_run runs, it also injects lineage tags on the run (lines 775-850):

- noted.hydra_config_hash - SHA256 of the resolved config (the primary key for "same config").
- noted.project_id - the noted project id (used by the Composer's Time Machine to filter runs that belong to this notebook's project).

- noted.git_commit - current commit SHA of the project directory, resolved via subprocess.
- mlflow.source.git.commit / mlflow.source.git.branch - the standard MLflow tags, populated the same way.

The git tags are best-effort: if the project is not a git repo, they are omitted silently. This is the one place where "silently" is acceptable because the absence of git info is diagnosable from the tag list - a missing tag is a visible null, not a masked failure.

3.3.5 target_mean / target_std for serving

During training, the notebook computes the target's scaler statistics (mean and standard deviation of the regression target on the train split). Two of the most load-bearing logged params:

```
mlflow.log_param("target_mean", float(train_target.mean()))
mlflow.log_param("target_std",  float(train_target.std()))
```

These are not training hyperparameters; they are *inference-time* stats that jena_client reads back to inverse-transform the model's scaled predictions into human-readable Kelvin. Without them, the downstream client would return scaled numbers that nobody could interpret. Logging them on the run ties the inverse-transform to the exact training run and its champion version.

3.3.6 Model Registry, Deploy / Unload / Try It

The Registry panel (frontend/js/panels/explorer/ExplorerRegistryViews.js) renders:

- A **Models tree** fetched from GET /api/registry/models. Each registered model is a tree node; its children are versions, each with its alias labels rendered inline (v7 @champion, v6, v5, etc.).
- A **Model detail view** (line 118-232) showing the version table, signature parsed from MLmodel YAML, flavors, and alias assignment buttons.
- A **Version detail view** (line 234-358) with three action buttons: Deploy, Unload, Try It.

Deploy (line 466+) instantiates ModelDeployer and posts to /api/serving/load as NDJSON streaming. Phases stream in: starting, downloading_artifact, loading_model, ready, failed. The button flips to "Unload" on success. State polling via /api/serving/health keeps the button state coherent across sessions and users.

Unload posts to /api/serving/unload and releases VRAM via the model loader's explicit del model; gc.collect() sequence (an earlier refactor neutered the runtime-install path that caused stale C-extension crashes).

Try It opens the ExplorerServingViews.showTryItPanel(...) panel: a form rendered from the model's signature, with each tensor-spec field auto-populated from a sample row of the train split. Submit hits the serving endpoint and the response appears inline.

3.3.7 Logged Models (MLflow 3.x)

The Logged Models view (ExplorerMlflowViews.js:595-815) is a nested tree under each run. Backend endpoints:

- GET /api/mlflow/runs/{run_id}/logged_models (mlflow.py:59) - MlflowManager.list_logged_models_for_run (mlflow_manager.py:224) scans the experiment's models/ directory, picks MLmodel files whose run_id matches, and returns a flat artifact tree for each Logged Model it finds.
- GET /api/mlflow/logged_models/{experiment_id}/{model_id}/download?path=... (mlflow.py:76) - streams a single file from a Logged Model's artifact root via MLflow's artifact proxy.

In the frontend, each file node (MLmodel, conda.yaml, python_env.yaml, requirements.txt, model.keras, etc.) opens a detail pane with the file contents highlighted by hljs. Binary files (.keras, .npy) render a placeholder instead of attempting to syntax-highlight.

3.4 Operations

What register_and_promote does

The full sequence for a single promotion:

1. Read @champion version if one exists: client.get_model_version_by_alias(name, "champion").
2. Read its test_mae metric via client.get_run(version.run_id).data.metrics["test_mae"].
3. Compare against the new run's test_mae.
4. Register the new artifact: mlflow.register_model(f"runs:/{run_id}/model", name) - returns a ModelVersion with a version number N+1.
5. If new_mae < champion_mae: client.set_registered_model_alias(name, "champion", new_version.version).
6. Return {promoted: bool, improvement_pct: float, new_version: int}.

Every step is idempotent by design: re-running against the same run_id produces the same new version number if called within a short window (MLflow coalesces), and alias assignment overwrites prior assignments.

Inspect a model's signature and parameters

From the Registry panel: 1. Click a model, then a version. The Version detail pane shows the signature parsed from MLmodel YAML: each input tensor's name, dtype, and shape;

each output tensor's name, dtype, and shape. 2. Click "Logged Model" in the same pane to open the Logged Model subtree. MLmodel contents render with hljs syntax highlighting. 3. The params table shows every key logged via log_param, including target_mean and target_std for Tutorial 3.

From MLflow's own UI (at :5000), the same data is available but without the noted-specific rendering.

How @champion drives serving

The serving container (client/app/model_loader.py) resolves @champion on every deploy:

1. Client POSTs /api/serving/load with {name, version} or {name, alias: "champion"}.
2. ModelLoader.load_by_alias(name, alias) calls the MLflow tracking API's /registered-models/{name}/alias/{alias} endpoint.
3. The response gives the current version number the alias points at.
4. ModelLoader.load_by_version(name, version) downloads the Logged Model artifact tree and calls mlflow.pyfunc.load_model(...).

Rolling back is a single alias-hop on the MLflow server - no redeploy required if the client is configured to re-resolve the alias on health-check intervals.

3.5 Discussion-ready talking points

Q: Why is Run Manager-only tracking a deliberate design? A: Because every notebook-surfaced MLflow call is a user-visible side-effect, and the user does not want to write start_run() / end_run() boilerplate in every cell. The Run Manager prelude owns the run lifecycle; the notebook only has to call log_params, log_metrics, and tensorflow.log_model. The if run is not None: guard in cell 117 is the only defensive concession - it lets the same notebook survive being run outside noted (in which case promotion is a no-op).

Q: How does the live metrics panel know about Keras callbacks? A: It does not know about Keras. It knows about mlflow.log_metric. The monkey-patch at prelude time wraps log_metric itself, so any source that calls it (Keras callback, a custom on_epoch_end, a manual log statement) emits a application/x-noted-metric display which the IOPub dispatcher forwards to the frontend over socket.io. The patch is library-agnostic; it works for sklearn, xgboost, or plain-Python training loops as long as they call MLflow.

Q: MLflow 2.x model artifacts vs MLflow 3.x Logged Models - why do both views exist in noted? A: Backward compatibility. A run produced against MLflow 2.x has its model artifact at {run_id}/artifacts/model/ inside the run. A run produced against MLflow 3.x additionally has a Logged Model entity at {experiment_id}/models/{model_id}/ with its own identity. noted's Registry view reads from the registered-model API (works for both). The Logged Models view (ExplorerMlflowViews.js:595) is a 3.x-only surface that shows the independent Logged Model entity, which is useful for inspecting artifacts that may not have been registered at all.

Q: How does @champion decouple deployment from version numbers? A: Version numbers are stable historical identifiers - v7 was produced at time T with these metrics, and that fact never changes. @champion is a movable pointer that the serving client resolves on every load (or on a health-check cadence). Rolling back from v7 to v6 is a one-line alias reassignment on the MLflow server; the serving client picks it up at its next resolve. Redeploy is not required as long as the client is alias-aware. The alternative - hard-coding a version number in the serving config - forces a redeploy for every rollback or promotion.

Q: What prevents two promotion attempts from racing? A: Nothing at the MLflow layer - set_registered_model_alias is last-writer-wins. In practice, noted's promotion is single-user from a notebook cell, and the DAG's promotion task is gated by the training task succeeding. If a multi-writer scenario became a concern, optimistic concurrency on the champion's run_id tag would be the minimal add.

Q: Why is target_mean / target_std logged as a param rather than as a file? A: Because params are trivially readable via the run's API without downloading any artifact. A file would require an additional round-trip and artifact-path knowledge. The two scalars are tiny, immutable once logged, and accessed by jena_client on every prediction to invert the scaling. Params are the right shape for that access pattern.

Q: What happens when an MLflow experiment is deleted out from under noted? A: A soft-deleted experiment is MLflow's foot-gun: get_experiment_by_name(...) returns the deleted experiment, start_run fails cryptically, and the user sees a confusing error with no recovery path in the UI. The known backlog item is to detect this in RUN_START_CODE, surface it as an explicit frontend notification, and offer restore/purge actions. Until that lands, the workaround is to purge the deleted experiment from the MLflow UI and let noted recreate it.

Q: Why does noted use MLflow's artifact proxy for Logged Model downloads instead of a direct file URL? A: Because the MLflow server is the authoritative resolver of artifact URIs, which may point at MinIO, local disk, or any artifact store. Going through the proxy means noted's frontend never has to know where artifacts physically live - the proxy translates experiment_id/model_id/path into whatever the backing store requires. This also gives a single place to add auth in the future.

4. DVC

4.1 Concept primer

DVC (Data Version Control) is a thin layer over git that adds one capability git refuses to do well: version large binary files. The mental model is simple: instead of committing a 43 MB CSV into git, you commit a tiny *pointer file* that contains the CSV's content hash, its filename, and its size. The actual bytes live in a remote object store (MinIO, S3, GCS, whatever), addressable by their hash. dvc pull reads the pointer, fetches the bytes into the working tree. dvc push uploads the bytes to the remote. Git continues to version the pointer.

Four concepts do most of the work:

1. **.dvc files.** A YAML manifest with three fields per tracked output: md5, size, path. Committing a .dvc file to git pins an exact content hash to a commit. Checkout an old commit, run dvc pull, and you get the bytes that matched that commit - guaranteed by the md5, not by the filename.
2. **Remote storage.** Configured in .dvc/config. A single remote has a URL (s3://bucket/prefix) and credentials. DVC uploads/downloads to/from this remote based on hash addressing. The remote is content-addressed: the same md5 is never uploaded twice.
3. **Stages and pipelines.** dvc.yaml defines processing stages with inputs, outputs, and a command. dvc repro runs stages whose inputs changed. This is *not* what noted uses - Airflow is the pipeline tool in noted's stack. DVC in noted is strictly the data-versioning primitive, not the orchestrator.
4. **Content addressing.** Two datasets with identical bytes have identical md5s regardless of filename. This means DVC naturally deduplicates: two .dvc files with the same md5 point at the same object in the remote. It also means changing a single byte produces a new object.

Why DVC and not git LFS? Git LFS stores large files on a git hosting provider (GitHub LFS, GitLab LFS). DVC stores them on *any* object store the team already owns. For a local-first, provider-agnostic stack like noted's, DVC's indifference to the host is the right shape. The other difference: LFS is tightly coupled to git's checkout mechanism; DVC is a separate tool you invoke explicitly. That separation is what lets noted read .dvc files without needing a git hook.

What DVC does not do. It does not schedule, train, or track experiments. It does not understand semantic data quality (that is Evidently). It produces no alerts. It is a hash-addressed blob store with a pointer-file syntax.

4.2 Where DVC lives in the notebook

The Tutorial 3 notebook makes no direct DVC calls. The one load-bearing line is in cell 16:

```
DATASET_PATH = str(PROJECT_ROOT / cfg.data.file)
df = ingest(DATASET_PATH)
```

cfg.data.file resolves to either data/jena_climate_2009_2016.csv (full 2009-2016 series) or data/jena_climate_2012.csv (one-year subset), depending on the Composer's data selection. Both files are DVC-tracked, both have corresponding .dvc manifests next to them, and both must exist in the working tree for the notebook to run.

The project's DVC state at v0.1 of this manual:

```
jena_weather/
  .dvc/config                          <- MinIO remote configuration
  .dvcignore                           <- empty
  data/
    jena_climate_2009_2016.csv        <- 43.2 MB, md5 959915f0...
    jena_climate_2009_2016.csv.dvc    <- pointer file, committed to git
    jena_climate_2012.csv             <-  5.2 MB, md5 d3956bd0...
    jena_climate_2012.csv.dvc         <- pointer file, committed to git
  src/data/
    filter_year.py                     <- derivation script
    ingestion.py                       <- CSV loader used by the notebook
```

src/data/filter_year.py (21 lines) is the *derivation script* that produced the 2012 subset. It takes jena_climate_2009_2016.csv as input, parses the Date Time column (format DD.MM.YYYY HH:MM:SS), and writes rows whose year matches the target. The script is committed to git but is not a DVC stage - it was run once manually, the output was dvc add-ed, and now the .dvc file is the artifact. This is a deliberate choice discussed in Section 4.5: Airflow is the pipeline tool, so DVC stages would duplicate orchestration responsibilities.

src/data/ingestion.py (line 7-13) is load_dataset(file_path) - a one-line pd.read_csv(path) wrapper with datetime parsing. It is filename-agnostic: whatever path you hand it gets read. The caller is responsible for ensuring the file exists (i.e. that dvc pull has been run at least once for the repo's current state).

4.3 How noted bridges to DVC

noted treats DVC as read-only metadata. The backend never runs dvc commands - it parses .dvc files directly because they are just YAML, and YAML is cheaper to read than to shell out to a CLI tool. This makes the integration resilient to DVC version drift: a future DVC release that adds new keys to the manifest does not break noted's hash extraction.

4.3.1 DvcManager: reading .dvc files

backend/app/managers/dvc_manager.py is the single parser.

- DvcManager.status() (line 217-284) walks a project directory, finds every *.dvc file, loads it as YAML, and extracts each outs block's {path, md5, size}. Returns a dict with tracked_files: [{path, hash, size}, ...].
- DvcManager.data_overview() (line 390-423) loops over all registered projects and aggregates a single catalog view.

No DVC CLI is invoked. There is no dependency on DVC being installed on the backend server. The only requirement is that .dvc files exist in the expected shape, which is a git-committed contract.

4.3.2 Dataset hash injection on run start

backend/app/managers/auto_instrumentation.py:152-162 defines _get_dataset_logging_code(dataset_hashes: dict). It emits Python code that, when executed in the kernel, calls:

```
mlflow.log_param("dvc_data_hash", hash_value)
mlflow.set_tag("dvc.data_hash", hash_value)
mlflow.set_tag("dvc.data_file", file_path)
```

get_run_start_code() (line 134-149) appends this blob to the Run Manager prelude whenever dataset_hashes is non-empty. The prelude executes before any notebook cell, so by the time cell 94 starts training, the active MLflow run already carries its data-lineage tags.

4.3.3 The run:execute handler's hash resolution

The backend decides which hashes to log. backend/app/main.py:673-749 is the handler:

1. Receive the payload from the frontend. If hydra_config is present in the payload (i.e. the notebook has a Hydra baseline set), ignore the frontend's datasets[] array entirely.
2. Compose the Hydra config and read cfg.data.file.
3. Call dvc_mgr.status(project_repo) and build a lookup {path: hash}.
4. Resolve cfg.data.file to its hash. Pass {cfg.data.file: hash} as dataset_hashes to execution_bridge.execute_run().
5. If no hydra_config is in the payload (legacy non-Hydra notebook), fall back to the frontend's datasets[] selections.

This bypass is deliberate. The Run Manager previously had its own dataset checkbox picker *in addition* to the Composer's data group selector. Two UIs could drift. Now, for any Hydra-using notebook, the Composer is the single source of truth and the Run Manager renders a read-only row showing the currently-selected dataset.

4.3.4 Data Catalog tree and version history

frontend/js/panels/explorer/ExplorerDataViews.js renders the Data tab.

- loadDataFiles() (line 41-66) fetches from /api/dvc/status and caches per-file metadata under _dataFileMeta.
- showDataFileDetail() (line 68-176) renders a detail card for a selected file: path, size, md5 in monospace, and a version-history list.
- Version history (line 93-176) fetches from /api/dvc/file-history, which resolves .dvc file contents across git history. Each historical version renders as a row with short commit, message, date, and size. Non-current versions have a “Checkout” button (line 140-167) which restores that version via a git + dvc pull sequence.

applyDataHealthDot() (line 236-251) adds a colored indicator to the Data tree’s root node based on the most recent Evidently quality snapshot. This is the cross-chapter tie-in to Chapter 5 - the dot reads from Evidently’s health endpoint but lives on the Data tree UI.

4.3.5 RunManagerPanel: Hydra-aware vs legacy mode

frontend/js/RunManagerPanel.js renders the Run tab. Two code paths:

- **Hydra-driven (line 262-292)** - if getHydraDataFile() returns a file (notebook has a Hydra baseline set), renders a single read-only row: [hydra icon] data/jena_climate_2012.csv from Hydra config. If the file is not DVC-tracked, an orange warning appears inline.
- **Legacy (line 295-334)** - if the notebook is non-Hydra, renders a multi-select checkbox list of all DVC-tracked files. Each checkbox toggles membership in run.datasets[], which the backend uses as the fallback hash list.

The first path is the supported flow. The second exists because noted does not force Hydra adoption on every notebook.

4.3.6 MinIO remote and .dvc/config

The jena_weather project’s .dvc/config (lines 1-8) points at:

```
[core]
    remote = minio
['remote "minio"']
    url = s3://noted-dvc
    endpointurl = http://noted-minio:9000
    access_key_id = admin
    secret_access_key = password
```

noted-minio is the in-compose hostname. From outside the compose network (e.g. a user’s host running dvc pull), the endpoint has to be http://localhost:9000 - this is the one

configuration that differs between container and host. No .dvc/config.local is checked in; container-baked credentials are acceptable here because MinIO is not exposed beyond the compose network in the default deployment.

The bucket is noted-dvc, shared by all noted projects. Content addressing makes the shared bucket safe: identical bytes produce identical paths inside the bucket, regardless of which project pushed them.

4.3.7 Airflow DAG dataset handling

dags/jena_training_pipeline.py:202-223 defines ingest_data(). It reads cfg['data']['file'] (default data/jena_climate_2009_2016.csv if not overridden) and loads the CSV. MLflow logging (line 371-397) logs the file path as a parameter but, at v0.1 of this manual, does **not** log the dvc.data_hash tag that the Run Manager injects via _get_dataset_logging_code(). This is an inconsistency flagged for future correction: DAG-produced runs appear in the Composer's Experiment Run dropdown but without the full data-lineage tag surface. The fix is to add an explicit client.log_param(run_id, "dvc_data_hash", hash) inside log_hydra_lineage or as its own task, resolving the hash the same way main.py:706-715 does.

4.4 Operations

Add a new tracked dataset

1. Place the file in a location inside the project, e.g. data/new_dataset.csv.
2. From the project root: dvc add data/new_dataset.csv. This creates data/new_dataset.csv.dvc and adds the file to .gitignore.
3. dvc push to upload to MinIO.
4. git add data/new_dataset.csv.dvc .gitignore && git commit -m "Add new_dataset".
5. In the Composer, either add it as an option to an existing group (config/data/new_dataset.yaml with file: data/new_dataset.csv) or add an override input in config.yaml so it can be selected.
6. Reopen the Data tab - the new file appears automatically. No noted restart.

Pull a specific version

1. git log data/jena_climate_2012.csv.dvc to find the commit that pins the version you want.
2. git checkout <commit> -- data/jena_climate_2012.csv.dvc
3. dvc pull data/jena_climate_2012.csv.dvc - DVC reads the pinned md5, downloads from MinIO.
4. Alternatively, use the Data tab's version-history Checkout button - it runs the same two commands under the hood.

Switch dataset via the Composer

1. Open the Composer.
2. In the data dropdown, pick jena_2012_dataset or jena_full_dataset.
3. Click Apply. The notebook's hydra_selections.group_selections.data is updated and its hydra_config_hash recomputed.
4. On next Run Manager execute, main.py resolves cfg.data.file and injects its md5 as the dvc_data_hash param. The Run's MLflow lineage now carries the dataset identity.
5. No manual dvc pull is needed if the file is already in the working tree, which it will be as long as the previous pull produced both variants.

Flow of the dataset choice into a run's lineage

The hash injection chain is:

Composer selection -> hydra_selections.group_selections.data -> run:execute payload -> main.py composes cfg -> cfg.data.file -> dvc_mgr.status() lookup -> dataset_hashes = {file: md5} -> get_run_start_code(dataset_hashes) -> prelude mlflow.log_param("dvc_data_hash", md5) -> run tags include dvc.data_hash and dvc.data_file -> Composer Time Machine filters and compare views can use these tags.

4.5 Discussion-ready talking points

Q: Why two separate dataset files instead of a single versioned one? A: Because the Composer dropdown presents both as parallel options for user choice, and the user flow demands both be available simultaneously. If it were a single file with two versions, picking "2012" would require a dvc pull of that specific version, overwriting the file and forcing all notebooks currently pointing at the "full" version to re-pull. Two files lets every notebook see both datasets side-by-side with no git checkout gymnastics. The content-addressed storage means the duplication is only in pointer files, not in bytes.

Q: Why DVC + MinIO instead of a full data lake? A: Because the project-scoped, local-first deployment model does not justify the operational cost of a data lake. MinIO gives S3-compatible storage in a single compose service. DVC gives git-native pointer files that are easy to review in PRs. A data lake would add catalog, governance, table formats, and ingestion pipelines - none of which are wanted at this scale. The upgrade path exists: a future deployment that needs cataloguing can swap the DVC remote to an S3-backed one and add Iceberg or Delta on top without changing the notebook code.

Q: What is the role of the derivation script filter_year.py? A: It is the *recipe* for how jena_climate_2012.csv was produced from jena_climate_2009_2016.csv. It is committed to git as source code, not registered as a DVC stage. The choice is deliberate: stages would make DVC an orchestrator, and noted already has Airflow in that role. For one-shot derivations, a committed script + dvc add of the output is simpler and does not require

every contributor to install DVC or run dvc repro. The provenance is readable from the repo - anyone can see the script that produced the file.

Q: What happens when the DVC remote is unreachable? A: dvc pull fails at the network layer. The notebook will then fail at cell 16 with a FileNotFoundError. noted does not proactively pull on notebook open - it assumes the user has pulled as part of project setup. A future improvement is to detect a missing file at cfg.data.file resolve time and surface a one-click “Pull from DVC” action in the Data tab, which would shell out to dvc pull with the right target. Until then, failure is loud: a Python exception in the cell output. Loud failure is better than silent degradation.

Q: Why does noted read .dvc files directly instead of using the DVC Python API? A: Because .dvc files are small, stable YAML. Loading them with yaml.safe_load is three lines and has no transitive dependencies. The DVC Python API would pull in DVC’s full package graph, its global config, and its command-line surface. None of that is needed for noted’s read-only hash extraction. The trade-off: noted does not implement arbitrary DVC features (stages, pipelines, experiments) because those require the DVC runtime. noted’s contract is “read hashes from .dvc files”, and everything else stays in the DVC CLI the user runs directly.

Q: How does noted know which project a dataset belongs to? A: By the project’s directory structure. dvc_mgr.status(project_repo) walks a specific path and finds every .dvc file under it. Projects are registered via NOTED.md (or the project registry), and each registered project has a root path. The same DVC remote can back many projects, and each project’s .dvc files are scoped to its tree. Cross-project dataset sharing would require copying .dvc files, which is why the content-addressed remote is valuable: same bytes, no re-upload.

Q: Is dvc.data_hash the single source of truth for “which dataset did this run see”? A: For Run-Manager runs produced in a noted-backed notebook, yes. Combined with the hydra_config_hash (which includes cfg.data.file in the resolved YAML) and the run’s git commit tag, it triangulates the data identity with high confidence. For DAG runs at v0.1 of this manual, the dvc.data_hash tag is not yet logged, which is tracked as a known gap. Closing that gap brings DAG runs to parity with Run Manager runs in the Compose/Time Machine dropdown.

5. Evidently

5.1 Concept primer

Evidently is a data and model monitoring framework. Where MLflow tracks run-level metadata (params, metrics, artifacts) and DVC tracks bytes, Evidently tracks *statistical properties of the data* itself: distributions, correlations, null rates, outliers, and how these change between two datasets. The output is always a report - an HTML document with charts, stats, and a verdict - that can be reviewed as a one-off or persisted in a workspace for time-series monitoring.

Four concepts structure every Evidently integration:

1. **Reports and presets.** A Report is a container for Metric objects (e.g. ColumnDriftMetric, DatasetSummaryMetric). A *preset* is a curated bundle of metrics wrapped in a single class: DataSummaryPreset (distribution summaries per column), DataDriftPreset (pairwise distribution comparison between two datasets), RegressionPreset (error metrics for a regression model's predictions). Presets are the default entry point; custom metric lists are for advanced use.
2. **Snapshots.** When a Report is run and saved into a workspace, the result is a *snapshot* - a serialized JSON payload with metadata (timestamp, tags) and the full metric output. Snapshots are the unit of persistence; they are also what the Evidently UI renders as dashboard rows.
3. **Workspace and project model.** A *workspace* is a storage root (local directory or remote HTTP service). Inside it, *projects* are named folders; inside each project, a time-ordered list of snapshots. The HTTP service (evidently/evidently-service) exposes a JSON API over the same data.
4. **The three preset families noted uses.** DataSummaryPreset (quality check on a single dataset), DataDriftPreset (train-vs-test or train-vs-prod comparison), and RegressionPreset (error distribution on held-out predictions). The Tutorial 3 notebook exercises the first two; RegressionPreset is reserved for production-monitoring loops not yet wired in.

What Evidently does not do. It does not ingest data, schedule jobs, or raise alerts. It computes statistics over what you hand it. Integration with a scheduler (Airflow), a tracker (MLflow), or an alerting system is the caller's responsibility. In noted, that glue is thin by design: Evidently is the stats engine; MLflow holds the run that the stats describe; Airflow runs the job that computes them.

Tagging is the hinge. Every snapshot in noted carries tags like ["data-quality", "jena-weather", "pipeline"] or ["drift", "jena-weather", "run-1"]. Tags make snapshots filterable in the Evidently UI and scopable in the backend's health endpoints. Without tags, the workspace becomes a flat pile of unlabelled snapshots that nobody can triage.

5.2 Where Evidently lives in the notebook

The Tutorial 3 notebook exercises Evidently in two cells: **cell 114** (data quality) and **cell 119** (drift). Both connect to the same RemoteWorkspace at http://noted-evidently:8000, both write into the same "Jena Weather" project, and both produce one snapshot per execution.

Cell 114 - data quality

```
from evidently import Report, Dataset, DataDefinition
from evidently.presets import DataSummaryPreset
from evidently.ui.workspace import RemoteWorkspace

ws = RemoteWorkspace("http://noted-evidently:8000")
# get or create project
projects = [p for p in ws.list_projects() if p.name == "Jena Weather"]
project = projects[0] if projects else ws.create_project("Jena Weather")

summary_report = Report(metrics=[DataSummaryPreset()],
                        tags=["data-quality", "jena-weather", "run-1"])
snapshot = summary_report.run(current_data=Dataset.from_pandas(df_features))
ws.add_run(project.id, snapshot, include_data=False)
```

The point of this cell is to write a baseline quality snapshot against the training data *before* any model is trained. The resulting snapshot surfaces distribution shapes, null counts, unique-value counts, and basic correlation. The Evidently UI (embedded in noted via the nginx proxy) renders it as a dashboard row that persists across sessions.

include_data=False means the snapshot carries only aggregates, not the raw rows. This keeps the evidently-data volume compact and avoids re-storing the dataset the DVC layer already has.

Cell 119 - drift report

```
from evidently.presets import DataDriftPreset

ref_dataset = Dataset.from_pandas(df_train[feature_cols])
cur_dataset = Dataset.from_pandas(df_test[feature_cols])
drift_report = Report(metrics=[DataDriftPreset()],
                      tags=["drift", "jena-weather", "run-1"])
snapshot = drift_report.run(current_data=cur_dataset, reference_data=ref_dataset)
ws.add_run(project.id, snapshot, include_data=False)
```

This is a train-vs-test drift check. Reference = train split, current = test split. DataDriftPreset computes per-feature distribution divergence (PSI / Wasserstein / chi-

squared depending on dtype) and a rolled-up dataset_drift_share metric. The UI renders red for features whose drift score exceeds the preset's threshold.

At v0.1 of this manual, **cell 119 does not link the snapshot to the MLflow run_id** - that linkage exists only in the Airflow DAG. The notebook path therefore produces an orphaned drift snapshot from the Compose/Time Machine perspective: visible in Evidently, but not cross-navigable to the training run that produced the model. Closing this gap is a one-line change (metadata={"run_id": active_run_id} on the Report) tracked for future work.

Known runtime caveat. Both cells assume df_train, df_test, and final_feature_cols are live in the kernel namespace. If the kernel was restarted between training and cell 114/119, those variables are gone and the cell raises NameError: name 'final_feature_cols' is not defined. The workaround is "Run All" from the top before reaching cells 114/119. This is a user-observable Run All vs Run Manager foot-gun addressed in Chapter 6.

5.3 How noted bridges to Evidently

noted treats Evidently as a thin integration: the charts live in the Evidently UI, the health signal lives in noted. The bridge is three pieces: an embedded service, a health-endpoint shim in the backend, and a tree-node dot in the frontend.

5.3.1 The Evidently service

services/docker-compose.yml:308-318:

```yaml
evidently:
  image: evidently/evidently-service:latest
  container_name: noted-evidently
  ports:
    - "8009:8000"
  volumes:
    - evidently-data:/app/workspace
```

The service runs the official Evidently HTTP server. Workspace data (projects, snapshots, tags) persists in the evidently-data named volume, declared at line 345. The volume was added after a container rebuild was found to wipe an entire notebook-run's snapshots - the workspace was previously ephemeral with no user warning.

The port mapping 8009:8000 exposes the service on the host for debug inspection. Inside the compose network, other services reach it as http://noted-evidently:8000, which is what the notebook cells use.

5.3.2 Nginx proxy and the Service tab

services/nginx/nginx.conf:157-188 defines the /evidently/ location block:

```
location = /evidently { return 301 /evidently/; }
location ^~ /evidently/ {
    proxy_pass http://evidently:8000/;
    sub_filter '"/api/' '"/evidently/api/';
    sub_filter_once off;
}
```

The sub_filter is a subpath-aware rewrite: Evidently's SPA hardcodes /api/ as the base path for its own AJAX calls, so the proxy rewrites it to /evidently/api/ on the fly. Without this, the embedded UI would send API calls to noted's own /api/ namespace and collapse into 404s.

frontend/js/app-tabs.js:47 lists evidently alongside mlflow, airflow, and minio as a service-typed tab. When the user clicks the Evidently icon in the side icon bar, frontend/js/menu-commands.js:93 calls app._onIconBarClick('evidently'), which adds a new tab containing an <iframe> pointed at /evidently/. The iframe loads the Evidently UI through the nginx proxy.

5.3.3 Backend proxy endpoints and Data Health

backend/app/routers/evidently.py exposes a small health-endpoint shim:

- GET /api/evidently/projects (line 16) - lists all projects in the workspace.
- GET /api/evidently/projects/{project_id}/data-health (line 36) - returns {status: "green"|"yellow"|"red", summary: "..."} derived from the most recent data-quality-tagged snapshot.
- GET /api/evidently/projects/{project_id}/drift-status (line 41) - returns a drift status computed from the latest drift-tagged snapshot's dataset_drift_share: green (<= 20%), yellow (> 20%), red (> 50%).

The manager layer (backend/app/managers/evidently_manager.py) calls Evidently's HTTP API directly. No Evidently Python library is used on the backend - the manager does requests.get("http://noted-evidently:8000/api/...") and parses JSON. This keeps the backend's dependency graph clean and insulates it from Evidently Python version drift.

frontend/js/panels/explorer/ExplorerDataViews.js:213 (updateDataHealthBadge) calls these endpoints, aggregates across projects, and stores the worst-case status in _dataHealthStatus. applyDataHealthDot (line 236) renders an 8 px colored dot (green #4caf50, yellow #ff9800, red #f44336) on the Data tree's root node with a tooltip showing the summary text. This is the only noted-native surfacing of Evidently state; all other inspection happens in the embedded UI.

5.3.4 The Airflow DAG tasks

dags/jena_training_pipeline.py mirrors the notebook with two dedicated tasks:

- evidently_quality (line 285) - runs DataSummaryPreset() on the engineered features with tags ["data-quality", "jena-weather", "pipeline"]. Runs in parallel with training (line 556).
- evidently_drift (line 519) - runs DataDriftPreset() on train-vs-test with tags ["drift", "jena-weather", "pipeline"]. Runs *after* training completes (line 560) because it needs the test split to exist.

Line 536 is the critical linkage the notebook is missing:

```
drift_report.set_metadata({"run_id": train_result["run_id"]})
```

The drift snapshot produced by the DAG carries the MLflow run_id of the training run it describes. In the Evidently UI, this appears as a metadata field on the snapshot; in a custom drill-down, a user can take that run_id and open the MLflow run to see the trained model's params, metrics, and Hydra config bundle.

5.3.5 Not yet implemented: quality gates

Evidently supports TestSuite - a report where each metric is wrapped in a pass/fail assertion with user-defined thresholds. At v0.1 of this manual, noted does not use Test Suites. All snapshots are *profiling* reports - they produce statistics, not verdicts. EvidentlyManager.get_data_health_status() (line 118) notes in a comment: "this DAG/notebook currently uses profiling reports only".

The quality-gate pattern is the right next step: wrap DataSummaryPreset in a TestSuite that fails if null rate exceeds X, unique-value count drops below Y, or distribution shape shifts outside a Kolmogorov-Smirnov bound. A failing Test Suite would turn the Data Health dot red automatically instead of requiring a manual dataset_drift_share threshold. This is in the backlog, not on the critical path for Tutorial 3.

5.4 Operations

Filter snapshots by tag

1. Open the Evidently tab from the icon bar.
2. Navigate to the "Jena Weather" project.
3. The project page shows all snapshots. Use the tag filter to narrow to data-quality, drift, or a specific run label.
4. Click into a snapshot to see the full report.

Configure a custom dashboard panel

Evidently's UI supports user-authored dashboard panels (lines, bar charts, text) that aggregate metrics across snapshots over time. At v0.1 of this manual the project has no custom dashboard - only the default per-snapshot views. Adding one is a UI-only action inside Evidently; noted does not inject dashboards programmatically.

Link a drift finding back to the model trained on that split

For **DAG-produced** drift snapshots: 1. Open the drift snapshot. 2. Read the run_id field from its metadata. 3. In noted, open the Registry or MLflow view, navigate to the run, and inspect params and the Hydra bundle. 4. Open the Composer in Experiment Run mode with that run selected to see the exact config that trained the model.

For **notebook-produced** drift snapshots (v0.1): the run_id linkage does not exist. The user has to correlate by timestamp or by tag, which is less reliable. Add the set_metadata({"run_id": active_run.info.run_id}) call in cell 119 to close this gap.

Future quality-gate Test Suite pattern

When implemented, the pattern is:

```
from evidently.tests import TestColumnShareOfMissingValues, TestColumnDrift
from evidently.test_suite import TestSuite

ts = TestSuite(tests=[
    TestColumnShareOfMissingValues("T_degC", lt=0.01),
    TestColumnDrift("T_degC", stattest="wasserstein", stattest_threshold=0.1),
])
ts.run(reference_data=ref_dataset, current_data=cur_dataset)
ws.add_run(project.id, ts.as_dict(), tags=["quality-gate", "jena-weather"])
```

The backend's data-health endpoint can then be extended to read the pass/fail status from the Test Suite output instead of inferring from the profiling report's aggregates.

5.5 Discussion-ready talking points

Q: Why does noted treat Evidently as a thin integration (badges in noted, charts in Evidently UI)? A: Because Evidently's UI is already comprehensive and maintained upstream. Re-implementing its charts inside noted would duplicate work and force noted to stay in lockstep with Evidently's internal data model. Embedding it via iframe + nginx proxy gives users the full upstream UI with one-click access. noted's contribution is the *health dot* - the summary signal that tells a user "is there something to look at?" without forcing them to open the embedded UI on every glance. One-glance signal + one-click deep dive is the right UX shape.

Q: Why is the train-vs-test drift framing meaningful for Tutorial 3? A: Because Tutorial 3 uses a time-ordered split: train is earlier months, test is later months in the same calendar year. Any distribution drift between these two is a *signal that the training assumption of "past resembles future" is weakening*. It is a weaker version of the production-drift question ("does today's traffic look like last month's training data?"), applied at dataset-preparation time rather than at inference time. Detecting meaningful drift in this framing is a justification for training on a shorter time window, for retraining more frequently, or for adding features that are more time-invariant.

Q: What does it mean when drift is flagged on a feature that was specifically engineered? A: Engineered features can drift for two reasons. (1) The raw inputs drifted and the engineered feature inherited the drift. (2) The engineering logic has a subtle bug whose outputs are sensitive to a distribution the reference split did not cover. Reading the drift report next to the raw-feature drift report disambiguates: if both show drift, the root cause is upstream; if only the engineered feature drifts, the engineering code is the suspect. This is why running DataSummaryPreset on both raw and engineered features is valuable even when only one is fed to the model.

Q: Why does noted poll Evidently for health instead of subscribing to a push event? A: Because Evidently's service does not expose event streams - its API is HTTP request/response. Polling on the Data tab open is cheap (a few hundred bytes per project) and does not block the UI. A future improvement would be to cache the last known status and only refresh on explicit user action or on a long debounce, which is a small optimization that is not yet warranted at the current data volume.

Q: Why are the DAG and notebook Evidently tasks near-duplicates instead of sharing a library function? A: Deliberately, to keep the notebook self-documenting. The cells that a reader is most likely to read contain the full call to Report([DataSummaryPreset()]).run(...).add_run(project.id, snapshot) - no level of indirection to follow. The DAG duplicates the logic because it runs in a different execution context (Airflow worker, not a kernel with user-scope variables). A shared helper would simplify the code at the cost of making the notebook cells harder to read in isolation. For pedagogical notebooks like Tutorial 3, readability wins.

Q: What is the risk of the include_data=False choice on snapshots? A: The risk is that, years from now, an engineer wanting to replay exactly what the distribution looked like cannot - the aggregates are all they have. The alternative (include_data=True) would embed the raw rows into the snapshot, bloating the evidently-data volume by gigabytes. The right pattern is to store hashes alongside: the snapshot's metadata can carry the DVC md5 of the dataset it was computed on, and replay is possible by fetching that dataset version. This would bring data-lineage to Evidently in the same way it already exists for MLflow runs. Not yet implemented.

Q: How does the "Jena Weather" project get created on first run? A: The notebook's cell 114 does ws.create_project("Jena Weather") if no project with that name exists. The DAG's evidently_quality task does the same. Either path creates the project; subsequent calls find it and reuse it. The race condition between the two is benign because Evidently's service serializes project creation. A more robust pattern would be to create the project once at noted-startup time (via a backend bootstrap task), but the lazy-creation approach is adequate for a single-user local stack.

Q: Can drift snapshots be deleted programmatically? A: Yes, via ws.delete_run(project_id, run_id), but noted does not expose this in its UI. The intended workflow is "snapshots accumulate, tags filter" rather than "snapshots get pruned". If the volume grows large enough to matter, a retention policy based on tag + age would be the

right shape (e.g. keep all drift snapshots, keep last 30 days of data-quality). Post-demo work.

6. End-to-End Scenarios

The first four chapters cover each MLOps tool in isolation: Hydra composes configs, MLflow tracks runs, DVC versions datasets, Evidently monitors data. This chapter combines them into workflows a developer actually performs. Every scenario below is a sequence of user actions that crosses at least three of the four integrations, and each maps to a concrete workflow you can run.

6.1 Reproduce a past run

The reproducibility story is the one the project was built around. The claim is that any previous run - however old, however many template edits have happened since - can be re-executed from its archived bundle and will produce byte-identical metrics (modulo GPU nondeterminism and seeded variance that is out of noted's scope).

Preconditions. A past run exists in MLflow with a hydra/ artifact tree. For runs produced before the MlflowSource.walk() fix, the bundle may be incomplete; those cannot be reproduced reliably. Runs produced after the fix are replayable.

The sequence:

1. **Pick the target run.** Open the Composer in the target notebook. Click the mode toggle to switch to **Experiment Run**. Pick the experiment from the first dropdown and the run from the second. The dropdowns filter to runs tagged with this notebook's project id.
2. **Apply.** Clicking Apply calls POST /api/hydra/load-bundle, which fetches the run's hydra/ artifact tree into HydraCache, validates that composition reproduces the archived hash, and pre-populates the Composer with the archived group_selections and overrides. The notebook's hydra_baseline_source is now mlflow://<run_id> and the badge reads RUN xxxxxx with a green dot (no drift yet).
3. **Execute.** Click Run in the Run Manager. The prelude injects cfg composed from the archived bundle, the run:execute handler resolves cfg.data.file and injects the matching dvc.data_hash, and MLflow begins a fresh run. The prelude's silent execution of get_run_start_code() installs the metrics monkey-patch (Chapter 3.3.2).
4. **Verify.** On completion, compare the new run's test_mae (and any other metric you care about) against the original. For pure-CPU, single-threaded, seeded training, they should be byte-identical. For GPU training, tiny floating-point differences in the 5th-6th decimal are normal and indicate the config reproduction worked - the variance is in the hardware, not in the config.

What has been proved? That the configuration identity survives time. Every template file may have been edited, every group file renamed, every notebook cell restructured - and the archived bundle composed with the archived selections still produces the same resolved config, same hash, same run. This is what the noted.hydra_config_hash tag is for and why the badge surfaces drift at the first opportunity.

Failure modes.

- *Hash mismatch on load-bundle*. Composition against the archived source did not reproduce the stored hash. Most commonly: a legacy partial bundle (produced before the MlflowSource.walk() fix). Detected and surfaced as a red X on the badge. The user has to pick a different run or resign themselves to an approximate replay.
- *dvc.data_hash is missing from the archived run*. Legacy DAG runs or older Run Manager runs may not carry the dataset hash. The replay will still run, but it uses whatever cfg.data.file resolves to in the current working tree - which may be a different version of the same filename than what produced the original. The fix is to re-run dvc pull against the git commit pinned at run creation, which noted.git_commit records.

6.2 Compare two runs

The Compare panel is how a user answers “which run was better, and why?”. Two runs in MLflow are selected; noted renders a side-by-side metric diff, a side-by-side params diff, and a Hydra config diff.

The sequence:

1. **Select both runs.** Open the Registry or MLflow view and shift-click two runs in the run list, or open two Time Machine snapshots from the Composer and add them to Compare.
2. **Open the Compare panel.** The panel shows:
    - A metrics table with the delta for every overlapping metric.
    - A params table with the delta for every overlapping param.
    - A Hydra config diff: either “HASH MATCH: runs saw identical configs” or “HASH DIFFER: see diff below” with a key-by-key comparison of the two resolved.yaml files.
3. **Trace the diff.**
    - *Same hash, different metrics*. The configuration was identical, so the metric delta comes from something outside Hydra’s responsibility: GPU variance, different dataset (check dvc.data_hash), different code (check noted.git_commit), different library versions in the env.
    - *Different hash*. The configuration differed. The key-by-key diff tells you which leaf changed. Each leaf maps back to either a Composer override input or a group selection. “Differed: training.epochs: 50 -> 10” means someone overrode epochs in the Composer.
4. **Open the offending run’s Composer.** Clicking the hash diff with a run loaded navigates back to the Composer with that run as Experiment Run baseline. The

current selections and overrides are displayed, and the diff is now explorable per field.

Why this works. Because all metric/param/config comparison is grounded in MLflow's own queryable metadata plus the archived Hydra bundle. noted does not store a separate comparison index - it asks MLflow, parses the bundle, diffs the YAMLs. Two runs with identical hashes are guaranteed to have seen identical configs, so the diff UI never displays "different configs" when they were actually the same.

6.3 Promote and serve

The promote-and-serve scenario is the shortest path from "I have a better model" to "the production client is using it". It touches MLflow (registry + aliases), noted-serving (the container), and jena_client (the external caller).

The sequence:

1. **Train.** Run the notebook via Run Manager. The prelude opens an MLflow run; cell 94 trains the model; cell 116 calls mlflow.tensorflow.log_model(...) to add the model artifact to the run; cell 117 calls register_and_promote(...).
2. **Register and promote.** register_and_promote reads the current @champion's test_mae, registers the new model as version N+1, and - if new_mae < champion_mae - reassigns @champion to version N+1 via client.set_registered_model_alias(...). This is a single MLflow API call. No file moves, no redeploy.
3. **Serving health refresh.** noted-serving polls @champion on a health-check interval. The next health tick resolves the alias to the new version number. If the serving process is already holding the previous champion in VRAM, it unloads and reloads - or, if "hot-swap" is not yet implemented, the Deploy button in the Registry view surfaces the new champion with a one-click reload prompt.
4. **jena_client picks up the new champion.** The standalone jena_client demo app queries noted-serving's /predict endpoint. It has no knowledge of version numbers - it sends a payload, receives a prediction, and inverse-transforms using the target_mean / target_std that noted-serving returns alongside the prediction. When @champion hops, the next request hits the new model transparently.

What has been proved? That aliases decouple version identity from deployment state. Rolling back is client.set_registered_model_alias("Jena Weather Forecaster", "champion", 6) - done, one line, no notebook re-run, no container rebuild. This is the MLOps idiom that makes the rest of the system tolerable to operate.

Observability during the deploy. The Deploy button in the Registry view streams NDJSON from /api/serving/load: phases like starting, downloading_artifact, loading_model, ready, or failed appear in the UI as they happen. A failure at any phase surfaces the traceback inline; the button does not silently revert.

6.4 Drift investigation

The drift scenario starts from a concerning Evidently snapshot and ends at a retraining decision. It crosses all four integrations in a single narrative.

The sequence:

1. **Notice the signal.** The Data Health dot on the Data tree root is yellow or red (Chapter 5.3.3). The user opens the tooltip: “Drift flagged on 4 features.”
2. **Open Evidently.** Click the Evidently icon in the side bar. The embedded UI shows the “Jena Weather” project. Filter by tag drift. Pick the most recent snapshot.
3. **Identify drifted features.** The snapshot lists per-feature drift scores. Say T_degC and rh are flagged red, wv is yellow, p is green. Click a drifted feature to see its distribution comparison chart.
4. **Navigate back to the MLflow run.** If the snapshot is DAG-produced (Chapter 5.3.4), its metadata includes run_id. Copy that run_id into the Registry or MLflow view to open the training run. If it is notebook-produced (v0.1 gap), correlate by tag and timestamp.
5. **Inspect the run’s Hydra config.** From the run detail, open the hydra/ artifact tree. Read selections.json to see what data group was selected (jena_full_dataset or jena_2012_dataset). Read resolved.yaml to see the features and splits the run actually used.
6. **Decide.** Did the drift occur because the train split included an anomalous period? Because the feature engineering was sensitive to a rare distribution? Because the dataset has actual real-world drift? The combined view (Evidently chart + Hydra config + MLflow params + DVC data hash) is what enables the decision.
7. **Act.** Possible follow-ups:
    - Retrain on a narrower window: switch Composer to jena_2012_dataset, click Run Manager, and compare against the full-series champion.
    - Add a feature more robust to the drifted variable (e.g. a rolling mean of T_degC instead of the raw value).
    - Re-engineer the feature whose drift was a bug, not a real signal.

What has been proved? That every MLOps signal in noted is cross-navigable. Evidently drift -> MLflow run -> Hydra config -> DVC data hash -> git commit. The chain is linear and unambiguous, which is the property that lets an engineer diagnose a drift finding in minutes rather than reconstructing context from scattered logs.

6.5 Failure modes

The scenarios above assume the happy path. This section lists the non-happy paths a reader is most likely to encounter.

Deleted experiment foot-gun

MLflow's soft-delete on an experiment leaves the experiment in a weird zombie state: get_experiment_by_name(...) returns the deleted experiment, start_run fails cryptically, and the user sees RESOURCE_DOES_NOT_EXIST with no actionable guidance. The current except: pass in RUN_START_CODE masks this failure, so the symptom surfaces much later as "my metrics are not appearing in the panel".

The future fix is to (a) remove the swallowing except: pass in RUN_START_CODE, and (b) detect the zombie state at prelude time, surface a notification in the frontend, and offer "Restore experiment" or "Purge experiment" actions. Until then, the workaround is manual: open the MLflow UI, find the soft-deleted experiment, purge it from the trash, and let noted recreate it on next run.

Run All vs Run Manager

The notebook has two execution paths. **Run Manager** goes through execute_run(), which installs the full prelude: cfg injection, metrics monkey-patch, run-start hook, dataset hash logging. **Run All** (the Play-all-cells button) goes through cell:execute one cell at a time, which injects cfg (Chapter 2.3.2) but does *not* install the metrics patch. Consequence: live metrics do not stream during Run All, and the MLflow run opened inside cell 116 is different from the one the prelude would have opened.

Symptom: after Run All, the MLflow run has no epoch-level metrics (only the final log_metric calls) and the live panel stays empty. The fix is either to always use Run Manager, or to document that Run All is a "quick sanity check" path and not the tracked execution path. The recommended workaround is "always use Run Manager for training"; the engineering fix is to have cell:execute also install the prelude when a run is about to start.

Missing scaler stats

target_mean and target_std are logged as params (Chapter 3.3.5). jena_client reads them on every prediction to invert the scaling. If a run is promoted without having logged these params - e.g. a legacy run, a DAG run before the params were added - jena_client falls back to returning scaled predictions or raises a KeyError.

Symptom: jena_client returns "0.27 degrees Celsius" (a scaled value) or an error. Diagnostic: open the champion run in MLflow, check the params table for target_mean / target_std. If missing, either retrain and re-promote, or manually set the params on the run via client.log_param(run_id, ...).

DVC remote unreachable

Symptom: dvc pull fails or cfg.data.file resolves to a path that does not exist. The notebook crashes at cell 16 with FileNotFoundError.

Diagnostic: check that noted-minio is running (docker ps | grep noted-minio), that the .dvc/config points at the right endpoint, and that the working tree has the file after dvc pull. No silent fallback - the error is loud by design (Chapter 4.5).

Composer Apply on an empty selection

Earlier, clicking Apply in Experiment Run mode without first picking a run would wipe group_selections to {}. The badge would go red, subsequent runs would fail to compose, and the user would not know why.

The fix disables the Apply button until a run is selected, and CSS styles disabled buttons distinctively (gray bg, not-allowed cursor) so the state is visible. An older notebook that carries a cleared group_selections from before the fix may still need its metadata manually repaired - reopen the Composer in Local mode, pick a set of groups, and re-Apply.

Stale notebook metadata

A notebook created before the M1 refactor may carry legacy flat-format hydra_selections like {"data": "default", "model": "gru_baseline", "scaler": "standard", "training": "default"}. The Composer validates each entry against the current schema and falls back to defaults for invalid values, but the badge will go orange until the user re-Applies to refresh the metadata into nested format. Cell 95 markdown that references cfg.training.epochs will also still work because the Hydra composition applies the inlined training block from config.yaml regardless of the legacy selection.

Bundle archival racing with MLflow artifact upload

The _log_hydra_bundle_for_run fire-and-forget thread can in theory race with the notebook's own tensorflow.log_model call. In practice MLflow's artifact upload is idempotent and append-only under hydra/, so the race is benign. If a future bug surfaces where the bundle is missing from a run while the tag is present, the diagnostic is to check the _log_hydra_bundle_for_run logs for a silent exception - historically this has been the MlflowSource.walk() recursion bug (Chapter 2.3.5). Failures are logged but not re-raised.

Container restart wipes the Evidently workspace

Without the evidently-data named-volume mount, a container rebuild wiped /app/workspace inside the Evidently container. Symptom: all snapshots disappear from the Evidently UI after a docker compose up --build. The fix (the evidently-data named volume in docker-compose.yml) persists the workspace across rebuilds. No action required except to rebuild after pulling the fix. **What this proves.** Each of the four MLOps integrations is individually useful. Combined, they produce workflows whose coherence comes from three cross-cutting primitives: the Hydra hash (config identity), the DVC md5 (data identity), and the MLflow run id (execution identity). Every scenario above is a walk through the graph these three primitives build. A reader asking "how does noted tie everything together?" should notice that the answer is the graph, not a centralized coordinator.

7. Frontend Architecture

7.1 Concept primer

noted's frontend is a single-page web application written in vanilla ES modules with no framework. The design choice that deserves the most attention is the deliberate *absence* of a bundler for the application code: all JS files are served directly to the browser as ES modules, and vendor libraries (CodeMirror, Wunderbaum, jsPanel, socket.io, KaTeX, echarts) are loaded via classic script tags from frontend/vendor/. The result is a zero-build dev loop - edit a file, hard-refresh the tab, see the change - which is the primary reason a project-scoped, rapidly iterated tool chose vanilla over React or Vue.

The shell layout is VS Code-shaped for a reason: the target user is a developer, the target task is "edit code, run it, inspect outputs", and the Code-style layout (icon bar + sidebar + tabs + right panel + status bar) is a well-understood mental model for that task. noted does not try to hide that it is a developer tool.

Four libraries do most of the heavy visual work: **Wunderbaum** for trees (Explorer, Data Catalog, Model Registry), **jsPanel** for floating/undockable windows, **CodeMirror 6** for every code editor (including every notebook cell), and **socket.io** for every backend event stream. Together they explain 80% of the frontend's surface area.

7.2 Entry point and module graph

frontend/index.html (121 lines) is the entry document. Lines 101-116 load 18 vendor scripts via classic <script> tags: socket.io, marked, hljs, jsPanel, KaTeX, Wunderbaum, xterm, notyf, echarts, dagre. Line 119 is the one and only ES-module entry point:

```
<script type="module" src="static/js/app.js"></script>
```

The /static/ prefix is a FastAPI mount (backend/app/main.py:1202) that points at /frontend/ on disk. There is no dist/ directory. Every .js file under frontend/js/ is the exact file the browser fetches.

frontend/js/app.js is the entry module. Lines 1-50 import the top-level classes: KernelClient, NotebookEditor, ChatPanel, ExplorerPanel, GitPanel, TabBar, MenuBar. The App class constructor calls six initialization helpers in sequence (lines 43-49):

```
initStatusBar(this);
initMenuCommands(this);
initChat(this);
initFileEditors(this);
initNotebooks(this);
initTabs(this);
```

Each helper registers panel-specific event handlers onto the singleton app instance. The app object is the central bus - every panel has a reference to it, every command is

dispatched through it, and its maps (_notebookEditors, _fileEditors, _documentTabs, _undockedPanels) are the canonical state of which UI surfaces are alive.

No framework. No virtual DOM, no reactivity, no component lifecycle. State is mutable; DOM updates are direct; events are explicit. The trade-off: every update path is visible in the code (a feature), but sharing UI between panels requires explicit wiring (a cost).

7.3 Layout: the VS Code-style shell

frontend/css/base.css:93-179 defines the top-level layout with flexbox:

- #app - flex column, full viewport height.
- #menu-bar, #toolbar, #info-bar - stacked at the top.
- #below-bar - flex row taking remaining height.
- #icon-bar - left vertical strip (~50 px), always visible.
- #sidebar-panel - Explorer / Git / TOC / Settings, collapsible.
- #content-area - the center, holds the tab bar and the active tab's content.
- #right-panel - Run Manager, Chat, or Doc view, collapsible.
- #status-bar - bottom bar (20 px, dark theme), always visible.

CSS is split across ~27 feature-scoped files (cell.css, notebook.css, tab-bar.css, icon-bar.css, sidebar.css, right-panel.css, ...) that all import from base.css. No CSS-in-JS, no preprocessor, no utility framework - plain CSS with a conventional BEM-ish class naming.

The layout's load-bearing affordance is that *every* content area (#sidebar-panel, #content-area, #right-panel) is independently resizable and collapsible. This is what lets the user dedicate most of their viewport to the thing they are focused on - a notebook cell, a Compose panel, a chat thread - without losing quick access to the others.

7.4 Wunderbaum trees

Wunderbaum is a vanilla-JS tree library bundled into frontend/vendor/wunderbaum/wunderbaum.umd.min.js. It exposes mar10.Wunderbaum on the global namespace; noted does not import it as an ES module.

Tree instances are created throughout the Explorer panels with a common pattern. ExplorerPanel.js:515-552 is a representative example:

```
this._tree = new mar10.Wunderbaum({
  element: treeEl,
  source: source,
  adjustHeight: false,
  selectMode: 'single',
  checkbox: false,
```

```
  icon: true,
  iconMap: FA_ICON_MAP,
  render: (e) => {...},
  lazyLoad: (e) => {...},
  activate: (e) => {...},
  click: (e) => {...},
  dblclick: (e) => {...},
});
```

Key conventions:

- source is either a static array (for known-at-build-time trees) or an async function that returns children on demand (for lazy-loaded trees).
- lazyLoad returns a promise resolving to the children when a user expands a node.
- activate is the primary click handler; dblclick handles the “open in tab” action.
- noted *never* calls resetLazy() on static-children nodes - a hard-learned lesson documented in memory. For dynamic trees, the correct refresh is addChildren() after removing the old children.

The Explorer tree has nodes for Projects, Git, TOC, Docs, Hydra, MLflow, DVC, Evidently, Serving. Each has its own view module under frontend/js/panels/explorer/Explorer*Views.js that handles rendering the detail pane when a node is activated.

7.5 jsPanel and the TabBar

TabBar (frontend/js/TabBar.js:1-282) manages tabs above the content area. Tabs are stored in a Map keyed by a stable key (e.g. notebook:<project>:<file>, doc:<category>:<name>, pyfile:<project>:<filename>). Tab properties include closable, preview, undockable, and an undocked flag.

Preview tabs (lines 55-57) replace the previous preview tab instead of stacking. This is the VS Code “single click = preview, double click = pinned” pattern. A preview tab is promoted to permanent when the user edits it or explicitly pins it.

Undocking (frontend/js/app-tabs.js:855-940) is implemented via jsPanel. The flow:

1. User clicks the undock icon on a tab.
2. app-tabs.js calls jsPanel.create({...}) with panelSize: '70vw 70vh', centered with offset, addCloseControl, boxShadow: 3.
3. The tab’s DOM element is *moved* (not cloned, except for canvas-heavy PDF tabs - see below) into the panel’s content area.
4. The tab bar shows the tab as undocked (greyed out or with an icon indicator).

5. The panel's onclosed callback re-docks unless the user clicked close with the _docking flag cleared.

A custom dock button is added to the jsPanel header (lines 922-936) so the user can re-dock without closing.

The PDF-undock bug was a reminder that cloneNode(true) does not copy canvas pixel data. After cloning a PDF tab's DOM into a jsPanel, every page's canvas was blank. The fix: after cloneNode, walk the canvases in the original and the clone in parallel and blit pixels with ctx.drawImage(orig, 0, 0). The same principle applies to any tab that holds imperative canvas state - cloning the DOM is necessary but not sufficient; the bitmap has to be explicitly copied.

7.6 CodeMirror 6 and the cell editor

Every code-edit surface in noted is a CodeMirror 6 instance. Unlike everything else in the frontend, CodeMirror *is* bundled - it has enough internal module structure that shipping its loose ESM would require ~30 HTTP requests per editor.

frontend/js/CellEditor.js:1-78 is the wrapper. Imports from the bundled codemirror.bundle.js:

- EditorView, EditorState, keymap
- Gutters: lineNumbers, lintGutter, highlightActiveLine
- Languages: python, javascript, markdown (plus YAML/JSON via legacy-modes)
- Themes: ayuLight, clouds, espresso, smoothy, tomorrow, oneDark
- autocompletion, syntaxHighlighting

The bundle is built via esbuild at scripts/build-codemirror/ and checked into the repo. package.json in that dir lists the exact dependencies. Rebuilding is a one-line npm run build and only happens when a new language or extension is added.

Theme switching uses CodeMirror's Compartment (line 64) so the theme can be reconfigured live without recreating the editor. Each cell gets its own editor; the cell type (code vs markdown) decides which language extension is mounted.

LSP integration is wired via codemirror-languageserver (dependency in scripts/build-codemirror/package.json:27). When a Python or R cell is focused, the editor connects to the backend's LSP proxy, which forwards requests to the language-specific LSP server (Pyright for Python, R LSP for R). Diagnostics render in the lint gutter; completions come through the autocomplete extension.

7.7 Panels and tabs: the lifecycle

There is no central panel registry. Panels are created *on demand* when their corresponding action fires and are tracked via app._* maps for lookup on re-open.

The typical lifecycle:

1. **User action** - click a tree node, a menu item, or a button.
2. **Handler in app-*.js** - checks if a tab for this resource already exists (app._notebookEditors.get(key), etc.).
3. **Create or focus.** If it exists, tabBar.activate(key). If not, instantiate the panel class (NotebookEditor, DocumentViewer, ...) and call tabBar.addTab(key, title, element, opts).
4. **Activation handler** (onActivateTab registered with TabBar) shows the activated tab's DOM element and hides the previous one.
5. **Close.** Either via the tab's X button or a close-all command. The panel's destroy() method (if any) releases listeners and DOM; the entry is removed from the app._* map.

Panel types currently in use:

- NotebookEditor - a full notebook with cell editors, toolbar, metrics bar.
- FileEditor - a single file edited as a CodeMirror buffer (non-notebook code).
- MediaViewer - images, video.
- DocumentViewer - markdown / PDF, rendered via marked + pdfjs.
- ChatPanel - the AI assistant conversation.
- ExplorerPanel - the left sidebar tree.
- Service tabs - iframe wrappers for MLflow, Airflow, MinIO, Evidently.

The deliberate absence of a framework means every panel has an unambiguous owner and an unambiguous cleanup path.

7.8 Key dispatchers and shortcuts

MenuBar.js:316-382 installs a global keydown listener. It parses modifier+key combinations (Ctrl+S, Ctrl+Shift+F, F12, ...) and looks them up in a shortcutMap populated from frontend/menu.json.

frontend/menu.json is the canonical source of truth for menu items, labels, shortcuts, and their command ids. Example entries:

```
{"id": "file.save", "label": "Save", "shortcut": "Ctrl+S"},
{"id": "edit.findReplace", "label": "Find and Replace", "shortcut": "Ctrl+H"},
{"id": "edit.formatDocument", "label": "Format Document", "shortcut": "Ctrl+Shift
+F"},
{"id": "edit.goToDefinition", "label": "Go to Definition", "shortcut": "F12"}
```

Commands are looked up in a registry populated by initMenuCommands and executed via executeCommand(id).

The CodeMirror guard (MenuBar.js:363-371) excludes standard editing shortcuts (Ctrl+Z/X/C/V/A) when the focused element is inside a CodeMirror editor. This prevents the menu system from stealing shortcuts that the editor handles natively.

Cell execution shortcuts (Shift+Enter for run, Ctrl+Shift+Enter for debug) are *not* in the MenuBar - they live on the NotebookEditor because they are cell-scoped rather than app-scoped. app-notebooks.js:896-911 also installs debug keys (F5 continue, Shift+F5 stop, F10/F11 step) scoped to the active notebook.

7.9 Status bar, icon bar, menu bar

frontend/js/app-status-bar.js is the status bar. On startup it fetches /api/system/info and populates pills: Host OS (golden), Container OS (green), Python version, branch, project, pipeline status. The Problems indicator counts diagnostics. Cursor info (Ln 42, Col 5) updates on cell focus. A socket listener on pipeline:status updates the pipeline pill live.

frontend/js/IconBar.js:7-154 is the left vertical icon strip. Two groups separated by a flex spacer:

- **Top**: Projects, Git, TOC, Assistant, Debug, Docs.
- **Bottom**: Airflow, MLflow, MinIO, Evidently, Settings.

Each icon is either SVG or a FontAwesome glyph. Click delegates to app._onIconBarClick(key), which either toggles the sidebar to the matching view or opens a service tab in the content area.

frontend/js/MenuBar.js renders the top menu bar from menu.json. File, Edit, View, Terminal, Help menus. Alt+F/E/V/T/H toggles the corresponding dropdown via keyboard. Menu items that lack a shortcut are click-only.

7.10 Socket.io and the event surface

frontend/js/KernelClient.js:1-80 initializes socket.io. The connect() method derives the socket path from the page URL (so the frontend works behind any reverse proxy) and sets transports to ['websocket', 'polling'] with 10 reconnection attempts.

Events consumed by the frontend:

- Connection: connect, disconnect, connect_error.
- Notebook: notebook:state, notebook:saved.
- Cell: cell:updated, cell:added, cell:deleted, cell:moved, cell:output, cell:execute_start, cell:execute_complete, cell:lock_changed, cell:diagnostics.
- Kernel: kernel:status.
- Users: user:joined, user:left.
- Runs: run:started, run:complete.
- Metrics: metrics:update (from the MLflow monkey-patch - see Chapter 3.3.2).

- Pipeline: pipeline:status, pipeline:task_status.

A custom on(event, callback) emitter (lines 194-215) lets panels subscribe to specific events without knowing about socket.io directly. The KernelClient acts as a pub/sub intermediary so that replacing socket.io in the future would not require touching every panel.

7.11 Discussion-ready talking points

Q: Why no framework? A: Because the project's iteration velocity depends on a zero-build dev loop. Adding React or Vue would require a bundler, a dev server, a source-map pipeline, and a mental model of component lifecycles. For a single-developer, rapidly iterated project, vanilla ES modules plus three or four well-chosen vendor libraries is strictly simpler. The cost is more explicit wiring; the payoff is that every update path is inspectable by reading the code rather than reasoning about a framework's abstractions.

Q: Why bundle CodeMirror but not the rest? A: Because CodeMirror 6 is internally modular to the point of being un-ship-able as loose files (~30 transitive imports per editor). Bundling it once into codemirror.bundle.js avoids 30 HTTP round-trips per notebook load. Wunderbaum and jsPanel ship as pre-bundled UMD files; socket.io and KaTeX have minified builds. The application code itself is cheap to load (native ES modules + HTTP/2 multiplexing) so there is no bundling win.

Q: How does a new panel get added? A: (1) Write the class at frontend/js/panels/YourPanel.js. (2) Import and instantiate it in an initXxx(app) helper in app-*.js. (3) Register the tab via app._tabBar.addTab(key, title, element, opts). (4) Handle activation/deactivation in onActivateTab. No framework ceremony; no manifest; no registration map. If you want it to appear in the icon bar or menu, add it to IconBar.js or menu.json.

Q: Why is jsPanel used for undocking but not for the main layout? A: Because the main layout is fixed, predictable, and always visible - flexbox is the right primitive. jsPanel is designed for floating, draggable, resizable windows that the user summons on demand. Using it for the main chrome would add unnecessary state (position, size, z-index) to surfaces that should not be moved.

Q: How does the frontend stay in sync with backend state? A: Through socket.io events that are *authoritative* for state transitions and *advisory* for polling. When a cell executes, the backend emits cell:execute_start and cell:output events; the frontend's NotebookEditor subscribes to these and updates the DOM directly. State that does not change often (project list, document catalog) is fetched via REST on demand and cached in memory. The rule: anything that could change during a user's session without them initiating it is delivered via socket.io; anything that only changes when the user clicks a thing is REST.

Q: Why vanilla CSS instead of Tailwind or a component library? A: Same reason as no framework. Tailwind adds a build step; a component library locks in visual decisions. noted's visual language is small and consistent enough that ~27 hand-written CSS files,

organized by feature, are easier to reason about than an atomic-class system. The themes are swappable via CSS custom properties, which is all the theming surface noted needs.

Q: What happens when socket.io disconnects? A: The KernelClient attempts 10 reconnects with exponential backoff. The status bar's connection pill turns red. Cell execution commands are buffered in the frontend and replayed on reconnect. Long-running execution results that were in-flight when the socket dropped are re-requested on reconnect via a backfill query. Silent degradation is explicitly avoided - a stale state is always visible to the user.

Q: Is the frontend tested? A: Not yet. The project is in its early iteration phase; the test pyramid is backend-first (see Module 8). Adding Playwright or Cypress for frontend E2E tests is in the backlog. The trade-off has been accepted: fewer safety nets, faster iteration.

8. Backend Architecture

8.1 Concept primer

noted's backend is a **FastAPI + socket.io + jupyter_client** stack running a single Python process. FastAPI serves REST endpoints over standard HTTP; socket.io handles bidirectional events over WebSocket; jupyter_client manages Python (and R) kernels via ZMQ. The three layers are composed inside a single ASGI app (socketio.ASGIApp wrapping the FastAPI app) so the whole service runs from one uvicorn command.

The architectural style is **manager-oriented**. Routing is thin - each router file has 50-200 lines and mostly delegates to a manager class under backend/app/managers/. The managers own state: the kernel sessions, the notebook files, the Hydra cache, the MLflow tracking URI, the DVC file catalog, the git working-tree knowledge. Routes are stateless functions that call into managers.

This separation is what makes the codebase readable at scale. There are 45 manager files, each ~100-500 lines, each owning one responsibility. No central ServiceLocator, no dependency injection framework - managers are singletons held on the FastAPI app object and reached via module-level helpers.

8.2 FastAPI app setup

backend/app/main.py is the entry module. It imports every router, instantiates the socket.io server, wires startup/shutdown hooks, and mounts static files.

Lines 30-34 create the socket.io server:

```
sio = socketio.AsyncServer(
    async_mode="asgi",
    cors_allowed_origins="*",
    max_http_buffer_size=100 * 1024 * 1024,  # 100 MB
)
```

The 100 MB buffer is load-bearing: cell outputs can carry large figures (matplotlib PNGs embedded as base64), chart renderings, or streamed log output. A smaller buffer silently drops messages.

Line 128 instantiates the FastAPI app with a lifespan context manager (lines 86-125) that:

- Starts the KernelManagerService background tasks.
- Pre-warms MLflow's tracking URI resolver.
- Initializes the MCP session manager (for the AI assistant's tools).
- Creates the Examples/Welcome.ipynb notebook if none exists.

Line 1257 wraps the FastAPI app as ASGI:

```
app = socketio.ASGIApp(sio, other_asgi_app=app)
```

This lets uvicorn app.main:app serve both HTTP (FastAPI) and WebSocket (socket.io) from a single process.

Middleware. Deliberately thin. Socket.io handles CORS internally. There is no JWT/OAuth middleware; secrets are validated at event or endpoint level (Section 8.9). Line 1202 mounts the frontend at /static and /wallpapers.

8.3 Routers

backend/app/routers/ contains 22 files, each an APIRouter mounted under /api in main.py lines 129-154. One-line summaries:

Router	Responsibility
notebooks	Notebook CRUD, project listing, cell-level operations
venvs	Python virtualenv management per project
documents	Knowledge Base documents catalog and file serving
git	Git status, branches, commits, diffs for project trees
files	Generic filesystem browse/upload (500 MB limit)
dvc	DVC status, file history, checkout
minio	MinIO bucket/object ops
projects	Project metadata and registration
mlflow	MLflow experiments/runs/registered models/Logged Models
export	Notebook export formats (HTML, PDF, .py)
hydra	Config schema, compose, experiments, runs, load-bundle
airflow	DAG listing, run triggers, status, logs
snapshots	Per-notebook local snapshot versioning
registry	Project registry + noted-wide metadata
serving	Model serving lifecycle (load, unload, health, predict)
reports	Generated report documents
graph_proxy	Knowledge Graph service proxy
llm	AI assistant API (uses NOTED_TERMINAL_SECRET auth)
lsp	LSP proxy (completion, diagnostics, goto-def)
dap	DAP proxy (debug adapter protocol)
evidently	Evidently health endpoints (projects, data-health, drift-status)
file_debug	Single-file debugger

Every router follows the same shape: a router = APIRouter(prefix="/api/X", tags=["X"]), a handful of endpoint functions that parse inputs, call a manager method, and return a Pydantic model or a dict. Thin by design.

8.4 Socket.io server

The socket.io event surface is the bidirectional contract between frontend and backend. main.py is the handler file; it registers one @sio.on handler per event, each delegating to managers.

8.4.1 Events the backend consumes

- **Connection:** connect (line 187), disconnect (line 192, schedules 15-second cleanup to tolerate brief network drops).
- **Notebook lifecycle:** notebook:open (line 224, joins a room named after the notebook, loads state, returns kernel status), notebook:close (line 675), notebook:save (line 692), notebook:relint (line 717).
- **Collaborative editing:** cell:lock/cell:unlock (lines 745/758), cell:update/cell:add/cell:delete/cell:move (lines 767-800). Notebook mutations broadcast to the room.
- **Execution:** cell:execute (line 809), run:execute (line 831).
- **Kernel control:** kernel:start/kernel:stop/kernel:restart/kernel:interrupt (lines 928-1086), heartbeat (line 1094).
- **Terminal:** terminal:auth (line 1106), terminal:start (line 1117), terminal:input (line 1152), terminal:resize (line 1170), terminal:kill (line 1190).

8.4.2 Events the backend emits

- **State:** notebook:state, notebook:saved, kernel:status.
- **Cells:** cell:updated, cell:added, cell:deleted, cell:moved, cell:lock_changed, cell:execute_start, cell:execute_complete, cell:output, cell:diagnostics.
- **Runs:** run:started, run:complete, metrics:update (from the MLflow monkey-patch).
- **Pipeline:** pipeline:status, pipeline:task_status (Airflow DAG progress).
- **Terminal:** terminal:output, terminal:exit, terminal:auth_ok, terminal:auth_failed.
- **Errors:** error with codes like NO_KERNEL, NOT_FOUND, LOCKED.

8.4.3 Routing IOPub messages to cell handlers

The interesting flow is the cell output dispatch. Here is the chain:

1. A notebook cell is executed via cell:execute. The backend calls ExecutionBridge.execute_cell().
2. That method sends an execute_request ZMQ message to the kernel via kc.execute(code) and records the msg_id in _pending[session_id][msg_id] = handler.
3. Meanwhile, an _iopub_loop() task (execution_bridge.py:485) polls kc.get_iopub_msg() continuously.

4. Each IOPub message has a parent_header.msg_id - the id of the request that caused it. _dispatch_iopub_msg() (line 525) looks up the handler by that id.
5. The handler processes the message by type and emits a cell:output event to the notebook's socket.io room.

This is what makes the frontend's live output rendering work. Kernels push on their own schedule; the backend multiplexes the push stream into per-cell event streams that the frontend subscribes to.

8.5 KernelManager

backend/app/managers/kernel_manager.py is the kernel lifecycle owner. Two classes:

- KernelSession (lines 13-42): a dataclass holding session_id, kernel_manager (from jupyter_client), kernel_cmd, language, project_id, notebook_path, client_sid, status, debug state, and a _cached_client.
- KernelManagerService (lines 45-547): manages all sessions.

8.5.1 Starting a kernel

start_kernel() (line 67) spawns a new kernel process:

1. Resolve the project path via ProjectRegistry.
2. Build the environment (PYTHONPATH, GPU libs, seed env vars).
3. Pick the kernel command based on the notebook's language metadata.
4. Call JupyterKernelManager.start_kernel(cwd=project_root, env=env).
5. Create and cache a client (line 170): kc = km.client().

Why cache the client? This is the memory-documented ZMQ identity gotcha. km.client() creates a new ZMQ connection with a fresh identity each time. Calling it twice in the same session produces two clients that both register with the kernel but only one can own the shell channel at a time. The race is silent - the second client's messages are delivered intermittently depending on which client the kernel's round-robin picked. The fix: eagerly create one client per session on kernel start, cache it on the session object, and always return the cached instance.

get_kernel_client() (line 480) implements the pattern: fast-path returns session._cached_client if its channels are running; slow-path creates a new client under an async lock to prevent concurrent creation during recovery.

8.5.2 Stopping, restarting, heartbeat

- stop_kernel() (line 405): kill the process, clean up channels.
- restart_kernel() (line 428): reuse the session_id but restart the process; the _cached_client is refreshed.

- heartbeat() (line 511): update last_heartbeat on a session. Used by an idle-timeout reaper that stops kernels after N minutes of inactivity.
- init_debugpy() (line 209): enables debugpy on demand and captures the listen port for DAP proxying.

8.6 ExecutionBridge

backend/app/managers/execution_bridge.py bridges socket.io events with the kernel's ZMQ channels. Its public surface is small; most of the file is dispatch logic.

Public methods:

- execute_cell(session_id, cell_id, code, ...) (line 88) - single-cell execution. Injects the Hydra prelude if the notebook has hydra_config (Chapter 2.3.2), wraps JavaScript cells in IIFE for re-runnability, returns when the kernel's execute_reply arrives.
- execute_run(session_id, project_id, cells, ...) (line 277) - Run Manager path. Silently injects get_run_start_code() before cells, then executes each in sequence, then injects get_run_end_code(). All cells share one MLflow run.
- stop_iopub_listener(session_id) (line 680) - stops the async task for a session; called on notebook close.

The _iopub_loop polls the ZMQ channel in an executor thread (because get_iopub_msg is blocking), wraps each message into an async callback, and delivers it to _dispatch_iopub_msg. That dispatcher is the single place that interprets IOPub message types (stream, display_data, error, execute_input, execute_reply) and translates them into cell:output, cell:execute_complete, or metrics:update socket.io emissions.

8.7 NotebookManager

backend/app/managers/notebook_manager.py:11 is NotebookManager. It is the filesystem-facing interface for .ipynb files.

Key methods:

- get_notebook(project_id, notebook_name) - loads from disk with path-traversal validation at line 20.
- update_notebook(...) - persists a dict to disk via json.dump.
- create_notebook(...) / delete_notebook(...) - file ops.
- create_project(...) / list_projects(...) - directory ops.
- ensure_welcome_notebook() (line 62) - bootstraps Examples/Welcome.ipynb at startup.
- _notebook_path(project_id, notebook_name) (line 17) - resolves to an absolute path via ProjectRegistry, rejects traversal.

Cells are not a separate entity in the backend - they are Python dicts inside the notebook's JSON. Mutation methods (add_cell, update_cell, delete_cell) modify the in-memory dict and persist the full notebook. Cell-level versioning is not supported; the unit of versioning is the notebook itself, via snapshots router + snapshot_manager.py.

8.8 ProjectRegistry

backend/app/managers/project_registry.py is the project discovery layer. Two sources:

1. **Internal projects** - subdirectories of data/projects/.
2. **Mounted projects** - external paths referenced by the user via data/NOTED.md frontmatter.

data/NOTED.md uses YAML frontmatter:

```
---
mounts:
  - name: "jena_weather"
    host_path: "/mnt/data/jena_weather"
  - name: "jena_client"
    host_path: "/mnt/data/jena_client"
---
```

Line 79 parses this frontmatter on each backend startup. Each mount becomes a project with source=mount, path=<resolved-mount-path>, host_path=<absolute-host-path>.

Key methods:

- resolve(project_id) (line 90) - project_id -> filesystem path. Strips the legacy __mount__: prefix on line 146 so older metadata keeps working.
- list_projects() (line 121) - all projects with metadata.
- is_internal() / is_mount() (lines 128 / 134) - type checks used by routers.

The mount resolution is performed at app startup by docker-compose.mounts.yml (auto-generated from NOTED.md) which Docker applies as additional bind mounts. The frontend's project list and the container's mount list agree because both derive from the same YAML source.

8.9 Auth and secrets

Auth is thin and event-scoped:

- **NOTED_TERMINAL_SECRET** (env var, lines 1106-1130 of main.py). If set, socket.io terminal:auth events require a matching value. If not set, terminal auth succeeds silently (dev mode). The same secret is used by routers/llm.py for its endpoints.

- **ANTHROPIC_API_KEY** (env var, loaded at anthropic_llm_manager.py:1). If set, Claude models are selectable in the AI assistant. If not set, only the local Gemma model works.

There is no JWT, OAuth, or session cookie layer. The default deployment assumes a single-user, local-network context. Multi-user / multi-tenant deployments require fronting the app with a reverse proxy that handles auth (e.g. oauth2-proxy + nginx).

8.10 Testing

tests/ exists and is organized as:

- tests/api/ - FastAPI endpoint tests numbered test_01_setup.py through test_31_nice_to_have.py.
- tests/e2e/ - Playwright browser automation tests.
- tests/kernel_tests/ - kernel-specific tests.
- tests/conftest.py (489 lines) - shared pytest fixtures: FastAPI test client, temporary project/notebook fixtures, kernel session management, socket.io connection helpers.

tests/pytest.ini configures testpaths = api e2e, markers api / e2e / socketio / slow, 120-second timeout, asyncio_mode = auto.

A Dockerfile + docker-compose.test.yml exist for CI-style containerized runs. The test pyramid is backend-heavy; frontend E2E coverage is minimal pending future investment.

8.11 Discussion-ready talking points

Q: Why a single ASGI app instead of separate FastAPI and socket.io processes? A: Because both layers need the same in-process state: the KernelManagerService, the cell output dispatch tables, the Hydra cache, the manager singletons. Splitting them would require IPC and a shared state backend (Redis?) that neither layer currently uses. The single-process design is fine for noted's scale (one user, one host); scaling out would first require extracting the kernel layer to a dedicated service.

Q: Why are managers singletons on the app object instead of DI'd into routers? A: Because the dependency graph is shallow and stable. Every manager is initialized once at startup; every router has access to every manager via from app.managers.xxx_manager import xxx_mgr. DI would add a dependency-scope concept (request / session / app) that noted does not need. The cost is that tests have to mock at the module level rather than override a provider; the benefit is that the code is simpler to read.

Q: Why does the backend poll the IOPub channel in a thread instead of using asyncio natively? A: Because jupyter_client's IOPub receive is blocking and does not expose an awaitable. Running it in loop.run_in_executor(...) keeps the main event loop responsive. The alternative (a fully async jupyter_client) does not exist as a maintained library. This is

the kind of integration trade-off that justifies keeping the ExecutionBridge layer thin and focused on the channel bridge.

Q: What is the unit of failure isolation? A: The kernel session. Each notebook has its own kernel process; a crash in one does not affect others. ExecutionBridge and KernelManagerService hold per-session state in dicts keyed by session_id. The backend process itself is a shared dependency - a crash there takes down all sessions. Restart resilience is owned by the compose restart: unless-stopped policy, not by the backend itself.

Q: Why is the project registry derived from NOTED.md rather than the backend's database? A: Because there is no backend database. noted is a filesystem-first tool - the working directory is the source of truth for projects, notebooks, configs, and data. NOTED.md sits alongside other project files and is human-editable. A database would move the source of truth out of the filesystem and make git diff less informative. The trade-off: scaling to hundreds of projects with complex metadata would eventually want a database; at noted's current scale, plain YAML is the right shape.

Q: How does the backend tolerate a network drop that kills a socket.io connection? A: disconnect schedules a 15-second grace cleanup (main.py:192). If the same client reconnects within that window, the cleanup is cancelled and the session resumes. If not, locks are released, rooms are cleaned, and the client is considered gone. Kernel sessions continue to run - they are not tied to socket.io lifetime - so a reconnecting user rejoins their running kernel with all state intact.

Q: What about scaling beyond a single machine? A: Not supported as-is. The kernel processes are local, the socket.io rooms are in-process, the ProjectRegistry reads from a single filesystem. Horizontal scaling would require extracting kernels to a dedicated service (the long-term Serving refactor Phase 0b is a first step in that direction), routing socket.io through Redis, and treating projects as a networked resource. None of this is in the current scope; it is future infrastructure work.

9. Notebook Execution Flow

9.1 Concept primer

Notebook execution in noted is the path a user's Shift+Enter press takes from a browser keystroke to a rendered output in the cell. This module traces the path end-to-end. Prior modules covered each stopping point in isolation: Chapter 2 described the Hydra injection, Chapter 3 described the MLflow prelude, Chapter 7 described the frontend's event dispatch, Chapter 8 described the backend's routers and managers. This module is the glue - a time-ordered walk through what actually happens when a cell executes.

There are two distinct execution paths in noted:

1. **Cell execution** - single cell, via cell:execute socket event. Triggered by Shift+Enter, the Play button on the cell toolbar, or a programmatic call from the AI assistant. This path does **not** open an MLflow run and does **not** install the metrics monkey-patch by default.
2. **Run execution** - a sequence of cells wrapped in an MLflow run, via run:execute socket event. Triggered by the Run Manager's Run button. This path installs the full prelude (Hydra injection, MLflow run-start, metrics patch, DVC hash logging) and runs all specified cells inside a single active run context.

Both paths converge on the same ZMQ execute_request call into the kernel. The divergence is only in what gets injected before the user's code runs.

9.2 Cell execution path

Step 1 - Frontend keydown

The user focuses a CodeMirror cell editor and presses Shift+Enter. The NotebookEditor class (frontend/js/NotebookEditor.js) has a keydown listener on the editor container that intercepts this combination, captures the cell's current code, and calls app._kernelClient.executeCell(cellId, code, opts).

Step 2 - Socket emit

KernelClient.executeCell() (frontend/js/KernelClient.js) emits a cell:execute event with payload:

```
{
  notebook_id: <current notebook>,
  session_id: <current kernel session>,
  cell_id: <cell being executed>,
  code: <cell content as string>,
  hydra_config: <null or {notebook_uid, baseline_source, group_selections, overri
des}>,
```

```
  debug: <false or true>
}
```

hydra_config is present when the notebook has a Hydra baseline; its composition identity (Chapter 2.3.3) is the *only* place the frontend needs to assert the Hydra state per cell execute - the backend re-composes from scratch on every request.

Step 3 - Backend handler

backend/app/main.py:809 is the handler. It:

1. Looks up the kernel session by session_id. If missing, emits error: NO_KERNEL and returns.
2. Updates the session's last_heartbeat.
3. Calls ExecutionBridge.execute_cell(session_id, cell_id, code, hydra_config=..., debug=...).

Step 4 - Hydra injection (if present)

Inside ExecutionBridge.execute_cell() (backend/app/managers/execution_bridge.py:88), if hydra_config is not None, _build_hydra_injection() is called. It:

1. Calls HydraManager.compose_from_source(...) to produce the resolved config.
2. Serializes it to JSON.
3. Constructs the Python prelude string (cfg = OmegaConf.create({...}), __noted_hydra_hash__ = 'sha256:...').
4. Executes it silently via _execute_silent() (execution_bridge.py:411), which sends a shell request and waits for the reply without surfacing any output.

The prelude returns before the user's cell code runs. By the time the cell body begins, cfg is available in the kernel's global namespace.

Step 5 - User code execution

The user's cell code is sent to the kernel via kc.execute(code, ...). This returns a msg_id. The ExecutionBridge records _pending[session_id][msg_id] = handler where handler is an object that accumulates output, tracks the execute_reply, and knows the cell_id to emit events against.

The kernel now begins executing. ZMQ IOPub messages flow back: execute_input (the code that was actually executed), stream (stdout/stderr), display_data (rich outputs), error (tracebacks), and finally execute_reply on the shell channel.

Step 6 - IOPub dispatch

_iopub_loop() (execution_bridge.py:485) polls kc.get_iopub_msg() in an executor thread. Each received message is handed to _dispatch_iopub_msg (line 525), which:

1. Extracts parent_header.msg_id from the message.
2. Looks up the handler in _pending[session_id][msg_id].
3. Switches on message type:
    - execute_input -> extract execution_count, forward to handler.
    - stream -> emit cell:output event to the notebook room with {type: 'stream', text, cell_id}.
    - display_data -> check for custom mime types (application/x-noted-metric, application/x-noted-run-start - see Chapter 3.3.2 and 2.3.3), otherwise emit cell:output with {type: 'display_data', data, cell_id}.
    - error -> emit cell:output with {type: 'error', traceback, ename, evalue, cell_id}.

Step 7 - Completion

When the shell channel's execute_reply arrives, the handler's done event is set. The handler's accumulator now has the final execution_count. The bridge emits cell:execute_complete to the room and removes the handler from _pending[session_id].

Step 8 - Frontend render

The frontend's NotebookEditor subscribed to cell:output and cell:execute_complete. As events stream in:

- cell:output events append to the cell's output area in DOM order. The renderer picks the representation based on mime type: text for stream, <img> for image/png, with marked for text/markdown, KaTeX for LaTeX, echarts for chart JSON.
- cell:execute_complete sets the execution counter in the cell gutter (In [N]) and transitions the cell state from running to idle.

9.3 Run execution path

The Run Manager path is structurally similar but has a longer prelude and executes multiple cells inside a shared MLflow run context.

Step 1 - Frontend Run click

The user opens the Run Manager panel (right sidebar), picks a set of cells (or accepts the default of all code cells), optionally overrides Hydra inputs, and clicks Run. RunManagerPanel.js emits run:execute with payload:

```
{
  notebook_id, session_id,
  cells: [{cell_id, code}, ...],
  hydra_config: {...},                  // same shape as cell:execute
  experiment_name: 'Jena Weather',
```

```
  run_name: 'gru_baseline_jena_2012',
  dataset_hashes: {...}                // ignored for Hydra-using notebooks
}
```

Step 2 - Backend handler

backend/app/main.py:831 dispatches to ExecutionBridge.execute_run(session_id, project_id, cells, ...) (execution_bridge.py:277).

Step 3 - Resolve dataset hashes

Before touching the kernel, main.py (line 673-749, on_run_execute) resolves dataset_hashes:

- If hydra_config is present, compose the cfg, read cfg.data.file, look it up in dvc_mgr.status(), and use {cfg.data.file: hash} as the single-entry dataset hashes dict. The frontend's datasets[] is ignored.
- If no hydra_config, fall back to the frontend's datasets[] as a list of file paths, resolved via the same DVC lookup.

This is Chapter 4.3.3's logic - Composer is the single source of truth for Hydra-using notebooks.

Step 4 - Prelude injection

ExecutionBridge.execute_run() calls AutoInstrumentation.get_run_start_code(experiment_name, run_name, dataset_hashes=..., hydra_hash=...) and silently executes the result.

The prelude is a single Python blob composed of:

- METRICS_HOOK_CODE - monkey-patches mlflow.log_metric, mlflow.log_metrics, and mlflow.start_run to emit display_data with custom mime types.
- RUN_START_CODE - calls mlflow.set_experiment(experiment_name) and mlflow.start_run(run_name=run_name), stores the run handle as run.
- _get_dataset_logging_code(dataset_hashes) - calls mlflow.log_param("dvc_data_hash", hash) and mlflow.set_tag("dvc.data_file", path) for each entry.
- _get_hydra_logging_code(hydra_hash) - records noted.hydra_config_hash on the active run.

After the prelude runs, the kernel has:

- mlflow.log_metric replaced with the live-streaming wrapper.
- mlflow.start_run replaced with the wrapper that emits the run-start hook.

- A running MLflow run with experiment, name, data hash tag, config hash tag set.
- run as a module-level variable holding the run handle.

Step 5 - Hydra injection (same as cell path)

The Hydra injection runs after the prelude so cfg is available before cell 11 (the seed cell, Chapter 2.2) runs.

Step 6 - Cell-by-cell execution

For each cell in the list, execute_run() executes the cell code via the same kc.execute(code, ...) mechanism used by the cell path. Each cell produces its own cell:output events and its own cell:execute_complete. The user sees them arrive in sequence in the Run Manager's execution log.

If a cell raises, the loop is *not* aborted by default - the next cell runs too. This matches Jupyter's semantics and lets the user see all output even if an early cell had a warning. The Run Manager UI surfaces each cell's status (idle/running/success/error) so execution progress is clear even across failures.

Step 7 - MLflow run-start event handler

One of the display_data messages emitted early in the run is application/x-noted-run-start (Chapter 3.3.3). The IOPub dispatcher picks this up, extracts the run_id, and fires _log_hydra_bundle_for_run(run_id) in a background thread. That thread:

1. Re-composes the Hydra config from the current hydra_selections.
2. Calls HydraManager.assemble_bundle_from_source(...) to build the hydra/ artifact tree.
3. Writes to a tempdir, uploads via client.log_artifacts(run_id, tmpdir, "hydra").
4. Tags the run with noted.hydra_config_hash, noted.project_id, noted.git_commit, mlflow.source.git.branch.

The upload is fire-and-forget - failures are logged but do not affect the running execution.

Step 8 - Metrics streaming

As the user's code calls mlflow.log_metric(...) (usually from a Keras on_epoch_end callback, Chapter 3.3.2), each call emits application/x-noted-metric display_data. The IOPub dispatcher intercepts these, suppresses them from cell output, and emits metrics:update socket events. The frontend's live metrics chart updates in real time.

Step 9 - Run completion

After the last cell, execute_run() injects RUN_END_CODE which calls mlflow.end_run(). The MLflow run is now closed - its metrics, params, and tags are frozen. The bridge emits run:complete with the run_id. The frontend's Run Manager shows the final state and links to the run in the MLflow view.

9.4 Execution contention: the collaborative editing case

noted supports multiple clients connected to the same notebook via socket.io rooms. Execution is kernel-scoped, not client-scoped - all clients in the room share one kernel. This means:

- A cell execution triggered by client A is visible to client B in real time. The cell:execute_start and cell:output events broadcast to the whole room.
- Concurrent edits to the same cell are prevented by per-cell locks (cell:lock / cell:unlock, main.py:745). A client acquires a lock before editing, releases it on blur. Attempted edits on a locked cell are rejected.
- Concurrent execution requests to the same cell are serialized by the kernel - the second execute_request queues until the first finishes. Visually, the second client sees their cell go from idle to queued to running.

9.5 Interruption and cancellation

A user can interrupt a running cell via the stop button on the cell toolbar or the kernel:interrupt menu action. This sends a SIGINT to the kernel process via jupyter_client's km.interrupt_kernel(). Python's normal signal-handling raises KeyboardInterrupt in the running cell code.

Important caveats:

- Interrupting a C-extension call (numpy operations, TensorFlow training) does not always work immediately because the C code may not check for Python signals. Interrupt is advisory; kernel restart is the hammer.
- Interrupt during _execute_silent prelude code is not supported - the prelude is expected to be short and side-effect-free enough that users never need to interrupt it.
- For long-running trainings, pressing interrupt usually takes effect at the next epoch boundary (when Keras's callbacks have a chance to check for signals). Patience; then restart if needed.

9.6 Error paths and the NO_KERNEL story

Several failure modes are handled explicitly:

- **No kernel yet.** If the user tries to execute before starting a kernel, the backend emits error: NO_KERNEL and the frontend shows a notification with a "Start Kernel" button.
- **Kernel crashed.** If the kernel process dies during execution, the IOPub channel closes and _iopub_loop catches the exception. Pending handlers are notified with an error. The session state flips to dead; the frontend shows the kernel-dead banner with a restart button.

- **Socket disconnected mid-execution.** The kernel continues running. The handler is still in _pending. When the client reconnects and rejoins the room, a state-refresh query can fetch the in-progress cell's current output. (Not fully implemented at v0.1 - reconnect after a long execute may lose the streaming log, though the final cell:output event is still persisted to the notebook file on save.)
- **Hydra compose failure.** If _build_hydra_injection fails (e.g. malformed YAML in the config tree), the injection is skipped, a warning is logged, and the cell runs without cfg - the user sees a NameError on the first cfg.X.Y access. The trade-off between hard-failing the cell vs. running it is biased toward letting the user see the error in their own code rather than a backend abstraction.

9.7 The Run All path: where it differs

"Run All" is a third path, technically. It is a frontend-only loop that issues cell:execute events one at a time for every code cell. The backend sees a series of single-cell executions.

Critical difference from Run execution: **Run All does not install the MLflow prelude**. Consequences:

- mlflow.log_metric calls in the user's code go through the unpatched function. They are logged to MLflow if a run is active, but no metrics:update socket events fire - the live metrics panel stays empty.
- There is no active run when cell 11 (seeding) runs. Cell 116 opens a new run inside its with mlflow.start_run(): block; this run has no noted.* tags and no hydra/ bundle archived against it because the run-start hook was never installed.

This is the known foot-gun documented as "Run All vs Run Manager" in Chapter 6.5. The pragmatic guidance is "use Run Manager for anything you want tracked"; the engineering fix is to have cell:execute check whether the next cell is likely to start a run, and pre-install the metrics patch.

9.8 Debug execution

Ctrl+Shift+Enter on a cell triggers a debug execution instead. The flow:

1. cell:execute is emitted with debug: true.
2. ExecutionBridge.execute_cell() checks the flag and calls _inject_debug_bootstrap() which ensures debugpy is listening (via KernelManagerService.init_debugpy()).
3. The cell code is wrapped in a pre-bounce debugpy.wait_for_client() if no DAP client is attached yet.
4. The frontend's DAP client connects through noted's DAP proxy (backend/app/routers/dap.py) to debugpy.

5. Once attached, breakpoints set in the cell are honored; the user can step via F10/F11, continue via F5, stop via Shift+F5.

Debug is per-cell; exiting a debug session does not leave the kernel in debug mode. The next non-debug execution runs normally.

9.9 Discussion-ready talking points

Q: Why are cell and run execution two paths instead of one? A: Because the prelude machinery (MLflow start_run, metrics patch, dataset hash logging, bundle archival) is expensive to install and pollutes the kernel's namespace. Single-cell debugging or exploration does not need any of that. The split lets users do quick iteration with minimal overhead and opt into the full tracking surface only when they are running a real experiment. The cost is that Run All produces "tracked" output that is less rich than Run Manager output - which is the foot-gun Chapter 6.5 documents.

Q: Why is the prelude executed silently instead of appearing as a cell? A: Because the notebook is meant to be the user's unit of authorship. A visible prelude cell - even one that says "# noted will replace this with your config" - is a surface the user would be tempted to edit, and breaking it would break the run. Silent injection keeps the notebook identical across users, across versions, and across hosts. The cost (documented as "tech-debt invisible preludes" in memory) is that the notebook is not portable outside noted without manual compose code.

Q: Why does _iopub_loop run in a thread instead of using asyncio natively? A: Because jupyter_client.KernelClient.get_iopub_msg() is a blocking call with no async variant. Offloading it to loop.run_in_executor(...) keeps the event loop free to handle other socket events, router requests, and heartbeats. An async-native jupyter_client would be nice; none exists, and writing one is a larger yak-shave than it is worth.

Q: What prevents two cells from interleaving their output if executed rapidly? A: The kernel's shell channel processes one execute_request at a time - the second queues. IOPub messages carry the parent_header.msg_id of the request that caused them, so the bridge's dispatcher routes each message to its correct cell handler regardless of interleaving order at the ZMQ layer. The frontend sees outputs in cell-specific streams.

Q: How does the backend know when a run is "done" vs "still writing the last metric"? A: The shell channel's execute_reply for the final cell in the run is the marker. Once that fires for the injected RUN_END_CODE, execute_run() considers the run complete and emits run:complete. The MLflow server may still be flushing artifacts to disk at that point, but the run record itself is closed.

Q: Can the backend replay a past execution? A: No. Replay is a Hydra-mediated workflow: load the archived bundle into the Composer (Section 6.1), click Run, observe the new run. There is no "re-run cell N with these exact inputs" primitive because cell N's inputs depend on its preceding cells' side effects, which are not captured in noted's state model. If true replay is ever needed, a full-notebook determinism harness would be required.

Q: What is the notebook:save contract? A: On save, the current notebook state - cells, cell outputs, cell metadata, notebook-level metadata - is serialized to the .ipynb file on disk. Metadata includes hydra_selections, hydra_baseline_source, and notebook_uid (Chapter 2.3.3). Cell outputs are saved, so reopening a notebook shows the last execution's output without re-running. This is standard Jupyter behavior; noted adds no twist.

10. Configuration Composer + Time Machine

10.1 Concept primer

The Configuration Composer is noted's UI-over-Hydra. It lets the user view, modify, and apply a composed configuration without writing YAML, without opening a shell, and without restarting the kernel. Time Machine is the same UI surface with the baseline source flipped from the project's working tree to a past MLflow run's archived bundle - turning the Composer into a config-reproducer for past experiments.

The two features share the same codepath. The only difference between "edit the current config" and "replay a past run" is which HydraSource implementation is servicing the composition (Chapter 2.3.1). This is the architectural payoff of the source abstraction - a single composition engine, two user-visible modes, zero duplicated logic.

Four design properties are worth naming:

1. **The Composer is read-write against *selections*, not templates.** Clicking Apply persists group selections and overrides to the notebook's metadata. It never edits a YAML file on disk. Template edits are still made the usual way (open config.yaml in the editor, save). This keeps the "templates vs config" distinction (Chapter 2.1) visible in the UI.
2. **The baseline badge is the integrity assertion.** After every Apply, the badge recomputes its state (green/orange/red) by comparing current selections against the active source. The user cannot be wrong about whether their config matches a baseline - the badge always tells the truth.
3. **Time Machine is composition, not checkout.** Loading a past run into the Composer does not modify any file in the working tree. No git checkout happens. The past run's bundle is composed against its archived templates, the result is shown in the Composer, and the user decides whether to Apply it (making it the new baseline for the notebook) or dismiss it.
4. **The HydraCache is the Composer's perf budget.** Each (notebook_uid, run_id) pair is one cache entry, backed by MLflow's artifact download on cache miss. Populated entries make Composer interactions instantaneous; cold cache entries take a few seconds for the initial fetch.

10.2 Where Time Machine lives

The Composer UI is one file: frontend/js/panels/explorer/ExplorerHydraViews.js. The Time Machine is not a separate component - it is the Experiment Run mode of the same Composer panel. A mode toggle (Local vs Experiment Run, rendered as two buttons at the top of the panel) swaps which HydraSource the backend uses for subsequent compositions.

Backend surface lives at backend/app/routers/hydra.py (the four endpoints in Section 2.3.5) plus three manager classes:

- HydraManager (backend/app/managers/hydra_manager.py) - composition, schema discovery, bundle assembly.
- HydraSource + LocalSource + MlflowSource (backend/app/managers/hydra_source.py) - source abstraction.
- HydraCache (backend/app/managers/hydra_cache.py) - in-memory LRU.

frontend/js/NotebookEditor.js owns the badge state and the notebook metadata contract.

10.3 The HydraSource abstraction

The filesystem-shaped contract (line 27 of hydra_source.py):

```
class HydraSource:
    def exists(self, path: str) -> bool: ...
    def read_text(self, path: str) -> str: ...
    def walk(self) -> Iterator[tuple[str, list[str], list[str]]]: ...
```

Three methods: “does this file exist”, “read this file as text”, “enumerate the tree”. That is the complete contract. HydraManager’s composition code uses only these three methods; any backing store that can satisfy them is a valid source.

10.3.1 LocalSource

LocalSource (line 56) reads from the project’s on-disk config/ directory. It is a thin wrapper over pathlib.Path - exists() does path.exists(), read_text() does path.read_text(), walk() does os.walk.

Used when baseline_source = "project://config/" (the default for any notebook that opted into Hydra).

10.3.2 MlflowSource

MlflowSource (line 113) reads from an MLflow run’s hydra/ artifact tree. It does not hold the tree directly - it delegates to HydraCache.fetch_from_mlflow(notebook_uid, run_id) which downloads the tree on first access and flattens it into dict[str, bytes].

_load_bundle() (line 129) is the lazy loader. walk() (line 175) reconstructs the directory tree structure from the flat bundle keys by splitting on / and grouping prefixes.

Used when baseline_source = "mlflow://<run_id>".

10.3.3 The walk() recursion bug

A load-bearing historical detail: the original MlflowSource.walk() only yielded the root directory. Subdirectories were never added to all_dirs, so assemble_bundle_from_source() copied only top-level files when the source was MlflowSource. This caused runs made against a pinned MLflow baseline to archive *incomplete* config/ trees (only config/config.yaml, no group files), which broke downstream composition.

The fix walks each nested dir in the first pass, ensuring subdirectories are enumerated. Bundles produced before the fix remain permanently incomplete - they cannot be replayed. New Run Manager runs produce correct bundles.

10.4 The HydraCache

HydraCache (backend/app/managers/hydra_cache.py:27) is an OrderedDict keyed by (notebook_uid, run_id) tuples, mapping to bundle dicts. MAX_ENTRIES=500 (line 24); overflow evicts the oldest entry FIFO.

Methods:

- get(key) (line 37) - fast-path read.
- put(key, bundle) (line 42) - store; evict oldest if full.
- fetch_from_mlflow(notebook_uid, run_id) (line 64) - download the hydra/ artifact tree from the specified run, flatten into a bundle dict, store and return.

The cache is ephemeral (no disk backing, lost on backend restart). This is intentional - MLflow is the ground truth; the cache exists only to avoid downloading the same bundle on every Composer interaction within a session.

Cache key design. Including notebook_uid as part of the key is deliberate. The same run can be loaded as a baseline from multiple notebooks, and each notebook may make different override changes on top. Keying by run_id alone would conflate these sessions; keying by (notebook_uid, run_id) keeps them distinct.

10.5 The four Composer endpoints

backend/app/routers/hydra.py exposes the Composer's backend surface.

10.5.1 GET /api/hydra/experiments/{project_id} (line 182)

Returns the list of MLflow experiments that have at least one run tagged with noted.project_id == project_id. Used by the Experiment dropdown in Time Machine mode. The filter by project tag is what keeps the dropdown from showing unrelated experiments from other projects.

10.5.2 GET /api/hydra/runs/{project_id}/{experiment_id} (line 233)

Returns runs within the experiment that have a hydra/ artifact bundle. Each entry includes run_id, run_name, start_time, status, and the noted.hydra_config_hash tag. The run dropdown shows these; sort order is newest-first.

Runs without a hydra/ bundle are filtered out - they cannot serve as a Time Machine baseline, so surfacing them would be misleading.

10.5.3 POST /api/hydra/load-bundle (line 307)

The critical endpoint. Request body: {notebook_uid, run_id}. Response body: the archived selections, the archived overrides, the resolved yaml, and a validation flag.

Server logic:

1. Call HydraCache.fetch_from_mlflow(notebook_uid, run_id) to get the bundle.
2. Parse selections.json to get archived group selections and overrides.
3. Re-compose from MlflowSource(notebook_uid, run_id) using those selections.
4. Compare the new sha256(resolved_yaml) with the run's stored noted.hydra_config_hash tag.
5. Return {group_selections, overrides, resolved_yaml, experiment_id, hash_matches: bool}.

If hash_matches: false, the Composer UI surfaces a red X on the badge and refuses to apply. This is the guardrail for replay: if recomposition does not reproduce the archived hash, the bundle is corrupt or the composer code is buggy, and either way applying it would be lying about the run's identity.

10.5.4 POST /api/hydra/compose-mlflow (line 277)

For live composition while the user is tweaking overrides on top of an MLflow baseline. Request body: {notebook_uid, run_id, group_selections, overrides}. Server composes against MlflowSource with the user's modifications applied and returns the resulting resolved_yaml and hash. Used by the Composer to render a preview without calling Apply.

10.6 Composer UI state machine

ExplorerHydraViews.js manages the Composer's state. The key state fields (stored on panel.content):

- mode - local or mlflow.
- experimentId, runId - valid only when mode is mlflow.
- groupSelections - {data: "jena_full_dataset", model: "gru_baseline", scaler: "standard"}.
- overrides - {seed: 42, "training.epochs": 10, ...}.
- schema - the schema object returned by get-schema-from-source (group options + override fields).
- resolved - the last-known resolved config (for preview display).

User actions and state transitions:

- **Click Local button** (_switchToLocal) - set mode to local. Preserve experimentId and runId for later (preview-only). Reload schema against project://config/. Repopulate

group dropdowns with schema defaults, preserve user's previous group_selections if they are still valid against the schema (D13 contract: switching modes does not wipe selections).

- **Click Experiment Run button** (_switchToMlflow) - set mode to mlflow. Load experiments list. Do not Apply yet. Update badge state immediately via _updateApplyButtonEnabled.
- **Pick experiment** - load the run list for that experiment.
- **Pick run** - enable Apply button.
- **Click Apply** - call load-bundle, then NotebookEditor.setHydraSelections(...), then _refreshActiveSchema(...), then update the badge.
- **Edit any override field** - trigger a debounced compose-mlflow (or compose-local) call to update the preview pane. Does not modify notebook metadata until Apply.

The Apply button is disabled by default (rm-btn:disabled CSS applied) until the state satisfies: mode + experiment + run are all selected (for mlflow) or at least one selection differs from defaults (for local).

10.7 Baseline badge state machine

frontend/js/NotebookEditor.js:2106 is _updateBaselineBadge(). Three labels, three dot states.

Labels:

- BASELINE (neutral gray) - when hydra_baseline_source === "project://config/".
- RUN xxxxxx (purple, short hash of the run_id) - when hydra_baseline_source is mlflow://....

Dots:

- Green check - no drift. Current selections match the baseline: for Local mode, match schema defaults; for MLflow mode, match archived selections.
- Orange exclamation - drift. Current selections differ from baseline. Tooltip includes a Drift: section listing the specific keys that differ.
- Red X - unreachable. The baseline source could not be loaded. For MLflow, means the run was deleted or the bundle was corrupt. For Local, means the config tree is missing or malformed (rare).

_computeBaselineBadgeState() (line 2166) is the state computer. It walks the current selections against the schema (which was refreshed against the current baseline) and per-key compares to the default. Mismatches accumulate into the drift list.

_selectionsEqual() (line 2250) treats undefined / null / empty-string as "not selected" - this is what prevents stale empty metadata from triggering phantom drift.

_refreshActiveSchema() (line 2399) is called after every Apply to re-fetch the schema against the (possibly changed) baseline source. Without this call, the badge would compare against a stale schema and could falsely report drift.

10.8 Operations

Add a new Composer override input

For an override to surface in the Composer UI, the corresponding leaf must exist in config.yaml (not in a group file - see Chapter 2.4 / the known gap). Once added to config.yaml, reload the Composer panel. _extract_schema will discover the new leaf and render it as an input.

Fix a stale-metadata notebook

If the badge is stuck orange and the tooltip lists keys that are not visibly wrong, the notebook likely has legacy flat-format hydra_selections. Open the Composer in Local mode, pick defaults for all groups, clear overrides, click Apply. This rewrites the metadata to the current nested format.

Clear the HydraCache

No UI for this; the cache is process-local. Restarting the backend clears it. On the next Composer interaction, the cache will refill from MLflow on first access.

Debug a load-bundle hash mismatch

1. Check the run's hydra/ artifact tree in MLflow UI. Verify it contains config.yaml, selections.json, resolved.yaml, plus the group files.
2. If files are missing, the run was produced before the walk() fix. The bundle is permanently incomplete; the run cannot be replayed.
3. If files are present but the hash still mismatches, diff the local composition's resolved.yaml against the archived one. The first divergence is the bug.

10.9 Discussion-ready talking points

Q: Why is Time Machine built on the same UI as the Composer rather than a separate panel? A: Because the action of loading a past run as a baseline *is* composition - just from a different source. Spinning up a second panel would duplicate all the dropdowns, override inputs, badge logic, and compose endpoint wiring. The source abstraction (LocalSource vs MlflowSource) is the right axis to split on; swapping the source behind a single UI is cheaper and keeps the user's mental model clean.

Q: What does the Apply button actually do? A: Three things in sequence. (1) Write hydra_selections and hydra_baseline_source to the notebook's metadata via PATCH /api/notebooks/.... (2) Fetch a fresh schema against the new baseline source to seed the drift comparison. (3) Recompute the badge. Nothing about the user's code is modified; no

Python is executed; no kernel state changes. Apply is pure metadata mutation plus UI refresh.

Q: Why validate the hash on load-bundle instead of trusting the archived bundle? A: Because the archive is only useful if composition against it reproduces the identity the run recorded. A bundle that composes to a different hash means either the bundle is corrupt, the composer code has regressed, or the Hydra library behavior has changed. Any of these is a reason to refuse the load rather than silently serve a subtly-different config. The red X on the badge is the user-visible evidence that the system caught a regression.

Q: What happens to notebooks that have no notebook_uid? A: They are treated as Hydra-unaware. The Composer panel still renders, but Apply generates a UUID on the fly and writes it into metadata before proceeding. The first Apply is therefore the moment a notebook commits to being a Hydra-using notebook. Until then, cfg is not injected, and the notebook behaves like a regular Jupyter notebook.

Q: Why does switching Local vs Experiment Run mode not immediately wipe the run selection? A: Because the user may flip modes to preview the alternative without committing. The D13 design contract is “switching modes is preview-only; Apply is the commit point”. This lets the user click around freely in the Composer without fear of losing state. State is only persisted when Apply is explicitly clicked.

Q: Why is the badge part of NotebookEditor rather than a separate module? A: Because the badge has to know about three pieces of notebook state simultaneously: metadata (baseline source, selections), live Composer state (is the user currently editing?), and the schema (what are the defaults?). All three are owned by NotebookEditor, so colocating the badge logic avoids cross-module coupling. The alternative (a separate Badge module subscribing to notebook events) would be more modular but would require propagating schema changes across a module boundary for no obvious win.

Q: Does Time Machine work across projects? A: Implicitly yes, but the UI does not encourage it. The experiments/{project_id} endpoint filters by project tag, so the dropdown only shows experiments tagged for the current project. If an identical notebook existed in two projects, their Time Machine lists would be disjoint by design. Cross-project replay would require opening the bundle directly via a run_id the user enters manually - no UI for that at v0.1.

Q: What is the relationship between the Composer and the Knowledge Graph? A: The Knowledge Graph (Module 13) reads the same MLflow tags and Hydra bundles the Composer does, but it visualizes the graph of runs / datasets / configs instead of showing one run at a time. A future feature would let the user click on a node in the graph to load its config into the Composer - the data is already in place; only the UI wiring is missing.

11. Model Serving

11.1 Concept primer

noted-serving is the inference container that turns a registered MLflow model into an HTTP endpoint. Its responsibilities are narrow: load the artifact, validate the request, run the prediction, return the response. The surrounding complexity - tracking which version to load, caching artifacts, managing VRAM, streaming deploy progress to the UI - is what the module is actually about.

Three ideas explain most of the design:

1. **Loader + FastAPI in one process.** The container is a single uvicorn process hosting a FastAPI app plus a ModelLoader singleton. All state is in-process memory. This is the Phase 0a design; it has a specific failure mode (stale C-extension imports after reloading) documented in Section 11.8 and a queued Phase 0b refactor that moves loading to a worker subprocess.
2. **NDJSON streaming for observability.** A model load can take 10-60 seconds depending on artifact size. Streaming per-phase progress (resolving, downloading, loading_model, ready) lets the frontend show what the backend is doing instead of a spinner that outlives the user's patience. This is the DeployEventStream pattern.
3. **Alias-driven deployment.** The serving container resolves @champion on every load request by querying MLflow. Clients do not see version numbers unless they specifically ask for them. Rolling back is an alias hop (Chapter 3.5), and the serving layer picks it up on the next deploy.

The external jena_client demo app is a separate project outside noted - a FastAPI + socket.io demo UI that proxies to noted-serving. It exists to prove the serving contract is usable from a standalone application, not just from noted's own Try It panel.

11.2 The serving container

client/Dockerfile (18 lines):

```
FROM python:3.12-slim
WORKDIR /app
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
COPY requirements.txt .
RUN uv pip install --system -r requirements.txt
COPY app/ app/
EXPOSE 5522
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5522"]
```

The image installs uv (a fast pip replacement from Astral) and uses it to install the requirements. Removing the --no-cache flag was the fix that restored proper Docker layer

caching - previously every rebuild re-downloaded 2 GB of CUDA wheels because uv was downloading fresh on every build.

client/requirements.txt lists the baseline dependencies:

- fastapi, uvicorn, httpx - the HTTP layer.
- mlflow - artifact resolution and pyfunc loading.
- numpy, pandas, scikit-learn - common data handling.
- tensorflow[and-cuda], torch, pytorch-lightning - major ML frameworks.
- xgboost, lightgbm - tree ensemble frameworks.
- boto3 - S3/MinIO access for artifact download.

The image is deliberately a *superset*. Phase 0a's working-by-baseline approach assumes that any model the user promotes will be loadable without additional installs - the superset covers TF, Torch, sklearn, XGBoost, LightGBM. For a model with exotic dependencies, Phase 0b (Section 11.8) is the correct escape hatch.

client/app/main.py is the FastAPI app (136 lines). Endpoints:

- GET /health - return the loader's status dict.
- POST /load - stream NDJSON progress, end with ready or error.
- POST /unload - free the loaded model.
- GET /schema - return the cached input/output schema.
- POST /predict - run inference on the request payload.

CORS is permissive (*) because the container runs inside the compose network and is reached only through the backend's proxy (backend/app/routers/serving.py). Exposing it directly would require tightening CORS.

11.3 ModelLoader and the RLock

client/app/model_loader.py is the state owner. The class is ~430 lines and holds:

- _lock - an RLock serializing load/unload operations.
- _status - one of idle, loading, ready, error.
- _model, _model_info - the loaded pyfunc model and metadata.
- _phase, _phase_detail, _phase_callback - for streaming deploy events.

Key methods:

load(model_name, version=None, alias=None) (line 67)

1. Acquire the RLock.

2. If the same model+version is already loaded, return immediately (idempotent).
3. Set status to loading, clear previous model.
4. Delegate to _load_inner(...).

_load_inner(...) (line 104)

1. Resolve version from alias if provided (query MLflow registered-models/{name}/alias/{alias} API).
2. Emit resolving phase with the resolved version string.
3. Download the model artifact. Three resolution strategies are tried in order:
   - Read model_uri tag from the registered model version.
   - Scan the experiment's models/ directory for a matching MLmodel (MLflow 3.x Logged Models).
   - Fall back to the legacy runs:/<run_id>/model URI.
4. Emit downloading phase with byte count updates.
5. Call mlflow.pyfunc.load_model(local_path).
6. Emit loading_model phase with framework detection progress.
7. Extract signature, flavors, framework name, parameter count, artifact size.
8. Cache the schema (via schema_builder.build_schema).
9. Emit ready phase with the full health payload.

The neutered _install_model_deps() (line 194-201)

An earlier version of the loader had a _install_model_deps() method that read the model's requirements.txt from the artifact and called uv pip install at runtime to pin the exact versions the model was trained against. This caused a subtle failure mode: installing numpy 2.x over an already-loaded numpy 1.x left TensorFlow holding stale C-extension pointers. Subsequent predictions would segfault or silently return garbage.

The fix: neuter _install_model_deps() to a no-op. Rely on the baseline image's superset and on MLflow's warning-mode loading (which tolerates version mismatches with a log message rather than refusing to load). The trade-off: for models with pins outside the baseline, loading will fail at import time rather than install-and-corrupt. Phase 0b solves this structurally by running the load in a fresh process each time.

unload() (line 328)

1. Non-blocking acquire of RLock (refuse if a load is in progress).
2. Null the model and model_info; set status to idle.
3. gc.collect() to force Python GC.
4. Framework-specific cleanup:

- tf.keras.backend.clear_session() for TensorFlow.
- torch.cuda.empty_cache() for PyTorch.
- jax.clear_caches() for JAX.

5. Release lock.

This is best-effort. In-process VRAM cleanup is never fully reliable - CUDA keeps a driver-level context per process, and frameworks maintain internal pools. Phase 0b's process-exit approach is the only clean guarantee; Phase 0a accepts the imperfection as a tradeoff at this scale.

get_health() (line 403)

Returns the current status dict *without acquiring the lock*. Must stay responsive while a load is in progress.

11.4 DeployEventStream and the NDJSON contract

client/app/deploy_stream.py (130 lines) bridges the synchronous ModelLoader.load() call to a FastAPI StreamingResponse.

The pattern:

1. Register a phase callback on the loader (loader.set_phase_callback(my_callback)).
2. Run the load in an executor thread via loop.run_in_executor(None, loader.load, ...).
3. The loader's internal _set_phase(phase, detail) calls fire the callback on each state transition.
4. The callback puts an event onto an asyncio queue.
5. The StreamingResponse yields each queued event as a JSON line followed by \n.

Event shape:

```
{"phase": "resolving", "detail": "version 7 (alias=champion)"}
{"phase": "downloading", "detail": "45 MB / 120 MB"}
{"phase": "loading_model", "detail": "loading tensorflow flavor"}
{"phase": "ready", "result": { ...health payload... }}
```

On failure:

```
{"phase": "error", "error": "Could not load model: ..."}
```

The frontend's ModelDeployer reads these events with a native ReadableStream + TextDecoder (no polling). The exact bytes are the contract.

11.5 Backend proxy and frontend Deploy

backend/app/routers/serving.py (138 lines) proxies every serving endpoint through noted's backend:

- GET /api/serving/health -> forwards to client /health.
- POST /api/serving/load -> forwards NDJSON stream. Uses httpx.AsyncClient with read=600s timeout and streams line-by-line.
- POST /api/serving/unload -> forwards POST.
- GET /api/serving/schema -> forwards.
- POST /api/serving/predict -> forwards.

The proxy exists so the frontend never has to know the serving container's address. CORS is handled once at the noted backend; secrets (if ever added) are enforced once. The trade-off is one extra hop per request; for model inference that is in the seconds range, the milliseconds of proxy overhead are invisible.

frontend/js/ModelDeployer.js (~158 lines) is the client-side streaming consumer. Key method _readStream() (line 112) uses response.body.getReader() + TextDecoder to buffer partial lines, parse JSON per newline, and dispatch onPhase(phase, detail) or the terminal onReady(result) / onError(msg) callback.

frontend/js/panels/explorer/ExplorerServingViews.js is the UI integration. showTryItPanel(modelName, version) (line 27) opens a jsPanel, polls /health to confirm the right model is loaded, and calls _buildInputForm() (line 117) to render a form from the schema. Each field is typed from the signature (float / int / text) or falls back to a JSON textarea for complex shapes. A "Sample" button auto-populates values from schema.example_input or randomly-generated values per-type.

11.6 Logged Model artifact proxy

Chapter 3.3.7 described the Logged Models view; the serving path uses the same backend endpoints to discover and download artifacts. For completeness:

- GET /api/mlflow/runs/{run_id}/logged_models (mlflow.py:59) lists Logged Model entities linked to the run.
- GET /api/mlflow/logged_models/{experiment_id}/{model_id}/download?path=X (mlflow.py:76) streams a single file via MLflow's artifact proxy with directory-traversal validation.

The ModelLoader's three-strategy artifact resolution uses these endpoints indirectly - mlflow.pyfunc.load_model("runs:/<run_id>/model") goes through MLflow's Python SDK, which resolves to the same artifact paths the proxy exposes.

11.7 jena_client external demo

/home/logus/env/iscte/jena_client/ is a separate project outside noted. Its purpose is to demonstrate that a standalone application can consume noted-serving's predictions through stable HTTP contracts.

Structure (web/backend/server.py, 150 lines):

- FastAPI + socket.io, serves a static frontend from web/frontend/.
- Proxies /api/health, /api/schema, /api/predict to http://noted-serving:5522 via httpx.
- Queries http://mlflow:5000 for model lists, version metadata, and **crucially** run parameters including target_mean and target_std (the scaler stats logged at notebook cell 116, Chapter 3.3.5).

The frontend is three dropdowns (project, model name, version with @champion default), a form for input features, and a result display. After receiving a scaled prediction from the serving endpoint, jena_client applies the inverse transform y_real = y_scaled * target_std + target_mean and displays the result in real units (degrees Celsius).

This is the proof that noted's training lineage survives the serving boundary. A future client in a different language (Go, TypeScript, Rust) would need to replicate the same three HTTP calls and the same inverse-transform math - nothing more.

11.8 Phase 0a vs Phase 0b

Phase 0a (current). Single-process serving. Baseline image has a superset of frameworks. _install_model_deps is neutered. Models with pins outside the baseline will fail at load time with a clear error. VRAM cleanup is best-effort. Fast enough for the current scope.

Phase 0b (deferred, designed but unshipped). Worker subprocess architecture. Plan in documents/serving_worker/serving_worker_plan.md (467 lines).

Key properties of Phase 0b:

- Each Deploy spawns a fresh Python interpreter via asyncio.create_subprocess_exec().
- Worker does uv pip install from the model's requirements.txt against a clean import state.
- Worker loads the model, exposes a mini-FastAPI on a localhost port, streams NDJSON to its stdout.
- Control plane (the original uvicorn process) proxies to the worker. Control plane never imports ML libraries, so it stays responsive.
- VRAM release guaranteed via process exit.
- Stale-import bug impossible because each Deploy is a fresh process.

Three optional layers on top:

- Layer 1 - uv cache volume shared across workers (faster installs).
- Layer 2 - per-model venvs with hash-based lookup (skip install if a matching venv exists).
- Layer 3 - worker pool with same-hash in-place model switch (swap models within a venv).

Estimated effort: 9-15 hours. Deferred because Phase 0a meets the current scope.

11.9 Operations

Deploy a model

1. Open the Registry view.
2. Select the model, then a version.
3. Click Deploy. The button streams phases in place: resolving -> downloading -> loading_model -> ready.
4. On ready, the button flips to Unload. The Try It button becomes enabled.

Unload a model

1. Click Unload on the deployed version's card. The loader releases the model and frees (best-effort) VRAM.
2. The Deploy button on that version becomes available again.
3. Refusal (e.g. a concurrent load in progress) shows an inline error; the button re-enables when safe.

Try a model

1. With a model deployed, click Try It.
2. A jsPanel opens with a form built from the model's signature. Each field is typed.
3. Click Sample to auto-populate, or enter values manually.
4. Click Predict. The request goes to /api/serving/predict, the response renders inline as a table, line chart, or scalar depending on output shape.

Debug a failing load

1. Check /api/serving/health - the error field has the exception message.
2. Check the noted-serving container logs: docker logs noted-serving --tail 200.
3. Common causes: artifact download fails (check MinIO), model signature cannot be parsed (check MLmodel file), framework version mismatch (a runtime-install path existed to fix this but was removed; Phase 0b is the proper fix).

Inspect the Logged Model artifacts

1. Open the run in the MLflow view or via the Registry.
2. Navigate to the Logged Models subtree.
3. Open MLmodel, conda.yaml, python_env.yaml, requirements.txt - all render with hljs syntax highlighting.

11.10 Discussion-ready talking points

Q: Why does the serving container use MLflow's pyfunc instead of a framework-specific load? A: Because pyfunc is the common interface that every flavor (tensorflow, pytorch, sklearn) implements. Loading via pyfunc lets the container be framework-agnostic at the load call site. Framework-specific operations (cleanup, parameter counting) branch on the detected flavor after the fact. The alternative - a big switch at load time - would duplicate logic per framework and make adding a new flavor a serving-side change.

Q: Why NDJSON streaming instead of server-sent events or WebSockets? A: Because NDJSON over a plain HTTP POST response is the simplest contract that gives streaming progress. SSE would work but requires a specific content type and a different client API. WebSockets would require a separate connection setup and ping/pong management. The Deploy stream is a one-shot linear sequence - it is born, streams, dies. HTTP+NDJSON matches that shape perfectly.

Q: Why is get_health() lock-free? A: Because health queries must respond during a load. If get_health acquired the same lock as load, a 45-second load would make the frontend's health polling hang, which would make the UI appear frozen. The lock-free read is safe because the fields it reads are atomic - Python's dict assignment is protected by the GIL for individual keys.

Q: What prevents two Deploys from racing? A: The RLock in ModelLoader.load(). The second Deploy waits for the first to finish or fail. The frontend's Deploy button is disabled during the stream, so the user cannot fire two from the same tab; concurrent Deploys from different tabs would see the second one queue.

Q: Why is the baseline image a superset instead of a minimal image? A: Because the cost of a missing package at Deploy time is user-visible (failed load with an import error), and the cost of an extra 2 GB in the image is only disk space. For local-scale deployment the trade favors breadth. For production at scale, Phase 0b's per-model venvs is the right inversion.

Q: How is the @champion alias resolved? A: The loader queries MLflow's registered-models/{name}/alias/{alias} endpoint, which returns the current version number the alias points at. Version number is then used for the artifact download. If the alias is reassigned between load and predict, subsequent predicts continue to hit the loaded version - there is no live-rebind. Rebinding requires Unload + Deploy; a future feature could add hot-swap.

Q: What stops the serving container from being called directly from outside noted? A: Nothing - port 5522 is exposed on the host by docker-compose.yml. The intent is the noted backend's proxy. Direct access works for debugging but bypasses whatever auth the backend might add in the future. Production should either remove the port binding or front the container with an auth proxy.

Q: Why is jena_client outside noted instead of integrated? A: Because it is proof-of-concept for the serving contract from an external client's perspective. If it were part of noted, it would be tempting to cheat - to call internal APIs, to reuse backend code, to share state. Keeping it in a separate repo forces the contract to be honest: three HTTP endpoints (/health, /schema, /predict) plus an MLflow API call to fetch scaler stats. That is the complete surface area a third-party integration needs.

12. AI Assistant + MCP

12.1 Concept primer

noted's AI assistant is a chat-style interface wired to two backends (local Gemma 4 via an OpenAI-compatible agent server, and Claude Sonnet/Opus/Haiku 4.x via the Anthropic API) with shared context plumbing, a tool-calling loop, a skills registry, and a write-confirmation pattern. The assistant is not a general-purpose chatbot - it is a developer copilot anchored to the state of the user's notebook, files, MLflow runs, and Hydra configuration.

Five design choices deserve surfacing:

1. **Both backends, always.** The same conversation, skills, tools, and context system work against local Gemma or Claude. No "Claude-only" paths. This is a load-bearing rule (memory documents it explicitly) because it keeps the assistant usable offline and keeps the cost curve bounded.
2. **Dynamic context injection.** The backend does not blindly send the whole notebook or every project file. A context router classifies the user's question against domain keywords (mlflow, airflow, hydra, files, ...) and injects only the relevant context blocks and skills. For Claude this saves ~2000 tokens per turn.
3. **Skills as inline micro-docs.** Each skill is a small markdown file with YAML frontmatter. The skill registry loads them at startup, and the context router auto-injects the ones whose triggers match the current user session. The LLM sees skills as part of its system prompt, not as retrieved documents - they are *always* considered when they match.
4. **Write confirmations.** Any tool that modifies state (cell edits, file writes) is gated by an explicit confirmation step. The LLM emits a "pending_action" event; the user sees a diff and clicks Approve or Reject. No silent mutation.
5. **MCP as the tool surface.** The assistant's ~24 tools are defined in MCP (Model Context Protocol) JSON Schema format. This makes them portable to external MCP clients and consistent whether the assistant calls them directly or a subagent does.

12.2 Frontend: ChatPanel + ChatService

frontend/js/ChatPanel.js is the chat UI. It lives in the right panel, undockable. Responsibilities:

- Render the conversation as message bubbles (user vs assistant) with markdown, hljs syntax highlighting, KaTeX, and copy buttons.
- Show a model selector dropdown populated by /api/llm/health.
- Offer a "think" checkbox (enables extended-thinking mode for Claude) and a debug checkbox (opens the debug log panel).

- Render streaming tokens as they arrive, with a typing indicator.
- Show tool-call badges inline as the LLM invokes them.
- Show skill badges when auto-injected skills are surfaced.
- Render the user's token-usage meter (input/output/budget %).

Key methods:

- startStreamingMessage() - creates the container for the next assistant response.
- appendToken(token) - appends a token and re-renders markdown on each update.
- finalizeStreamingMessage(thinking) - applies syntax highlighting and inserts the collapsible "reasoning" section if Claude's thinking was included.
- appendToolBadge(toolInfo) - shows tool name + args.
- updateTokenUsage(usage) - updates the footer meter.

frontend/js/ChatService.js is the streaming client:

- On init, checks /api/llm/health, loads chat history, wires STT/TTS services.
- On send, POSTs to /api/llm/chat as SSE. Each data: line is parsed as JSON; tokens are appended; tool calls, pending actions, and skill badges are dispatched to ChatPanel.
- Parses <think>, <voice>, <tool_call> tags from the local LLM's output via a ThinkingParser (the local model is prompted to emit these explicitly; Claude uses native tool-use content blocks).
- Sends pending_action confirmations back to /api/llm/confirm when the user approves.

12.3 Backend: /api/llm/* router

backend/app/routers/llm.py exposes the HTTP surface:

- POST /api/llm/chat (line 202) - streaming SSE endpoint. Assembles context, runs the tool loop, streams tokens.
- POST /api/llm/confirm (line 689) - approves or rejects a pending write tool; resumes the stream with the result.
- GET /api/llm/health (line 864) - returns the list of available models plus the active one.
- POST /api/llm/model (line 854) - switch active model. If the target starts with claude-, NOTED_TERMINAL_SECRET must be provided (line 857) - the gate for API-cost models.
- GET/DELETE /api/llm/history/{client_id}/{project_id} - per-project chat history.

- POST /api/llm/complete - single-turn code completion (used by the autocomplete integration).
- GET/POST /api/llm/skills - list / retrieve skill metadata and content.
- POST/GET/DELETE /api/llm/debug - toggle debug logging and retrieve events.

12.4 The manager stack

backend/app/managers/ contains the LLM layer split across six files.

12.4.1 llm_router.py (120 lines)

LLMRouter is the backend switch. _is_anthropic(model_id) checks for the claude- prefix. _active_manager() returns the right manager. Public methods (chat_stream, chat, complete, health) delegate to the active manager with a thin adapter.

health() queries both backends and returns a merged model list so the frontend dropdown shows every option that is currently reachable.

12.4.2 anthropic_llm_manager.py (250+ lines)

Implements the Anthropic Messages API. ANTHROPIC_MODELS (line 40) lists the three active IDs: claude-sonnet-4-6, claude-opus-4-6, claude-haiku-4-5-20251001.

chat_stream() (line 156) POSTs to https://api.anthropic.com/v1/messages via aiohttp, streams the SSE response, and normalizes each chunk into the common event shape: {"choices": [{"delta": {"content": "..."}}]} for text, {"tool_call": {...}} for native tool-use content blocks.

Extended thinking is turned on by setting thinking={"budget_tokens": 8000} with temperature=1.0 when the user checks the Think box (lines 138-142).

Message normalization (lines 98-120) merges consecutive same-role messages because the Messages API rejects repeated user or assistant turns.

12.4.3 llm_manager.py (120 lines)

Implements the local Gemma 4 path via an OpenAI-compatible agent server (http://agent_server:7701 by default). chat_stream() POSTs to /v1/chat/completions with stream=True and yields chunks.

A single-quirk detail (lines 51-52): the Gemma 4 model occasionally hallucinates tool-call results; the fix is to set a stop token on <tool_call|> which the prompt template uses to separate tool emission from continued generation.

12.4.4 llm_context.py (200+ lines)

build_context_message(ctx, managers) (line 42) is the context assembly. It builds a single user-role message holding every relevant context block, returned as (message_dict, skill_names).

Context blocks it may include:

- _notebook_block - current notebook state: cells, indices, a selection of outputs.
- _file_block - in-memory editor state of any open file (up to 20 k chars).
- _run_block - the active MLflow run's metrics/params, or a summary of the active experiment.
- _config_block - the resolved Hydra config.

Skill injection (lines 92-99) asks the SkillRegistry for static skills whose triggers match the current context (e.g. notebook_cell_selected, mlflow_run_in_context, hydra_config_in_context). These are prepended to the system prompt automatically.

12.4.5 llm_tools.py (150+ lines)

TOOL_DESCRIPTIONS (line 20) lists the ~24 tools the assistant can call: MLflow queries, Airflow DAG inspection, DVC hash lookups, file ops, Hydra config queries, notebook mutations, skill retrieval, subagent invocation, write ops, lint diagnostics, web fetch.

parse_tool_call(text) (line 114) extracts <tool_call>{...}</tool_call> from text (local-LLM path). For Claude, native tool_use content blocks are parsed by anthropic_llm_manager.py directly.

is_write_tool(tool) is the predicate that gates confirmation: tools like update_cell, insert_cell, update_file, create_file, fix_lint_issues are write tools and trigger the pending-action flow. Read tools execute immediately.

execute_tool() dispatches to the right manager - notebook_mgr, mlflow_mgr, dvc_mgr, etc. - and returns a result string that the LLM sees as the tool's response.

12.4.6 llm_agents.py (150+ lines)

AgentRegistry (line 44) loads AGENT.md files from .noted/agents/. Each defines a subagent: name, description, model (default Haiku 4.5 for speed), tools (restricted set), max_tokens, system_prompt.

run_subagent(task, agent_name, managers) (line 115) runs a subagent as a fresh conversation with no parent history, a tool loop limited to MAX_AGENT_ROUNDS=4, and a compact summary as the return value. Always uses Anthropic (because subagents are typically small, fast, parallel tasks).

12.4.7 llm_memory.py and llm_debug.py

llm_memory.py - per-client per-project history. append(key, role, content), get_messages_for_llm(key), and get_compaction_input() for auto-summarization of old history.

llm_debug.py - a ring buffer of timestamped debug events (api, tool, skill, file, llm, context). When the user toggles debug in the UI, events are emitted to the frontend's debug panel. Critical for diagnosing why a specific tool call fired or a specific skill was injected.

12.5 Skills system

data/skills/ holds the skill library. Each skill is a folder containing SKILL.md:

```
---
name: mlflow-run-interpretation
description: Explain what a run's metrics and tags mean in context.
triggers: [notebook_cell_selected, mlflow_run_in_context]
priority: 2
max_tokens: 500
---
# When a run's metrics are shown ...

...skill content as markdown...
```

backend/app/managers/llm_skills.py parses the YAML frontmatter (SkillRegistry.load_skills, line 41) and exposes get_skill(name) for explicit retrieval and get_static_skills(conditions) for auto-injection.

Skill categories currently populated (~42 total):

- **Airflow (8)**: dag-creation, dag-overview, performance, scheduling, sweep-strategy, task-debugging, task-dependencies, trigger-config.
- **DVC (5)**: best-practices, checkout, lineage, sync-debugging, tracking, versioning.
- **Evidently (3)**: data-quality, drift-detection, monitoring.
- **Hydra (5)**: composition, groups, pipeline-integration, setup, sweep-design, templates.
- **MLflow (10)**: artifacts, hyperparameter-analysis, model-registration, reporting, run-comparison, run-debugging, run-interpretation, serving, snapshots, training-curves.
- **noted core (5)**: auto-instrumentation, coding-conventions, lineage, notebook-resolution, platform-overview.
- **General (2+)**: ml-workflow-guidance, python-linting, web-fetch, noted-troubleshooting.

Skills can reference each other via references/ subfolders. The registry exposes get_skill_reference(skill_name, ref_path) for following cross-references.

12.6 MCP tool surface

backend/app/mcp/tools.py (200+ lines) defines the tools in MCP JSON Schema format. The same definitions are consumed by the assistant's tool loop *and* (via MCP) by external MCP clients that connect to noted.

Read-tier tools (auto-execute, no confirmation):

- MLflow: get_experiment_runs, get_run_details, compare_runs.
- Airflow: list_dags, get_dag_status, get_task_log.
- DVC: get_dvc_* (status, file history).
- Files: get_file_contents, list_files, search_files.
- Hydra: get_hydra_config.
- Notebook: get_notebook_cells, scroll_to_cell.
- Knowledge: query_knowledge_graph, get_skill.
- Agents: run_agent.
- Web: fetch_url.

Write-tier tools (confirmation required):

- update_cell, insert_cell, batch_update_cells - notebook mutations.
- update_file, create_file - file writes.
- get_lint_diagnostics, fix_lint_issues - lint-driven edits.

12.7 Dynamic context router

backend/app/mcp/context_router.py is the budget manager. When the user sends a message, classify_domains(message, context) (line 118) scores the message against per-domain keyword lists (mlflow, airflow, dvc, files, hydra, notebook, linting, knowledge, skills, web).

select_tools(message, context, all_tools) returns the filtered tool list: tools that match the classified domains plus an always-included set (get_file_contents, get_notebook_cells, scroll_to_cell). This is the save-2000-tokens-per-turn optimization for Claude; the local LLM ignores it and gets all tools because its context is larger relative to the tool definitions.

12.8 Web fetch via Camoufox

backend/app/managers/web_fetch_manager.py (144 lines) wraps Camoufox (an anti-detect Firefox) as a singleton browser instance.

_ensure_browser() (line 45) recycles the browser after 50 requests or 1 hour to avoid memory bloat and to reset tracking state. fetch_url(url) (line 105) is the async wrapper -

renders the page, waits for domcontentloaded, extracts HTML, strips to text, truncates to ~10 k chars. Falls back to plain httpx if Camoufox is unavailable.

The LLM reaches this via the fetch_url tool. When invoked, the tool result (text + URL) is injected back into the conversation so the LLM can cite it in its answer.

12.9 Debug assistant

Distinct from llm_debug.py (which is the debug log), there is a llm_debug.py for per-cell guided debugging. It ties a cell execution's traceback + surrounding context into a targeted prompt that tries to localize the bug and propose a fix. Because it is a write-gated flow (any proposed fix is a update_cell tool call), it goes through the pending-action confirmation just like manual edits.

12.10 Operations

Add a new skill

1. Create data/skills/my-new-skill/SKILL.md with YAML frontmatter.
2. Pick triggers from the existing condition vocabulary (notebook_cell_selected, mlflow_run_in_context, hydra_config_in_context, etc.).
3. Save. The SkillRegistry reloads on next backend restart. For hot reload, POST /api/llm/skills/reload (if exposed - otherwise restart).
4. Verify auto-injection by triggering the matching context and checking the skill badges in the ChatPanel.

Add a new tool

1. Define the tool in backend/app/mcp/tools.py with MCP JSON Schema.
2. Add an if tool_name == "..." branch in llm_tools.execute_tool() that delegates to a manager method.
3. Decide read-tier vs write-tier (is_write_tool(tool)).
4. If write-tier, the pending-action flow is automatic.
5. Update the context_router's domain classifier if the new tool should be context-scoped.

Switch from local to Claude

1. Ensure ANTHROPIC_API_KEY env var is set for the backend container.
2. POST /api/llm/model with {"model": "claude-sonnet-4-6", "secret": "..."}.
3. The frontend dropdown reflects the new active model on the next health refresh.

Investigate why a skill was not injected

1. Enable the debug panel (POST /api/llm/debug { "enabled": true }).

2. Send the triggering message.
3. In the debug panel, look for skill category events. Each line shows the skill name and the trigger conditions that were satisfied.
4. Cross-reference with SkillRegistry's loaded skills (GET /api/llm/skills).

Inspect a tool call in flight

1. With debug enabled, send a message that should invoke a tool.
2. Watch for tool category events: tool_call_start, tool_result, timings.
3. If the tool raised, the traceback is in the tool_result event.

12.11 Discussion-ready talking points

Q: Why wire both local and Claude instead of picking one? A: Because the two have different strengths and different costs. Local Gemma 4 is free and offline; Claude is more capable and pay-per-token. The ability to swap mid-conversation lets users iterate quickly on local for cheap turns and escalate to Claude for difficult ones. Forcing one path would either price out small experiments or cap the ceiling on complex reasoning.

Q: Why is the context router a separate layer? A: Because context shape differs between backends. Claude has a 200k context window and can accept large tool definitions, but tokens are billable. Gemma's context is tighter, but tokens are free. The router lets each backend see the right shape: Claude gets a filtered tool list (save money); Gemma gets the full tool list (use the context you have). Bolting context decisions into each manager would duplicate logic.

Q: Why are skills a file-based registry instead of a database? A: Same reason the project registry is file-based (Chapter 8.8). Skills are co-located with the project's state, reviewable in git, and editable with the same tools that edit code. A database would require a separate admin UI. Scaling to thousands of skills would eventually warrant indexing; at 42 skills, plain Markdown is the right shape.

Q: Why do write tools require confirmation? A: Because the assistant will sometimes propose changes that are wrong, plausibly wrong, or right-but-surprising. Unconditional auto-apply would create a class of bugs where the user does not know what changed until something breaks later. Pending-action + diff keeps the user in the loop without forcing them to hand-write every change the LLM suggests.

Q: How does the MCP tool layer help external integrations? A: Because MCP is a portable spec. The same tool definitions that power noted's own assistant can be served to an external MCP client (e.g. a VS Code extension with MCP support). The plumbing is already there; exposing it is primarily an auth and transport concern. This is why the tool definitions live in backend/app/mcp/tools.py rather than in llm_tools.py.

Q: Why is run_agent a tool rather than a direct backend feature? A: Because the assistant chooses when to delegate. Subagents are useful for tasks that are independent, parallelizable, or that benefit from a fresh context (e.g. "research how to do X"

vs. “refactor this notebook”). Letting the LLM decide to delegate gives better outcomes than hardcoding the decision into the backend. Agents registered in .noted/agents/ act as the specialization vocabulary.

Q: How does the conversation memory handle long sessions? A: llm_memory.py has a get_compaction_input() method that returns old messages when history exceeds a threshold. The assistant is prompted to compact them into a summary, which replaces the old messages in storage. This is the same pattern most chatbots use; noted’s version is per-client-per-project so the compaction scope is natural.

Q: What is the failure mode when the agent_server is down? A: The local backend’s health check fails, the model dropdown shows only Claude options (if the key is set) or nothing, and any attempted local call returns a clear error. No silent fallback; the user has to pick a working model explicitly.

13. Knowledge Graph

13.1 Concept primer

The Knowledge Graph is noted's answer to "show me how everything connects". Each MLOps subsystem produces its own entities: MLflow has runs and models, DVC has datasets and versions, Airflow has DAGs and tasks, Hydra has configs and groups, noted has projects and notebooks. Each subsystem also has its own navigation surface. The Knowledge Graph is a *unified view* over all of them - one screen where a user can see a run's training data, its config, its DAG run, its champion promotion, its Evidently snapshot, and its downstream serving state, all as nodes and edges.

Three properties drive the design:

1. **Read-only aggregation.** The graph service owns no ground truth. It scans MLflow, Airflow, DVC, Hydra, and the filesystem on demand, assembles an entity-relationship graph, caches it, and serves it to the frontend. Mutations to the underlying data are done through the original subsystems; the graph reflects them on the next scan.
2. **Perspectives as filters.** A full graph with 100+ runs quickly becomes unreadable. Perspectives (Lineage, Performance, Versioning, Pipeline, Overview, Tags) are named filters that surface only the entity types and relationships relevant to a specific question. The same underlying graph is rendered six different ways.
3. **Three.js force-directed 3D.** The visual is a WebGL scene with force-directed layout. It looks more interactive than a static diagram and scales to hundreds of nodes without becoming illegible. Clicking a node opens a draggable detail panel; clicking "Open in Explorer" teleports back to the relevant noted view.

The graph is a **separate microservice** (noted-graph container) rather than part of the backend. This isolates its dependencies (it does not need any ML libraries) and keeps its scan operations from blocking the main backend's event loop.

13.2 The noted-graph service

graph/Dockerfile (14 lines) is Alpine Linux + Python 3.12. It installs a minimal set: fastapi, uvicorn, requests, pyyaml. No ML libraries, no socket.io, no heavy dependencies. Listens on port 5523.

docker-compose.yml defines the service:

```
noted-graph:
  build: ../graph
  container_name: noted-graph
  environment:
    MLFLOW_TRACKING_URI: http://mlflow:5000
    AIRFLOW_API_URL: http://noted-airflow-apiserver:8080
```

```
    AIRFLOW_BASE_PATH: /airflow
    AIRFLOW_USERNAME: airflow
    AIRFLOW_PASSWORD: airflow
    NOTED_API_URL: http://noted:8123
    PROJECTS_DIR: /app/data/projects
    MOUNTS_DIR: /app/mounts
    GRAPH_PORT: "5523"
  volumes:
    - ../data:/app/data:ro
  ports:
    - "5523:5523"
```

The service reads data/ read-only and calls MLflow / Airflow / noted's REST APIs for the rest of its inputs.

13.3 Entity and edge model

graph/app/models.py (lines 7-31) defines two pydantic models: Entity and Graph.

Entity is {id, type, label, properties, tags}. The id is "{type}:{source_id}" - e.g. run:abc123..., data_file:jena_climate_2012.csv, model_version:Jena Weather Forecaster:v7. The prefix disambiguates across sources; the source_id is the primary key in the source system.

Entity types (from the design doc):

- **Projects** - project, file, notebook.
- **MLflow** - experiment, run, snapshot, model, model_version, tag.
- **DVC** - data_file, data_version.
- **Hydra** - config, config_group, config_option.
- **Airflow** - dag, dag_task, dag_run.
- **Environment** - environment (virtual env metadata).

Relationship types (edges):

- contains, belongs_to, version_of, snapshot_of.
- produces, uses_data, uses_config, executed_by, executed_as.
- has_task, depends_on (DAG topology).
- parameterized_by, runs_in, tagged_with.
- promoted_to, derived_from, code_at, scheduled_as.

Each relationship carries a properties dict with the metadata that links the endpoints (e.g. {hash: "sha256:abc..."} on a uses_config edge so a user can see *which* config hash ties the run to that config group).

13.4 Scanners: populating the graph

graph/app/graph_builder.py:24-88 orchestrates five scanners, run in sequence:

1. **filesystem_scanner.py** - walks PROJECTS_DIR and MOUNTS_DIR, discovers projects, notebooks, files. Produces project, notebook, file entities and contains edges.
2. **mlflow_scanner.py** (lines 18-80+) - queries MLFLOW_TRACKING_URI. Discovers experiments (line 26), runs (line 40), snapshots (runs tagged with noted.snapshot=true, line 52), registered models (line 71), and versions. Emits experiment, run, model, model_version, snapshot entities plus belongs_to, snapshot_of, version_of, promoted_to edges.
3. **dvc_scanner.py** - parses .dvc files plus git log to discover tracked files and historical versions. Emits data_file + data_version entities.
4. **hydra_scanner.py** - scans each project's config/ directory. Emits config, config_group, config_option entities plus contains edges.
5. **airflow_scanner.py** (lines 21-80+) - queries Airflow's REST API (GET /dags, /dags/{id}/tasks, /dags/{id}/dagRuns). Emits dag, dag_task, dag_run entities plus has_task, depends_on, executed_as edges.

After scanning, relationship_resolver.py (lines 17-258) builds cross-source edges. Its job is "this MLflow run has a noted.hydra_config_hash tag, find the config node with that hash, add a uses_config edge". Key resolvers:

- run -> data via dvc.data_hash tag (line 60).
- run -> config via noted.hydra_config_hash tag (line 90).
- snapshot -> commit via git (line 114).
- dag_run -> mlflow_run via run_id in task logs (line 131).
- notebook -> environment via venv name (line 150).
- project -> experiment via naming convention (line 232).

This is the layer that makes the graph *a graph* rather than five disconnected per-source clusters.

13.5 Perspectives: filtered views

graph/app/views.py:18-86 defines BUILTIN_VIEWS:

- overview - top-level entities (project, experiment, dag, model) with radial layout.
- lineage - data_file, run, model_version focus with emphasized uses_data, produces, uses_config. Hierarchical layout.
- performance - run + snapshot with color-by-metric and size-by-metric.

- versioning - data_version, model_version, snapshot, timeline layout, color-by-recency.
- pipeline - dag, dag_task, dag_run, hierarchical layout, color-by-status.
- tags - user-selected tags define the view dynamically.

apply_view(graph, view) (lines 91-131) takes the full graph and:

1. Filters out entities whose type is not in the view's primary/secondary sets.
2. Filters out relationships whose endpoints no longer exist after step 1.
3. Annotates each remaining entity with _view_role (primary / secondary / tertiary) - used by the frontend for color and size.
4. Annotates relationships with _emphasized if they are in the view's emphasized list.

Custom perspectives are persistable. views.py:135-214 reads and writes .noted/graph_views/*.json per project. save_custom_view writes; list_views merges built-ins and custom.

13.6 Three.js rendering

frontend/js/knowledge-graph/KnowledgeGraph3D.js is the WebGL scene.

Construction (line 13): Three.js scene + camera + WebGLRenderer (lines 61-72), OrbitControls (line 76), ambient + 2 directional lights (lines 83-90). Mesh bookkeeping via nodeMeshes dict, edgeLines array, HTML overlay labels dict.

Force-directed layout (_computeForceLayout(), lines 142-229):

- Initialize positions uniformly random in a sphere.
- For 200 iterations:
  - For each node pair, compute repulsion F = repulsion / dist^2.
  - For each edge, compute attraction toward the other endpoint.
  - Apply a weak gravity toward the origin.
  - Damp velocity by 0.85 per step.
- Scale final positions by 0.15.

Node rendering (_createNode(), lines 233-276) picks geometry by entity type from ENTITY_STYLES (sphere / box / cylinder / octahedron / cone) and material by view role (primary = saturated, secondary = medium, tertiary = dim).

Edge rendering (lines 280-302) uses THREE.LineBasicMaterial. Emphasized edges are blue; normal edges are gray; dotted where semantically weaker.

Interactions:

- Hover (lines 341-479) raycasts per mouse move, scales hovered node, shows a floating detail panel.
- Click pins the detail panel and calls onEntityClick(entity).
- Drag a node (lines 407-430) moves it via a plane intersection; on release, runs a 90-frame settling simulation.

The detail panel (_showDetailPanel, lines 483-583) is an HTML overlay with the entity's icon, label, property rows, and two buttons: "Open in Explorer" (calls onEntityNavigate(entity)) and "Pin". The panel is draggable.

13.7 Graph proxy

backend/app/routers/graph_proxy.py (42 lines) is a tiny pass-through. A single catch-all endpoint:

```
@router.api_route("/{path:path}", methods=["GET","POST","PUT","DELETE"])
```

Forwards all /api/graph/* requests to GRAPH_URL (env var, default http://noted-graph:5523). Preserves query params. Returns 503 on ConnectionError.

This keeps the frontend from having to know the graph service's hostname and lets the backend transparently add auth or caching later.

13.8 Knowledge Graph endpoints

graph/app/routers/graph.py (lines 27-128):

- GET /graph/{project_id} (line 32) - full scanned graph with entities + relationships.
- GET /graph/{project_id}/neighborhood/{entity_id} (lines 54-80) - BFS N hops from a seed entity; used by the "show neighborhood" action.
- GET /graph/{project_id}/entity/{entity_id} (line 110) - single entity + direct relationships.
- POST /graph/{project_id}/invalidate (line 126) - invalidate cache; forces a rescan next request.

graph/app/routers/search.py (lines 28-51):

- GET /search/{project_id}?q=... - text match on entity labels and properties. Supports type filtering, metric threshold queries (val_loss < 0.1), and tag queries (#deployed).

graph/app/routers/views.py (lines 14-71):

- GET /views/{project_id} - list all views (built-in + custom).
- GET /views/{project_id}/{view_name} - graph filtered by the named view.
- POST /views/{project_id} - save a custom view.

- DELETE /views/{project_id}/{view_name} - remove a custom view.

13.9 Perspectives UI

frontend/js/knowledge-graph/GraphPanel.js (lines 22-146) is the panel shell. A jsPanel-hosted window with:

- A search input and search button (line 57/64).
- A view selector dropdown with the built-in views (lines 71-81).
- A refresh button that invalidates the graph cache and re-scans.
- A 3D rendering area wired to KnowledgeGraph3D.
- An info bar showing entity count, relationship count, and current view name (line 93-96).

On view change (line 113), the panel fetches GET /api/graph/views/{project_id}/{view_name} and re-renders.

Search result dropdown (lines 193-229) shows up to 15 matches; clicking a result highlights the entity in the 3D scene.

13.10 Dagre and hierarchical layouts

The design doc mentions dagre for hierarchical/left-to-right layouts in the Lineage and Pipeline views. At v0.1 of this manual, the layout algorithm is force-directed only - dagre is a planned enhancement. When implemented, it will apply to specific views where the node type ordering has a clear topological meaning.

13.11 Operations

Rebuild the graph

1. Click the refresh button in the GraphPanel, or
2. POST /api/graph/{project_id}/invalidate, then reload.

A rescan takes a few seconds for a small project, ~20s for 100+ runs.

Add a new entity type

1. Define the type in the scanner responsible for its data source.
2. Give it a unique id prefix (mytype:).
3. Emit {id, type, label, properties, tags} from the scanner.
4. Add visual style to ENTITY_STYLES in the frontend (GraphNodeRenderer.js).
5. Decide which built-in views should show it and update their primary/secondary lists in views.py.
6. Rebuild.

Add a new perspective

1. Define the view in BUILTIN_VIEWS in views.py with primary, secondary, emphasized, layout, optional color_by/size_by.
2. Add an option to the GraphPanel's view selector.
3. Test with a populated graph.

Debug a missing edge

1. Pick two entities that should be connected.
2. Check the relationship_resolver.py logic for the relevant edge type - it is the most likely source of "should have but didn't".
3. Verify the tags/properties on both endpoints: for a uses_config edge, both the run and the config must share the same noted.hydra_config_hash.

13.12 Discussion-ready talking points

Q: Why a separate microservice instead of bolting the graph into the main backend? A: Because the scan operations are I/O-heavy (multiple HTTP calls to MLflow, Airflow, plus filesystem walks) and do not share much with the rest of noted's backend. Running them in the main process would block the event loop on every rebuild. The isolated service can cache aggressively, scan asynchronously, and fail independently without affecting notebook execution.

Q: Why Three.js instead of a 2D graph library like D3? A: Because 3D gives more visual room before overlapping edges become unreadable. With hundreds of nodes, 2D layouts quickly degrade; 3D with force-directed layout and a rotatable camera lets the user resolve clutter by viewing angle. The cost is that 3D requires a WebGL-capable device; the fallback (an HTML table of nodes and edges) is available for environments that do not support it.

Q: Why read-only scans instead of real-time subscriptions? A: Because none of the source systems (MLflow, Airflow, DVC) emit structured change events that the graph could subscribe to. Polling them individually would multiply traffic; scanning on demand with a cache is simpler and bounded. The invalidate endpoint lets the user force a refresh when they know they changed something.

Q: Why are perspectives baked into the backend instead of computed on the frontend? A: Because the frontend has to render every node and edge it receives. Filtering in the backend reduces payload size significantly - a typical Lineage view is 10-15% of the full graph. This matters both for network transfer and for the Three.js scene construction cost.

Q: How do you handle the case where MLflow has hundreds of runs? A: Two mitigations. (1) Time range filters (a query param filters runs by date). (2) Neighborhood queries (/graph/.../neighborhood/{entity_id} returns a bounded subgraph around a seed). The Overview view avoids showing individual runs at all - it aggregates at the experiment level.

For a true “show me everything”, the force layout settles on clumps that are still navigable by zoom.

Q: What is the relationship between the Knowledge Graph and the Composer? A: Both read from the same MLflow tags and Hydra bundles. The Composer surfaces one run at a time for editing; the Graph surfaces many runs for context. They are complementary views over the same data. A future feature would let the user right-click a run in the graph and “Load into Composer” to immediately edit its config.

Q: Impact Analysis - what breaks if I change this? A: Mentioned in the README roadmap as T-5.1. The idea: right-click a node, get a directed BFS traversal downstream (what depends on this?) and upstream (what does this depend on?). For a data_file node, downstream traversal shows every run that used that file, every model promoted from those runs, every deployment that serves those models. Not yet implemented; the data is already in the graph.

Q: How does the graph interact with the AI assistant? A: Via the query_knowledge_graph tool (Chapter 12.6). The LLM can ask “what runs use the jena_2012_dataset?” and the tool runs a filtered query against the graph service. This is the canonical way for the assistant to reason about lineage without having to re-implement scanning logic.

Q: What is the maximum graph size that the 3D scene can handle? A: Empirically, ~500 entities render smoothly; ~2000 become sluggish on mid-range hardware. Beyond that, the force-directed iteration cost dominates. The mitigation is to use views to filter before rendering; Overview is explicitly designed to keep node count bounded by aggregating at the experiment level rather than the run level.

14. Multi-Language Runtime

14.1 Concept primer

noted supports multiple programming languages via a **strategy pattern** applied consistently across three concerns: the LSP server (language intelligence: completion, diagnostics, goto-def), the DAP adapter (debugging: breakpoints, stepping, variable inspection), and the package manager (install / remove / list libraries). A new language is added by authoring three strategy classes plus a runtime.json manifest; the rest of the backend routes to the right strategy by looking up the runtime's language_id.

Four design properties are worth naming:

1. **Per-language kernel commands.** Each runtime has a kernel_cmd in its runtime.json. The backend's KernelManagerService (Chapter 8.5) templates that command with the project path and starts the kernel. Python uses ipykernel; R uses ark; JavaScript uses ijskernel. The kernel protocol (ZMQ + Jupyter messaging) is the same across all three.
2. **Per-language LSP servers.** CodeMirror in every cell connects to a WebSocket LSP proxy that forwards JSON-RPC to the language's LSP server. Python uses ruff (lint) + jedi (completion). JavaScript uses biome + typescript-language-server. R uses the languageserver R package.
3. **Per-language DAP adapters.** Python via debugpy over the Jupyter control channel. JavaScript via vscode-js-debug over TCP. R DAP is deferred (Section 14.4).
4. **Per-language package managers.** Python = pip or uv; JavaScript = pnpm; R = renv. Each is implemented as a subclass of BasePackageManager with a common install_stream() interface so the UI renders identical progress output regardless of language.

The runtime metadata is file-based. data/runtimes/{language}/{version}/runtime.json declares a language version; data/environments/{language}/{version}/{env_name} holds per-env state (installed packages, launcher scripts). This keeps the runtime registry inspectable and editable without a database.

14.2 Language strategies

backend/app/managers/language_strategies.py (530 lines) defines the base class and three concrete implementations.

BaseLanguageStrategy (the abstract class) has four responsibilities:

- Expose the list of LSP server configs: [{server_type, command, args}, ...].
- setup_debug(session) - start a DAP transport against the running kernel.
- wrap_code(code, cell_id) - prepare code for execution (e.g. inject filename metadata for debug lookups).

- enrich_diagnostic(diag) - transform language-server-raw diagnostics into noted's display shape.

The registry at the bottom of the file (line 516) is a dict mapping kernel_language to the strategy instance: "python" -> PythonStrategy(), "javascript" -> JavaScriptStrategy(), "r" -> RStrategy().

14.2.1 PythonStrategy (lines 84-308)

- LSP servers: ruff server --preview (linting, formatting) and jedi-language-server (completion, hover, goto). Both run in the project's venv.
- DAP: creates a BlockingKernelClient on the Jupyter control channel, wraps it in ZMQDebugTransport. debugpy runs inside the kernel; the control channel tunnels DAP messages.
- wrap_code() injects filename/line metadata on every cell so debugpy's setBreakpoints can map breakpoints to the right source.

14.2.2 JavaScriptStrategy (lines 310-476)

- LSP servers: biome lsp-proxy (linting, formatting) and typescript-language-server --stdio (completion, hover, refactor).
- DAP: launches vscode-js-debug, which opens a TCP port on the V8 Inspector protocol. TCPDebugTransport relays DAP JSON-RPC over that port.
- build_debug_all_script() processes cell markers (// %%) into boundary markers so a "Run All" with breakpoints can map each breakpoint to the right cell.

14.2.3 RStrategy (lines 478-512)

- LSP server: R --slave --no-save -e "languageserver::run()". Single server providing completion, hover, formatting. Diagnostics from lintr are enriched into the message + label shape (lines 78-88) so the UI can render rule codes cleanly.
- DAP: stubbed. setup_debug() raises NotImplementedError. Reason discussed in Section 14.4.
- wrap_code() only adds line padding; no debugpy-like instrumentation is available.

14.3 Python runtime

Python 3.12 is the primary runtime. The runtime.json (at data/runtimes/python/3.12/runtime.json) declares:

- executable: python3.12
- env_create_cmd: {executable} -m venv {env_path}
- kernel_cmd: {env_path}/bin/python -m ipykernel_launcher -f {connection_file}
- kernel_language: python

- env_post_create_cmds: install ipykernel, mlflow, hydra-core after venv creation.

Package manager is pip or uv (pip_manager.py, 151 lines). The default is uv pip install --python {env_path}/bin/python for speed; legacy pip is kept as a fallback. install_stream() runs the install in a PTY, parses the output line-by-line, and forwards progress events over the WebSocket.

LSP routing uses jedi for navigation (completion, hover, goto-definition) and ruff for diagnostics. The LSP proxy rewrites virtual URIs (cell references) to real file URIs before handing them to jedi, then rewrites them back in the response (python_strategy.py:98-99).

DAP routing: debugpy listens inside the kernel. The dap_manager.py layer opens a TCP socket to debugpy via the Jupyter control channel; breakpoints, step events, and variable inspections flow through that tunnel.

14.4 R runtime

R is Phase 2-complete: kernel and LSP work; DAP is deferred. Six versions are supported: **3.6.3, 4.0.5, 4.2.3, 4.3.3, 4.4.2, 4.5.1**. Each has its own runtime.json at data/runtimes/r/{version}/runtime.json.

14.4.1 The ark vs IRkernel split

R notebooks in noted use **ark** (/usr/local/bin/ark) as the kernel, not the more commonly known IRkernel. The reasons:

- ark is Positron's native R notebook kernel - a Rust binary that wraps R via system() calls. It is faster, has better session management, and integrates with Positron's graphics display pipeline.
- IRkernel was considered and rejected because its package discovery and startup profile was slower, and because ark's output channel semantics line up more cleanly with noted's IOPub dispatcher.

LSP is a separate process: the languageserver R package runs via R --slave --no-save -e "languageserver::run()". ark is used only for execution; LSP is independent.

R versions 3.6.3 and 4.0.5 do *not* have languageserver available on CRAN. lsp_manager.py:169-178 checks the binary at startup and falls back to "kernel-only mode" for those versions - notebooks still execute but without LSP features.

14.4.2 Why R DAP is deferred

T-5.R6 (R Debug) was planned but deferred. Three blocking issues:

1. **ark does not expose DAP outside Positron.** Positron's DAP integration lives in Positron's Rust-side IPC layer, not in ark itself. Extracting it would require either contributing upstream or re-implementing DAP on top of ark's existing protocol.
2. **vscDebugger reverse-protocol.** The only standalone R DAP implementation is vscDebugger, which uses a reverse startDebugging pattern requiring child session

spawning for the R evaluator subprocess. Wiring this through noted's proxy is non-trivial.

3. **Protocol translation.** Even if the above were solved, ark's internal debug messages and vscDebugger's DAP output use different schemas. Translating between them requires writing a compatibility layer that has to stay in sync with upstream changes.

The trade-off: users debug R by inserting print() or browser() statements. Full DAP support is queued pending either a Positron ark contribution or a vscDebugger release that ships a standalone DAP server.

14.4.3 R environment setup

Each R env has an auto-generated bin/Rscript launcher (env_manager.py post-create step) that injects:

- R_HOME, LD_LIBRARY_PATH - point at the correct R installation.
- RENV_PATHS_* - point at the env's renv library.
- RENV_CONFIG_SYNCHRONIZED_CHECK=FALSE - disables renv's startup check that would otherwise slow down every kernel launch.

renv_manager.py (170 lines) implements the R package manager: list_packages() reads renv/library DESCRIPTION files; install_packages() spawns Rscript -e "renv::install('pkg')" with line-buffered output; remove_packages() calls renv::remove() followed by renv::snapshot() to pin the change.

14.5 JavaScript runtime

JavaScript uses **ijskernel** (an npm-installed kernel) as the execution engine.

Runtime manifest (runtime.json): - kernel_cmd: {env_path}/node_modules/.bin/ijskernel --protocol=5.1 {connection_file} - env_create_cmd: {executable} -m init (pnpm init for the project env). - package_manager: pnpm.

LSP: biome for linting and formatting; typescript-language-server for completion and navigation. Both are installed into the project's pnpm workspace so their versions track the user's own typescript/biome versions.

DAP: vscode-js-debug listens on a TCP port once the kernel is put into debug mode. TCPDebugTransport in javascript_strategy.py:363-393 relays DAP messages over that port. build_debug_all_script() rewrites cell markers into file-line mappings so breakpoints set in one cell hit correctly during a Run All execution.

Package manager is pnpm (pnpm_manager.py, 146 lines). The normalization step (lines 41-49) converts pnpm's {dependencies: {name: {version}}} dict layout into the flat [{name, version}] shape that noted's UI expects.

14.6 Package manager strategy

backend/app/managers/package_managers/ holds the per-language implementations plus a base class.

base.py defines:

- PmContext - a dataclass carrying the runtime spec, env path, a template resolver, and process registration callbacks (for cancellation).
- BasePackageManager - abstract class with list_packages(), install_packages(), install_stream(), remove_packages().

Concrete subclasses:

- pip_manager.py - Python. Uses uv pip by default with pip fallback. PTY-based output streaming.
- pnpm_manager.py - JavaScript. PTY-based.
- renv_manager.py - R. Uses Rscript -e "renv::install(...)" with stdout readline (no PTY; R's output is line-buffered enough to avoid TTY detection).

EnvironmentManager.install_packages() (env_manager.py:599) dispatches by the runtime's language field. A new language's package manager is registered via the same lookup.

14.7 Environment management

backend/app/managers/env_manager.py (600+ lines) is the top-level env owner.

Key subsystems:

- **RuntimeRegistry** (lines 12-75) - scans data/runtimes/ at startup, loads every runtime.json, validates required fields, exposes get_runtime(language, version).
- **EnvironmentManager** (lines 77-600+) - per-runtime env lifecycle. Discovers envs via recursive scan of data/environments/{lang}/{ver}/{env}/. Creates envs via the runtime's env_create_cmd + env_post_create_cmds. Generates per-env launcher scripts (e.g. R's bin/Rscript).

The flat-to-hierarchical migration (lines 88-108) handles older installs where envs lived at data/environments/{name} without language/version subdirs. On startup, orphaned flat envs are moved under python/3.12/.

Venv repair (lines 140-245) runs on every backend startup for Python venvs. Symlink targets and shebang lines are fixed up in case the Python binary moved (common after container rebuilds). This avoids forcing users to recreate venvs after image changes.

backend/app/managers/venv_manager.py (106 lines) is a thin legacy wrapper over EnvironmentManager that preserves the older flat-name API for any code that still uses it.

14.8 LSP proxy

backend/app/routers/lsp.py (150+ lines) is the WebSocket endpoint. One connection per client per server. The proxy:

1. Accepts the connection with a server_type query parameter (jedi, ruff, biome, typescript, r).
2. Resolves the project, env, and runtime via the ProjectRegistry.
3. Asks the language strategy for the (command, args) to launch the server.
4. Spawns the LSP server subprocess with that env's activated PATH.
5. Relays JSON-RPC messages between browser WebSocket and server stdio (Content-Length framing on both sides).

LSPProxyManager (lsp_manager.py:131-200+) caches server subprocesses per (project_id, env_name, server_type) tuple so rapid reconnections reuse the same process.

Diagnostic enrichment (lsp.py:30-49) calls the strategy's enrich_diagnostic(). For Python/ruff, the output is "rule-code|category|message" with \x1f separators, letting CodeMirror render rule codes as clickable pills.

Frontend side: frontend/js/CellEditor.js connects via codemirror-languageserver, which speaks LSP over the WebSocket and populates the gutter, the hover tooltip, the autocomplete menu, etc.

14.9 DAP proxy

backend/app/routers/dap.py is the DAP WebSocket endpoint. Two transport types, both DAP over Content-Length framing:

- **Python: ZMQ tunnel.** ControlChannelDispatcher (dap.py:35-104) multiplexes DAP requests/replies over the Jupyter control channel. A single-reader pattern routes responses by msg_id because the Jupyter control channel does not support multiple concurrent receivers.
- **JavaScript: TCP.** DAPProxyManager (dap_manager.py:141-188) opens a TCP connection to vscode-js-debug's V8 Inspector port.

Cell-to-file mapping:

- Python pre-processes the setBreakpoints request: it calls the dumpCell kernel command to create a temp file for the cell (language_strategies.py:211-223), then maps the breakpoint file URI to the temp file.
- JavaScript writes each cell's code to /tmp/noted_js_cell_{hash}.js at execute time, and breakpoints set in those files just work.

Debug session lifecycle:

1. setup_debug(session) creates the transport.

2. handle_handshake(session) sends initialize, attach, and configurationDone.
3. Relay loop shuttles messages between WebSocket and transport until disconnect.
4. Teardown calls disconnect on the adapter and closes the transport.

14.10 Discussion-ready talking points

Q: Why a strategy pattern instead of per-language branches in the router? A: Because each concern (LSP, DAP, PM) has its own dispatch shape and adding a language means updating N routers. The strategy pattern centralizes per-language knowledge in one class, makes language additions local (write a new strategy, register it), and keeps routers ignorant of which language they are serving.

Q: Why is ark preferred over IRkernel for R? A: Because ark is purpose-built for notebook-style execution (originally by Posit for Positron) and has better graphics handling, faster startup, and cleaner output channel semantics. IRkernel is fine but older; the trade-off was worth the dependency on a Rust binary that needs to be in the image.

Q: What is the cost of not having R DAP? A: Moderate. Users debug R by browser() statements or print() inspection. For small notebooks this is adequate; for complex R codebases it is a visible gap. The deferral is pragmatic - the integration work does not fit in the Tutorial 3 timeline, and R users in the immediate cohort have not flagged it as blocking.

Q: Why separate runtime.json files per version instead of a single multi-version file? A: Because each version may have different kernel commands, different post-create steps, different LSP availability. Keeping them separate means adding a new version is copy-paste + edit, no schema changes. It also lets individual versions be removed without affecting others.

Q: Why is venv repair idempotent and automatic? A: Because container rebuilds are routine. Every rebuild of the main noted image moves /usr/local/bin/python3.12 to a new inode; Python venv symlinks encode the old inode. Without automatic repair, every rebuild would force users to recreate every Python env, which is noisy. The repair path is O(number of envs) on startup and is negligible in practice.

Q: Why use PTY for pip/pnpm streaming but not R? A: Because pip and pnpm detect whether stdout is a TTY and change their output shape accordingly. Without a PTY, their output is flat and hard to parse; with a PTY, they emit progress bars and richer output. R's renv uses line-buffered stdout and does not care about TTY, so a simpler readline loop suffices.

Q: How does the LSP proxy handle multiple clients on the same file? A: Each WebSocket connection gets its own server subprocess. The LSP protocol is inherently per-client (the server maintains per-document state for each client), so sharing across connections would require conflating their document states. The subprocess cost is acceptable for a small-team, single-user deployment; at scale, a shared server with client-ID-prefixed documents would be the next step.

Q: What about adding Rust, Go, Julia support? A: Each would need (1) a kernel (there are community Jupyter kernels for all three), (2) LSP integration (rust-analyzer, gopls, LanguageServer.jl), (3) DAP integration (lldb-dap for Rust, delve for Go, JuliaInterpreter for Julia), (4) a package manager adapter (cargo, go mod, Pkg.jl). The plumbing is there; each language adds a few hundred lines of strategy code plus a runtime.json.

15. Infrastructure & Deployment

15.1 Concept primer

noted ships as a **Docker Compose stack** of 13 services. The stack is designed to run on a single host - either the user's workstation or a small single-node server - without requiring Kubernetes, a service mesh, or a managed database. A handful of named volumes persist state; the rest is rebuildable from the compose file.

Three architectural choices drive every infrastructure decision:

1. **Local-first.** Every service binds to the same Docker network (noted-network) and addresses peers by hostname. There is no external service dependency: MLflow, MinIO, Airflow (with its Postgres + Redis), Evidently, the Knowledge Graph, and the serving container all live inside the compose stack.
2. **File-first state.** The filesystem is the source of truth for projects, notebooks, configs, and data. Databases (Postgres for Airflow, SQLite for MLflow) are internal plumbing; the user-facing state lives in files that git can version. This is the property that makes NOTED.md and documents.json work as registries.
3. **Filesystem + process isolation, not network isolation.** Each service's private state (Postgres data, MLflow tracking dir, MinIO buckets) sits in a named volume. Containers do not share volumes unless they must. Network isolation inside the compose network is flat (every service can reach every other service) because everything is local.

The compose file is the canonical deployment artifact. docker-compose.yml in services/ is what a reviewer or a new operator reads to understand what runs where.

15.2 The compose graph

services/docker-compose.yml declares the services. Grouped by role:

Noted core

- noted - the main backend + frontend. Image logus2k/noted, built from the repo root. Ports 8123 (HTTP) and 3719 (websocket-only in some deployments).

Model lifecycle

- mlflow (ghcr.io/mlflow/mlflow:latest) - tracking server, port 5000, SQLite backend at /mlflow/mlflow.db, artifact root /mlflow/artifacts. Command line installs plotly at runtime for chart rendering in the MLflow UI.
- minio (minio/minio:latest) - object storage, ports 9000 (S3 API) and 9001 (web UI). Used for DVC remote storage and MLflow artifacts if configured.

Pipelines

- postgres (Postgres 16) - Airflow metadata DB.

- redis (Redis 7.2-bookworm) - Airflow Celery broker.
- airflow-apiserver (port 8080), airflow-scheduler, airflow-dag-processor, airflow-worker (Celery worker), airflow-triggerer - the full Airflow 3.x worker stack.
- airflow-init - one-shot DB migrations + admin user creation; terminates after startup.
- airflow-cli (profile debug) - ad-hoc Airflow CLI container for debugging.
- flower (profile flower) - Celery monitoring UI, port 5555.

Monitoring

- evidently (evidently/evidently-service:latest) - data monitoring, port 8009:8000. Persists workspace in the evidently-data volume (Chapter 5.3.1).

Auxiliary services

- noted-graph - the Knowledge Graph service (Module 13), port 5523.
- noted-serving - the model serving container (Module 11), port 5522.

Each service attaches to the shared noted-network. Hostnames inside the network mirror the service names: mlflow, minio, noted-evidently, noted-graph, noted-serving, etc.

15.3 Named volumes

Four named volumes at the bottom of docker-compose.yml persist state across container rebuilds:

- postgres-data - Airflow's metadata database.
- mlflow-data - MLflow's SQLite DB plus artifact root (runs, models, Logged Models).
- minio-data - MinIO buckets (DVC storage, any artifact stores backed by S3).
- evidently-data - Evidently's workspace (projects, snapshots, tags). Added after a rebuild was found to wipe every Evidently snapshot.

A compose-level down -v removes them; a normal down keeps them. User data is therefore safe across routine rebuilds and upgrades as long as the operator does not use -v.

Bind mounts handle user-editable state:

- ../data:/app/data on the noted service - the project directory, documents catalog, skills, environments. All user-facing state lives here.
- ../.noted:/app/.noted - noted's internal config (agents, view customizations).
- ../data:/app/data:ro on noted-graph - read-only access to the same tree.

Airflow mounts its own dag / log / plugin directories from the repo (lines 28-33).

15.4 Auto-generated mount file

noted supports bind-mounting external project directories via YAML frontmatter in data/NOTED.md (Chapter 8.8). The compose file does not include these mounts directly - they are generated on demand.

The pattern:

1. User edits data/NOTED.md to add a mount: mounts: [{name: jena_weather, host_path: /mnt/data/jena_weather}].
2. A helper (scripts/generate_mounts.py or similar) reads the frontmatter and writes data/docker-compose.mounts.yml with the corresponding volumes: section for the noted service.
3. The operator starts the stack with docker compose -f services/docker-compose.yml -f data/docker-compose.mounts.yml up -d.

The comment in docker-compose.yml:59 documents this pattern: # Include with: -f ../data/docker-compose.mounts.yml.

This keeps the compose file itself stable - operators who do not use mounts see a clean file; operators who do get a parallel mounts file generated from their NOTED.md edits.

15.5 Nginx proxy

An nginx reverse proxy fronts the stack in production-like deployments. Its config (services/nginx/nginx.conf, mentioned in Chapter 5.3.2) handles:

- / -> noted backend (port 8123).
- /mlflow/* -> mlflow (port 5000) with static prefix rewriting.
- /airflow/* -> airflow-apiserver (port 8080) with proxy-fix headers.
- /evidently/* -> evidently (port 8000) with SPA base-path rewriting (the sub_filter from Chapter 5.3.2).
- /minio/* -> minio (port 9001 for the web UI).
- /llm/* -> the agent_server (port 7701) for the local LLM.

The proxy is what lets every service present a unified URL space under one origin (e.g. https://logus2k.com/) rather than forcing users to remember 7 different ports.

15.6 Environment variables and .env

The services/.env file holds all the service-level config the compose file references. Template is services/.env.example; operators copy it and fill in their values.

Critical variables:

- NOTED_TERMINAL_SECRET - Gate for terminal and Claude-model LLM access (Chapter 8.9, 11.3).
- ANTHROPIC_API_KEY - Optional; enables Claude backends.
- MINIO_ROOT_USER, MINIO_ROOT_PASSWORD - MinIO admin credentials, used by DVC's .dvc/config as access_key_id / secret_access_key.
- _AIRFLOW_WWW_USER_USERNAME, _AIRFLOW_WWW_USER_PASSWORD - Airflow admin credentials.
- AIRFLOW_UID - UID for the Airflow container's owner (defaults to 50000).
- _PIP_ADDITIONAL_REQUIREMENTS - extra pip installs added to Airflow workers on startup (for project-specific imports).

The compose file references these via ${VAR:-default} syntax. Missing .env falls back to defaults, which works for first-boot but not for anything that needs an API key.

15.7 Service dependencies and startup order

Airflow has a known-complicated startup dance because its metadata DB must be migrated before workers can connect:

1. postgres starts, healthcheck waits for pg_isready.
2. redis starts, healthcheck waits for redis-cli ping.
3. airflow-init runs (DB migrate, create admin user), exits with status 0.
4. airflow-apiserver, airflow-scheduler, airflow-dag-processor, airflow-worker, airflow-triggerer all start, waiting for airflow-init to complete and for postgres/redis to be healthy.

The depends_on + condition: service_healthy / service_completed_successfully blocks in the compose file enforce this order. restart: always on the long-running services handles the case where a service starts slightly too eagerly.

Other services have looser dependencies:

- noted-graph depends on nothing explicitly - it polls MLflow / Airflow / filesystem and degrades if any are unavailable.
- noted-serving depends on nothing - it lazy-loads models on demand.
- evidently is standalone.
- mlflow and minio are standalone.

This keeps the startup graph shallow and avoids tangling non-critical services with critical ones.

15.8 Healthchecks

Every long-running service has a healthcheck. A sample:

- Airflow apiserver: curl --fail http://localhost:8080/api/v2/version
- Airflow scheduler: curl --fail http://localhost:8974/health
- Postgres: pg_isready -U airflow
- Redis: redis-cli ping
- Flower: curl --fail http://localhost:5555/
- Airflow worker: celery inspect ping

noted itself does not define a healthcheck (the container's uvicorn process is trusted). Adding one would be a simple curl --fail http://localhost:8123/api/system/info.

15.9 Operational patterns

Starting the stack

```
cd services/
docker compose up -d
```

Wait ~30-60 seconds for Airflow's init to complete. Then open http://localhost:8123 for noted, http://localhost:5000 for MLflow, http://localhost:8080/airflow for Airflow, etc.

Starting with mounts

```
docker compose -f docker-compose.yml -f ../data/docker-compose.mounts.yml up -d
```

(Assuming data/docker-compose.mounts.yml has been generated from NOTED.md.)

Rebuilding a single service

```
docker compose build noted          # rebuild image
docker compose up -d noted          # recreate with new image
```

For the noted service specifically: because frontend/ and backend/ are COPY'd into the image (not bind-mounted), changes require --build. Only data/ is bind-mounted for live updates.

Attaching to logs

```
docker compose logs -f noted        # single service
docker compose logs -f              # all services
```

Running Airflow CLI

```
docker compose --profile debug run --rm airflow-cli <command>
```

The debug profile is necessary because the CLI container is not started by default.

Stopping and cleanup

```
docker compose down                 # stop, keep volumes (user data preserved)
docker compose down -v              # stop, remove volumes (DESTRUCTIVE)
```

The -v form nukes every named volume. Only run it when you mean “wipe all tracked state”.

Rebuilding after changes to data/

No action required. data/ is bind-mounted into the noted and noted-graph containers, so edits appear live. In-memory caches (e.g. the DocumentManager catalog) may need a container restart (docker compose restart noted) to pick up changes to data/documents/documents.json.

15.10 Resource considerations

Approximate footprint at idle:

- noted - 500 MB - 2 GB RAM depending on loaded kernels.
- mlflow - ~200 MB.
- minio - ~100 MB.
- evidently - ~200 MB.
- noted-graph - ~50 MB (Alpine-based, minimal deps).
- postgres - ~150 MB.
- redis - ~30 MB.
- Airflow stack (5 services) - ~1.5 GB total.
- noted-serving - ~500 MB - 4 GB RAM + VRAM depending on loaded model.

Total idle: ~3-4 GB RAM. Under active training, the serving container plus kernel memory can push this to 8-16 GB depending on model and batch size.

Disk: MinIO + MLflow + Airflow logs + DVC cache can grow to tens of GB over time. The named volumes live under Docker’s managed storage area; docker system df shows usage.

15.11 GPU access

For CUDA-enabled training and serving:

1. Install the NVIDIA Container Toolkit on the host.
2. Add runtime: nvidia and environment: NVIDIA_VISIBLE_DEVICES=all to the services that need GPU (noted and noted-serving).
3. Verify inside the container with nvidia-smi.

The default docker-compose.yml in the repo does not include GPU settings because the compose file should work on CPU-only hosts too. A separate docker-compose.gpu.yml overlay can be included with -f for GPU deployments.

15.12 Discussion-ready talking points

Q: Why Docker Compose instead of Kubernetes? A: Because the target deployment is a single host. Kubernetes would add etcd, an API server, kubelet, CNI, and a learning curve for a deployment that fits on one machine. Compose is the right tool for "I want these N services to run together on this host". Scaling to multi-node would justify Kubernetes; at noted's current scope, it would be a distraction.

Q: Why so many services (13) instead of consolidating? A: Because each service has a well-defined boundary and an upstream image that is maintained independently. Consolidating MLflow + MinIO + Airflow into a single mega-container would mean owning each of their upgrade cycles. Keeping them separate lets the noted project focus on integration code; upstream projects focus on their own code; compose handles the orchestration.

Q: Why is Airflow's Celery stack kept instead of running LocalExecutor? A: Because the DAGs in noted's target use case (Tutorial 3's jena_training_pipeline) are non-trivial in compute and isolating workers from the scheduler reduces crash blast radius. LocalExecutor would simplify the stack but would mean one Python process running both DAG parsing and task execution. For the demo, CeleryExecutor is a bit of overkill; for production it is the correct default.

Q: Why does noted serve frontend static files itself instead of having nginx do it? A: Because in the default single-host deployment there is no need for a separate static-file server. FastAPI's StaticFiles mount is fast enough at noted's traffic volume. In a deployment with nginx, nginx can still proxy /static/* directly to the frontend dir if the operator wants; the noted service would continue to work.

Q: How is horizontal scaling supposed to work? A: Not trivially. The noted backend holds kernel sessions, socket.io rooms, and in-process manager state - none of which are shareable across instances. Scaling out requires extracting kernels to a dedicated service, routing socket.io through Redis, and treating project state as a networked resource (Chapter 8.11). This is future infrastructure work, not a compose file change.

Q: What is the backup story? A: Named volumes can be backed up with docker run --rm -v <volume>:/source -v $(pwd):/dest alpine tar czf /dest/<volume>.tar.gz -C /source .. For a complete snapshot, stop the stack first. DVC-tracked files are already in the MinIO remote and the git repo, so they replay on next dvc pull. The MLflow + Evidently + Airflow + Postgres volumes are what need explicit backup.

Q: What breaks if the host reboots mid-operation? A: restart: always policies bring services back. Running kernels are lost (they are in-process state of the noted container). Any in-flight cell execution is lost; the user has to re-execute after reconnecting. MLflow runs that were mid-stream may be left in RUNNING state in the DB; Airflow tasks that were running on a worker are marked failed and can be retried.

Q: How is secret management handled? A: Via the .env file on disk. This is adequate for single-host local deployments. For production, the operator should replace it with Docker secrets, a secrets manager (Vault, AWS Secrets Manager, etc.), or at minimum an encrypted volume. noted does not force a secret management story; the .env is a sensible default that most small deployments can start with.

Q: Why bind-mount ../data/ instead of copying it into the image? A: Because data/ is user-facing state that changes frequently. Copying it into the image would mean every dataset edit, every new notebook, every document addition requires a rebuild. Bind-mount keeps the state on the host where git can see it and the user can edit it with their normal tools. The cost is that the image is not self-contained - it needs a matching data/ directory at runtime - but for a developer tool that is the right trade.

Q: What is the upgrade path? A: Pull the latest docker-compose.yml + noted image, run docker compose pull && docker compose up -d. Named volumes survive, so MLflow runs, Evidently snapshots, and Airflow history are preserved. If a migration is required (e.g. a new Postgres version), the operator follows the specific migration step listed in the release notes. This is the pattern Airflow itself uses and is well-documented in the Airflow upgrade guides.